

Laboratorio 6 — Análisis de redes sociales: YouTube

Cuaderno autónomo y reproducible con las 10 actividades completas del laboratorio (1. carga e integración, 2. calidad y limpieza, 3. análisis exploratorio, 4. red bipartita autor-video, 5. proyecciones, 6. topología y fragmentación, 7. comunidades, 8. centralidad y puentes, 9. contenido y sentimiento, 10. interpretación y conclusiones). Se trabaja sobre dos conjuntos de datos:

- `data/youtube_videos.csv` — información de videos y canales (293 videos).
- `data/youtube_comments.csv` — comentarios publicados en una selección de videos (406 comentarios).

Principio metodológico central. Los datos no permiten identificar quién respondió a quién. La variable `reply_count` indica *cuántas* respuestas recibió un comentario, pero no identifica a los autores de esas respuestas. Por tanto **`reply_count` nunca se interpreta como una arista entre usuarios.**

Objetivo. Estudiar la estructura de participación de los usuarios, la relación entre canales y temas, y el contenido de las conversaciones. Todas las tablas, métricas y figuras se reconstruyen ejecutando este cuaderno de forma reproducible.

1. Carga, comprensión e integración de los datos

1.1 Carga de los archivos

Se cargan ambos CSV con `pandas`. La ruta es relativa a la carpeta del proyecto para que el cuaderno sea reproducible.

```
In [1]: import unicodedata
import html
import re
import json
from ast import literal_eval
from collections import Counter, defaultdict
from pathlib import Path

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# La figura se muestra en pantalla dentro del notebook
matplotlib.use("module://matplotlib_inline.backend_inline")
sns.set_theme(style="whitegrid", font_scale=0.9)
plt.rcParams["figure.dpi"] = 100

DATA_DIR = Path("data")
VIDEOS_CSV = DATA_DIR / "youtube_videos.csv"
COMMENTS_CSV = DATA_DIR / "youtube_comments.csv"

videos_raw = pd.read_csv(VIDEOS_CSV)
comments_raw = pd.read_csv(COMMENTS_CSV)

print("youtube_videos.csv ->", videos_raw.shape)
print("youtube_comments.csv->", comments_raw.shape)

youtube_videos.csv -> (293, 20)
youtube_comments.csv-> (406, 17)
```

```
In [2]: videos_raw.head()
```

Out[2]:	video_id	title	channel_name	channel_id	source_que
0	-5puKGEqcUc	INSIVUMEH pronostica incremento de lluvias par...	T13 Noticias Guatemala	UCq0Cm-3SKthEySQc2JZBi1A	guatemala lluvi
1	-E7OPOLjMug	BERNARDO ARÉVALO CALIFICA CAMBIO EN EL MP COMO...	IDocumenta	UCgltjn_ZFWwcv1MlpKlkqgQ	@GobiernodelaRepublicadeGuater
2	-KDgIrlzRKo	¡HISTÓRICO! México recupera petróleo robado po...	México Poder	UC-DpoeBbCMOMOH1-j71KPqA	guatemala notici
3	-O4Pa7qS9WQ	Viaje completo Ciudad de Guatemala al IRTA de...	Guate Maya Trip	UCSQM9AVr3rr7_GF8RH5KFGw	guatemala trafi
4	06mFNPU0aB8	Capturan a presuntos delincuentes disfrazados ...	Noti7	UCVpSRoZgngfSL03Nlbjtq9A	guatemala notici

In [3]: `comments_raw.head()`

Out[3]:	video_id	comment_id	video_title	channel_name	channel_id
0	j43HgwYFKfk	Ugw-J65a1iYL9hqhELh4AaABAg	La cooptación de Walter Mazariegos en la USAC	Quorum	UCE4rsXcgDb6e1-a9iTbWzfg
1	06mFNPU0aB8	Ugw-ZT9t9wU2V-tCaUZ4AaABAg	Capturan a presuntos delincuentes disfrazados ...	Noti7	UCVpSRoZgngfSL03Nlbjtq9A
2	j43HgwYFKfk	Ugw0xaOb2CYXXoudtwj4AaABAg	La cooptación de Walter Mazariegos en la USAC	Quorum	UCE4rsXcgDb6e1-a9iTbWzfg
3	j43HgwYFKfk	Ugw0xgUc2ISpBr5_T654AaABAg	La cooptación de Walter Mazariegos en la USAC	Quorum	UCE4rsXcgDb6e1-a9iTbWzfg
4	OkXIHX0hx-8	Ugw1ZzA21njWhaqQQTh4AaABAg	EE.UU. envía a mexicanos deportados a Guatemal...	Noticias Telemundo	UCRwA1NUcUnwsly35ikGhp0A

1.2 Unidad de observación, llave primaria y variables relevantes

Archivo	Unidad de observación	Llave primaria	Llave foránea	Variables relevantes
				channel_id title view count

youtube_videos.csv	Un video	video_id	—	channel_id, title, view_count, publish_date, category, source_query, source_group, keywords
youtube_comments.csv	Un comentario principal	comment_id	video_id	author channel id, text, like count text, reply count, source_query, source_group

- `video_id` identifica de forma única a cada video y relaciona ambos archivos.
- `comment_id` identifica de forma única a cada comentario principal.
- `channel_id` identifica de forma única a cada canal (es preferible a `channel_name`, que puede cambiarse o repetirse).
- `author_channel_id` identifica de forma única a la cuenta que publicó el comentario (distinto del canal del video comentado).

```
In [4]: def identificar_estructura(df, nombre, llave):
        print(f"{nombre}: {df.shape[0]:,} filas x {df.shape[1]} columnas")
        print(f"  Llave primaria      : {llave}")
        print(f"  Duplicados de llave: {int(df[llave].duplicated().sum())}")
        print(f"  Valores unicos      : {df[llave].nunique():,}")
        print("  Variables:", ", ".join(df.columns))

        identificar_estructura(videos_raw, "youtube_videos.csv", "video_id")
        print()
        identificar_estructura(comments_raw, "youtube_comments.csv", "comment_id")

youtube_videos.csv: 293 filas x 20 columnas
  Llave primaria      : video_id
  Duplicados de llave: 0
  Valores unicos      : 293
  Variables: video_id, title, channel_name, channel_id, source_query, source_group, dataset_sources,
channel_handle, published_time, view_count_text, description_snippet, video_url, query_hits, keywords,
description, view_count, publish_date, upload_date, category, owner_handle

youtube_comments.csv: 406 filas x 17 columnas
  Llave primaria      : comment_id
  Duplicados de llave: 0
  Valores unicos      : 406
  Variables: video_id, comment_id, video_title, channel_name, channel_id, author_name,
author_channel_id, text, source_query, source_group, dataset_sources, author_handle, published_text,
like_count_text, reply_count, is_pinned, viewer_rating
```

1.3 Relación entre canal, video, autor, comentario, categoría y consulta

- **Canal-video:** un `channel_id` puede publicar varios videos; cada video pertenece a un único canal.
- **Video-comentario:** `video_id` es llave primaria en `videos` y llave foránea en `comments`. Un video puede tener muchos comentarios.
- **Autor-comentario:** `author_channel_id` identifica la cuenta que publicó el comentario. **No** es el `channel_id` del canal comentado.
- **Categoría:** variable que describe la categoría asignada al video (p. ej. *News & Politics*). Se hereda al comentario únicamente a través de la unión por `video_id`.
- **Consulta de búsqueda:** `source_query` y `source_group` describen cómo se recolectó el video/comentario (procedimiento de muestreo). No implican que ese sea el tema definitivo del contenido.

```
In [5]: # Comprobacion rapida de la relacion autores vs canales
        # Un mismo autor_channel_id puede NO coincidir con channel_id del video que comenta
        ejemplo = comments_raw[["comment_id", "channel_id", "author_channel_id"]].head(3)
        print(ejemplo.to_string(index=False))
        print()
        print("Cuentas unicas:")
        print("  canales de video      :", comments_raw["channel_id"].nunique())
        print("  autores                :", comments_raw["author_channel_id"].nunique())
```

	comment_id	channel_id	author_channel_id
Ugw-J65a1iYL9hqhELh4AaABAg	UCE4rsXcgDb6e1-a9iTbWzfg	UCdFlugHJJJa4l3YqWuNRmvXw	
Ugw-ZT9t9wU2V-tCaUZ4AaABAg	UCVpSRoZgngfSL03Nlbjtq9A	UCv11tzQeBeGy6efPTRJXSCw	
Ugw0xa0b2CYXXoudtwJ4AaABAg	UCE4rsXcgDb6e1-a9iTbWzfg	UCRAquv8el-tQ30bN7MlmySQ	

Cuentas unicas:

canales de video	: 8
autores	: 332

1.4 Integración mediante video_id

Se aplica una unión **muchos-a-uno** desde `comments` hacia `videos` con `video_id`. Se reporta cuántos comentarios pudieron asociarse con un video.

```
In [6]: # Referencia de videos con la informacion que interesa propagar a los comentarios
video_reference = videos_raw[[
    "video_id", "title", "channel_id", "channel_name", "channel_handle",
    "view_count", "publish_date", "category", "source_query", "source_group", "video_url"
]].copy()

integrated = comments_raw.merge(
    video_reference,
    on="video_id",
    how="left",
    validate="many_to_one",
    indicator=True,
    suffixes=("_comentario", "_video"),
)

matched = int((integrated["_merge"] == "both").sum())
integrated["asociado_con_video"] = integrated["_merge"].eq("both")
integrated = integrated.drop(columns=["_merge"])

print(f"Comentarios asociados a un video: {matched}, de {len(comments_raw):,}")
print(f"Porcentaje: {100 * matched / len(comments_raw):.1f}%")
print(f"Sin coincidencia: {len(comments_raw) - matched}")

# Videos que reciben al menos un comentario en el archivo
print(f"Videos con comentarios recolectados: {comments_raw['video_id'].nunique()}")
```

Comentarios asociados a un video: 406 de 406
 Porcentaje: 100.0%
 Sin coincidencia: 0
 Videos con comentarios recolectados: 19

2. Calidad, limpieza y preprocesamiento

En esta sección se trabaja sobre copias para no mutar los originales. Se conservan los datos sin procesar y se construyen versiones normalizadas.

```
In [7]: videos = videos_raw.copy()
comments = comments_raw.copy()

print("Copias creadas. Originales intactos.")
```

Copias creadas. Originales intactos.

2.1 Diagnóstico inicial de calidad

Se revisan: dimensiones, tipos de variables, valores faltantes, duplicados, variables constantes, valores atípicos y consistencia entre identificadores/nombres/handles.

```
In [8]: def diagnosticar(df, nombre, llave):
    # Duplicados, faltantes y constantes
    duplicados_total = int(df.duplicated().sum())
    constantes = []
    for col in df.columns:
        s = df[col]
        try:
            n = s.nunique(dropna=False)
        except TypeError:
```

```

n = s.map(lambda v: json.dumps(v, ensure_ascii=False, sort_keys=True)
          if isinstance(v, (list, dict)) else str(v)).nunique(dropna=False)
if n <= 1:
    constantes.append(col)
print("=" * 70)
print(f"{{nombre}}")
print("=" * 70)
print(f"  Filas          : {df.shape[0]:,}")
print(f"  Columnas         : {df.shape[1]:,}")
print(f"  Duplicados (fila): {duplicados_total}")
print(f"  Duplicados llave : {int(df[llave].duplicated().sum())}")
print(f"  Celdas faltantes : {int(df.isna().sum().sum())}")
print(f"  Constantes       : {'', '.join(constantes) if constantes else 'Ninguna'}")

diagnosticar(videos_raw, "youtube_videos.csv", "video_id")
diagnosticar(comments_raw, "youtube_comments.csv", "comment_id")

```

=====

youtube_videos.csv

=====

```

Filas          : 293
Columnas       : 20
Duplicados (fila): 0
Duplicados llave : 0
Celdas faltantes : 77
Constantes      : Ninguna

```

=====

youtube_comments.csv

=====

```

Filas          : 406
Columnas       : 17
Duplicados (fila): 0
Duplicados llave : 0
Celdas faltantes : 406
Constantes      : is_pinned, viewer_rating

```

In [9]:

```

print("FALTANTES EN youtube_videos.csv")
print(videos_raw.isna().sum()[videos_raw.isna().sum() > 0].to_string() if (videos_raw.isna().sum() > 0).any() else "Ninguna columna con NaN.")

```

FALTANTES EN youtube_videos.csv

```

published time    13
view_count_text   13
description_snippet 25
description        26

```

In [10]:

```

print("FALTANTES EN youtube_comments.csv")
print(comments_raw.isna().sum().to_string())
print()
print("Blancos como texto vacio (por si no son NaN):")
blancos = comments_raw.astype("string").apply(lambda s: s.str.strip().eq("").sum())
print(blancos[blancos > 0].to_string() if (blancos > 0).any() else "Ninguno.")

```

FALTANTES EN youtube_comments.csv

```

video_id          0
comment_id        0
video_title       0
channel_name      0
channel_id        0
author_name       0
author_channel_id 0
text              0
source_query      0
source_group      0
dataset_sources   0
author_handle     0
published_text    0
like_count_text   0
reply_count       0
is_pinned         0
viewer_rating     406

```

Blancos como texto vacio (por si no son NaN):

```

like_count_text    189

```

In [11]:

```

# Tipos observados
print("TIPOS - youtube_videos.csv")
print(pd.DataFrame({"variable": videos_raw.columns, "tipo":

```

```
videos_raw.dtypes.astype(str).values}).to_string(index=False))
```

```
TIP0S - youtube_videos.csv
      variable  tipo
      video_id  str
      title     str
      channel_name str
      channel_id str
      source_query str
      source_group str
      dataset_sources str
      channel_handle str
      published_time str
      view_count_text str
      description_snippet str
      video_url      str
      query_hits     str
      keywords       str
      description    str
      view_count     int64
      publish_date   str
      upload_date    str
      category       str
      owner_handle   str
```

```
In [12]: print("TIP0S - youtube_comments.csv")
print(pd.DataFrame({"variable": comments_raw.columns, "tipo":
comments_raw.dtypes.astype(str).values}).to_string(index=False))
```

```
TIP0S - youtube_comments.csv
      variable  tipo
      video_id  str
      comment_id str
      video_title str
      channel_name str
      channel_id str
      author_name str
      author_channel_id str
      text        str
      source_query str
      source_group str
      dataset_sources str
      author_handle str
      published_text str
      like_count_text str
      reply_count   int64
      is_pinned     bool
      viewer_rating float64
```

```
In [13]: def iqr_atipicos(series):
num = pd.to_numeric(series, errors="coerce").dropna()
if num.empty:
    return 0
q1, q3 = num.quantile([0.25, 0.75])
iqr = q3 - q1
if iqr == 0:
    return int(((num < q1) | (num > q3)).sum())
return int(((num < q1 - 1.5 * iqr) | (num > q3 + 1.5 * iqr)).sum())

print("VALORES ATIPICOS (metodo IQR)")
print("  videos.view_count   :", iqr_atipicos(videos_raw["view_count"]))
print("  comments.reply_count:", iqr_atipicos(comments_raw["reply_count"]))
print("  comments.like_count_text:", "(ver seccion 2.4, aun en texto)")
```

```
VALORES ATIPICOS (metodo IQR)
videos.view_count   : 49
comments.reply_count: 30
comments.like_count_text: (ver seccion 2.4, aun en texto)
```

```
In [14]: # Consistencia entre identificadores, nombres y handles
print("CONSISTENCIA DE IDS / NOMBRES / HANDLES")
print("  video_id duplicado en videos           :", int(videos_raw["video_id"].duplicated().sum()))
print("  comment_id duplicado                    :",
int(comments_raw["comment_id"].duplicated().sum()))
print("  channel_id con varios channel_name      :", int((videos_raw.groupby("channel_id")
["channel_name"].nunique() > 1).sum()))
print("  author_channel_id con varios nombres :", int((comments_raw.groupby("author_channel_id")
["author_name"].nunique() > 1).sum()))
```

```
print(" upload_date != publish_date      :", int((videos_raw["upload_date"] !=
videos_raw["publish_date"]).sum()))
print(" owner_handle != channel_handle   :", int((videos_raw["owner_handle"] !=
videos_raw["channel_handle"]).sum()))
```

CONSISTENCIA DE IDS / NOMBRES / HANDLES

```
video_id duplicado en videos      : 0
comment_id duplicado              : 0
channel_id con varios channel_name : 0
author_channel_id con varios nombres : 0
upload_date != publish_date      : 0
owner_handle != channel_handle    : 0
```

2.2 Variables que no pueden usarse o requieren precaución

Variable	Tratamiento / justificación
viewer_rating	Se excluye por estar vacía en los 406 registros; no aporta información.
is_pinned	Constante False ; no discrimina registros. Se conserva para auditoría pero no se usa.
published time / published_text	Son tiempos relativos al momento de recolección; no se convierten en fechas exactas.
view count text / like_count_text	Conteos en formato de texto (p. ej. 2,390 vistas); requieren limpieza (sección 2.4). Para videos se usa view_count ya numérico.
source_query / source_group	Describen la cobertura de recolección , no el tema definitivo del contenido.
reply_count	Indica cuántas respuestas recibió el comentario, pero no identifica a los autores de esas respuestas . Nunca se usa para crear aristas entre usuarios.
Conteos observados	Son fotografías del momento de recolección; no directamente comparables sin considerar antigüedad y cobertura.

2.3 Normalización de identificadores y nombres

Se normalizan los **identificadores** con Unicode NFKC y recorte de espacios, manteniendo video_id , comment_id , channel_id y author_channel_id . Los **nombres y handles** se limpian solo para presentación (etiquetas), sin sustituir los IDs.

```
In [15]: def normalize_identifier(value):
        if pd.isna(value):
            return pd.NA
        limpio = unicodedata.normalize("NFKC", str(value)).strip()
        return limpio if limpio else pd.NA

def normalize_display_name(value):
    if pd.isna(value):
        return pd.NA
    limpio = unicodedata.normalize("NFKC", html.unescape(str(value)))
    limpio = re.sub(r"\s+", " ", limpio).strip()
    return limpio if limpio else pd.NA

# Identificadores
for col in ("video_id", "channel_id"):
    videos[col] = videos[col].map(normalize_identifier)
for col in ("video_id", "comment_id", "channel_id", "author_channel_id"):
    comments[col] = comments[col].map(normalize_identifier)

# Nombres / handles (solo presentacion)
for col in ("title", "channel_name", "channel_handle", "owner_handle"):
    videos[col] = videos[col].map(normalize_display_name)
for col in ("video_title", "channel_name", "author_name", "author_handle"):
    comments[col] = comments[col].map(normalize_display_name)

print("IDs y nombres normalizados.")
```

IDs y nombres normalizados.

```
In [16]: # Comprobacion: video_id sigue siendo unico y consistente
print("video_id duplicados tras limpieza :", int(videos["video_id"].duplicated().sum()))
print("comment_id duplicados tras limpieza:", int(comments["comment_id"].duplicated().sum()))
print("channel_id NaN en videos:", int(videos["channel_id"].isna().sum()))
print("author_channel_id NaN en comments:", int(comments["author_channel_id"].isna().sum()))
```

```
video_id duplicados tras limpieza : 0
comment_id duplicados tras limpieza: 0
channel_id NaN en videos: 0
author_channel_id NaN en comments: 0
```

2.4 Conversión a numérico de las variables de conteo en texto

Se definen un conversor que reconoce separadores de miles y sufijos `K`, `M`, `B`, `mil` y `millones`, y una bandera de validez.

- `view_count_text` se compara con `view_count` (ya numérico) para auditar el conversor.
- `like_count_text` los espacios en blanco/vacíos se interpretan como `0` (ausencia de un conteo visible) y quedan documentados con la bandera de validez.

```
In [17]: def parse_count(value):
        """Convierte conteos como '2,390 vistas', '1.2 K' o blancos a enteros."""
        if pd.isna(value):
            return 0, True
        raw = unicodedata.normalize("NFKC", str(value)).strip().lower()
        if raw in {"", "-", "--", "nan", "none"}:
            return 0, True
        raw = raw.replace("vistas", "").replace("vista", "").replace("likes", "").strip()
        match = re.search(r"([+]?[0-9]+(?:[.]\d+)?)\s*([kmb]|mil|millones)?", raw)
        if not match:
            return 0, False
        number_text, sufijo = match.groups()
        sufijo = sufijo or ""
        if sufijo:
            number_text = number_text.replace(",", ".")
            mult = {"k": 1_000, "mil": 1_000, "m": 1_000_000, "millon": 1_000_000,
                    "millones": 1_000_000, "b": 1_000_000_000}.get(sufijo, 1)
            return int(round(float(number_text) * mult)), True
        digitos = re.sub(r"[^0-9+\.]", "", number_text)
        try:
            return int(digitos), True
        except ValueError:
            return 0, False

# --- videos: view_count_text vs view_count ---
parsed_views = videos["view_count_text"].map(parse_count)
videos["view_count_text_numerico"] = parsed_views.map(lambda x: x[0])
videos["view_count_text_valido"] = parsed_views.map(lambda x: x[1])
videos["view_count"] = pd.to_numeric(videos["view_count"],
errors="coerce").fillna(0).astype("int64")

diff = int((videos["view_count_text_numerico"] != videos["view_count"]).sum())
print(f"Casos donde el conversor difiere de view_count: {diff}")
print("Ejemplo de parseo:")
print(videos[["view_count_text", "view_count_text_numerico",
"view_count"]].head(5).to_string(index=False))
```

Casos donde el conversor difiere de view_count: 67

Ejemplo de parseo:

view_count_text	view_count_text_numerico	view_count
2,390 vistas	2390	2357
4 vistas	4	4
29,736 vistas	29736	29736
383 vistas	383	383
6,692 vistas	6692	6692

```
In [18]: # --- comments: like_count_text ---
        parsed_likes = comments["like_count_text"].map(parse_count)
        comments["like_count"] = parsed_likes.map(lambda x: x[0]).astype("int64")
        comments["like_count_valido"] = parsed_likes.map(lambda x: x[1])

# reply_count ya es numerico en el archivo, pero se limpia por seguridad
comments["reply_count"] = pd.to_numeric(comments["reply_count"],
errors="coerce").fillna(0).astype("int64")

print("Conteos like no validos (no parseables):", int((~comments["like_count_valido"]).sum()))
print("Valores de like_count_text invalidos:")
print(comments.loc[~comments["like_count_valido"], ["like_count_text",
"like_count"]].to_string(index=False))
```



```

Conteos like no validos (no parseables): 0
Valores de like_count_text invalidos:
Empty DataFrame
Columns: [like_count_text, like_count]
Index: []

```

Documentación de separadores y abreviaturas: el conversor trata la coma/espacio como separador de miles (2,390 → 2390) y el punto como separador decimal cuando hay sufijo (1.2 K → 1200). Los sufijos K / mil → ×1000, M / millón / millones → ×1 000 000, B → ×1 000 000 000. Los valores vacíos en una variable de "me gusta" visible se codifican como 0 y se marcan para auditoría.

2.5 y 2.6 Versiones del texto y decisiones de limpieza

- **texto_original** : conserva el comentario exacto para auditoría y análisis de sentimiento posterior.
- **texto_limpio** : se deriva de forma reproducible con estas decisiones:

Paso	Decisión
Minúsculas	<code>.casefold()</code> (Unicode)
URL	se eliminan (<code>https?://...</code> , <code>www.</code>)
Hashtags	se separan: se extrae el término del <code>#</code> (se conserva como palabra)
Menciones	se eliminan (patrones <code>@usuario</code>)
Puntuación	se reemplaza por espacio
Números	se eliminan (solo se conservan letras con tildes y ñ)
Stopwords	se eliminan las de español (lista manual)
Lematización	no se aplica : un modelo lingüístico externo añadiría descargas y una versión distinta; se prioriza reproducibilidad local y se documenta la decisión
Emojis	se eliminan (no son letras ni espacios según categoría Unicode)
Mínimo	tokens de ≥ 3 caracteres

```

In [19]: STOPWORDS_ES = {
    "a", "aca", "ahi", "al", "algo", "algun", "alguna", "algunas", "alguno", "algunos",
    "alla", "alli", "ante", "antes", "aquel", "aquella", "aquellas", "aquello", "aquellos",
    "aqui", "asi", "aun", "aunque", "bajo", "bien", "cada", "casi", "como", "con", "contra",
    "cual", "cuando", "de", "del", "desde", "donde", "dos", "durante", "e", "el", "ella",
    "ellas", "ello", "ellos", "en", "entre", "era", "eramos", "eran", "eras", "eres", "es",
    "esa", "esas", "ese", "eso", "esos", "esta", "estaba", "estaban", "estado", "estamos",
    "estan", "estar", "estas", "este", "esto", "estos", "fue", "fuera", "fueron", "ha", "hace",
    "hacia", "han", "hasta", "hay", "la", "las", "le", "les", "lo", "los", "mas", "me",
    "mi", "mis", "mismo", "mucho", "muy", "nada", "ni", "no", "nos", "nosotros", "nuestra",
    "nuestro", "o", "otra", "otras", "otro", "otros", "para", "pero", "poco", "por", "porque",
    "que", "quien", "se", "sea", "ser", "si", "sin", "sobre", "son", "su", "sus", "tambien",
    "te", "tener", "tiene", "tienen", "todo", "todos", "tu", "tus", "un", "una", "uno", "unos",
    "usted", "ustedes", "ya", "yo", "video", "videos", "youtube", "canal", "suscribete", "gracias",
    "más", "solo", "sólo",
}

def clean_text(value):
    """Devuelve el texto limpio a partir de un valor (str/NaN)."""
    if pd.isna(value):
        return ""
    texto = unicodedata.normalize("NFKC", html.unescape(str(value))).casefold()
    texto = re.sub(r"https?://\S+|www\.\S+", " ", texto) # URL
    texto = re.sub(r"#([\wÁÉÍÓÚÑáéíóúñ]+)", r"\1 ", texto) # hashtag -> palabra
    texto = re.sub(r"(?:^|\s)/@[\w.%-]+", " ", texto) # menciones
    texto = "".join(ch if unicodedata.category(ch)[0] in {"L", "Z"} else " " for ch in texto) #
    punt/numeros/emojis
    tokens = re.findall(r"[a-záéíóúñ]+", texto)
    tokens = [t for t in tokens if len(t) >= 3 and t not in STOPWORDS_ES]
    return " ".join(tokens)

comments["texto_original"] = comments["text"].astype("string").fillna("")
comments["texto_limpio"] = comments["texto_original"].map(clean_text)

print("Ejemplo de limpieza:")
muestra = comments[["text", "texto_original", "texto_limpio"]].iloc[:3]
for _, r in muestra.iterrows():

```

```
print("ORIG -", r["texto_original"][:80])
print("LIMP -", r["texto_limpio"][:80])
print()
```

Ejemplo de limpieza:

ORIG - Ese corrupto amigo de la vieja fiscal los tengo que verbose en la carcel

LIMP - corrupto amigo vieja fiscal tengo verbose carcel

ORIG - Están jóvenes porque no buscan un trabajo, tuvieron suerte que no hay policías

LIMP - están jóvenes buscan trabajo tuvieron suerte policías gusta vando vestido mujer

ORIG - Me dejaron con ganas de demandar la ilegalidad de las reuniones virtuales, esto

LIMP - dejaron ganas demandar ilegalidad reuniones virtuales maquila

2.7 Cuantificación del efecto de la limpieza

Se cuantifican registros modificados, textos vacíos y duplicados antes y después de la limpieza.

```
In [20]: efecto_limpieza = pd.DataFrame([
    {"metrica": "Registros originales", "valor": len(comments)},
    {"metrica": "Registros eliminados", "valor": 0},
    {"metrica": "Textos modificados", "valor": int((comments["texto_original"].str.strip() !=
comments["texto_limpio"]).sum())},
    {"metrica": "Textos vacíos antes", "valor":
int(comments["texto_original"].str.strip().eq("").sum())},
    {"metrica": "Textos vacíos después", "valor":
int(comments["texto_limpio"].str.strip().eq("").sum())},
    {"metrica": "Textos duplicados antes", "valor":
int(comments["texto_original"].duplicated().sum())},
    {"metrica": "Textos duplicados después", "valor":
int(comments["texto_limpio"].duplicated().sum())},
    {"metrica": "Conteos like no válidos", "valor": int((~comments["like_count_valido"]).sum())},
])
print(efecto_limpieza.to_string(index=False))
```

	metrica	valor
	Registros originales	406
	Registros eliminados	0
	Textos modificados	403
	Textos vacíos antes	0
	Textos vacíos después	6
	Textos duplicados antes	2
	Textos duplicados después	11
	Conteos like no válidos	0

3. Análisis exploratorio

3.1 Descripción de los elementos mínimos

```
In [21]: import pandas as pd

resumen = pd.DataFrame([
    {"metrica": "Videos", "valor": len(videos)},
    {"metrica": "Canales de videos", "valor": videos["channel_id"].nunique()},
    {"metrica": "Comentarios", "valor": len(comments)},
    {"metrica": "Autores de comentarios", "valor": comments["author_channel_id"].nunique()},
    {"metrica": "Videos con comentarios recolectados", "valor": comments["video_id"].nunique()},
    {"metrica": "Canales con comentarios recolectados", "valor": comments["channel_id"].nunique()},
    {"metrica": "Comentarios asociados a video", "valor": matched},
    {"metrica": "Me gusta acumulados en comentarios", "valor": int(comments["like_count"].sum())},
    {"metrica": "Respuestas acumuladas", "valor": int(comments["reply_count"].sum())},
])
print(resumen.to_string(index=False))
```

	metrica	valor
	Videos	293
	Canales de videos	97
	Comentarios	406
	Autores de comentarios	332
	Videos con comentarios recolectados	19
	Canales con comentarios recolectados	8
	Comentarios asociados a video	406
	Me gusta acumulados en comentarios	2325
	Respuestas acumuladas	51

```
In [22]: # Videos por canal
videos_por_canal = videos.groupby(["channel_id", "channel_name"], as_index=False).agg(
    videos=("video_id", "nunique"), visualizaciones=("view_count", "sum")
).sort_values("videos", ascending=False)
print("VIDEOS POR CANAL (top 10)")
print(videos_por_canal.head(10).to_string(index=False))
```

VIDEOS POR CANAL (top 10)			
channel_id	channel_name	videos	visualizaciones
UC-ZtFBHzgkYFh7whZkuR_Ag	Gobierno de la República de Guatemala	32	56052
UCXa6LVjBzUtmM3cwb9e2w	Municipalidad de Guatemala	27	452548
UCE4rsXcgDb6e1-a91TbWzfg	Quorum	18	31267
UCEp0Y2svuxbsJ-zsAPT9Fg	PrensaLibreOficial	17	424888
UC8u2nvyHewt-mjdWSj0HJmQ	Ministerio de Gobernación	14	141661
UCAivrAhro2BpsJdrPe-IyRg	Diario de Centro América	11	4769
UCAutgrPkiLhZxTB-Ujje1fw	CIV -Ministerio de Comunicaciones	11	5881
UCW_PHqwqCCalvQGOEsVU0jw	Concejales de Oposición - Muni Guate	10	2106
UCHsYnV67K3J0yJ4QeciVCww	PNCdeGuatemala	10	329798
UC6SA1dVHvAkc6yNS_t5cbGw	Conred	8	1693

```
In [23]: # Comentarios y autores unicos por video
comentarios_por_video = comments.groupby("video_id", as_index=False).agg(
    comentarios=("comment_id", "size"),
    autores_unicos=("author_channel_id", "nunique"),
    me_gusta=("like_count", "sum"),
    respuestas=("reply_count", "sum"),
)
tmp = comentarios_por_video.merge(
    videos[["video_id", "title", "channel_name", "view_count", "category"]],
    on="video_id", how="left"
).sort_values("comentarios", ascending=False)
print("TOP 10 VIDEOS POR PARTICIPACION")
print(tmp.head(10).to_string(index=False))
```

TOP 10 VIDEOS POR PARTICIPACION					
video_id	comentarios	autores_unicos	me_gusta	respuestas	
title		channel_name	view_count		category
n8iP75gIpmw	161	128	358	3	
Qué rico come tu diputado				Quorum	11775 News & Politics
j43HgwYFKfk	50	49	91	0	
La cooptación de Walter Mazariegos en la USAC				Quorum	10156 News & Politics
6W4u8sGEnGM	45	32	90	11	
Inician los trabajos de recuperación del Puente Belice II. Gobierno de la República de Guatemala					
11670 Entertainment					
lj983NWyAQY	25	25	1697	16	
Plan 2032 Ciudad de Guatemala		Municipalidad de Guatemala			304089 News & Politics
PjmxCj-a9Hg	25	19	20	4	
Conferencia de Prensa del Gobierno de Guatemala. #LaRondaGt		Gobierno de la República de Guatemala			
3628 Entertainment					
OkXLHx0hx-8	25	18	25	13	EE.UU. envía a mexicanos deportados a
Guatemala antes de su regreso a México Noticias Telemundo					Noticias Telemundo
14200 News & Politics					
yLZS3JiEBg8	16	16	13	0	
Arroz con pollo a la MONOPOLIO				Quorum	1954 News & Politics
06mFNPu0aB8	14	13	12	1	Capturan a presuntos
delincuentes disfrazados de mujer señalados de cometer asalto					Noti7
6692 News & Politics					
ndAZjHqzzT8	12	10	3	1	
Internet: escoger el menos malo				Quorum	717 News & Politics
is3Mk5oC19g	7	7	0	1	Capturan a ladrón que había
quedado grabado mientras robaba en una parroquia de Retalhuleu					TN23 Guatemala
4616 News & Politics					

```
In [24]: # Comentarios por canal
comentarios_por_canal = comments.groupby(["channel_id", "channel_name"], as_index=False).agg(
    comentarios=("comment_id", "size"),
```

```

    autores_unicos=("author_channel_id", "nunique")
).sort_values("comentarios", ascending=False)
print("COMENTARIOS POR CANAL (top 10)")
print(comentarios_por_canal.head(10).to_string(index=False))

```

COMENTARIOS POR CANAL (top 10)

channel_id	channel_name	comentarios	autores_unicos
UCE4rsXcgDb6e1-a9iTbWzfg	Quorum	256	214
UC-ZtFBHzgkYFh7whZkuR_Ag	Gobierno de la República de Guatemala	70	50
UCRwA1NUcUnwslY35ikGhp0A	Noticias Telemundo	25	18
UCXa6LVjBzUtmC3cwbd9e2w	Municipalidad de Guatemala	25	25
UCVpSRoZgngfSL03Nlbjtq9A	Noti7	14	13
UCEepOY2svuxbsJ-zsAPT9Fg	PrensaLibreOficial	7	7
UCt5Dw7ePnn0AqPqRF8Xpjkw	TN23 Guatemala	7	7
UCdSVqHGHzyMTXSafUHY-e-w	Noticiero Guatevisión 13 Hrs.	2	2

In [25]: # Autores

```

autores = comments.groupby("author_channel_id", as_index=False).agg(
    author_name=("author_name", lambda s: s.dropna().mode().iloc[0] if not s.dropna().empty else
    ""),
    author_handle=("author_handle", lambda s: s.dropna().mode().iloc[0] if not s.dropna().empty else
    ""),
    comentarios=("comment_id", "size"),
    videos_unicos=("video_id", "nunique"),
    canales_unicos=("channel_id", "nunique"),
    me_gusta_recibidos=("like_count", "sum"),
    respuestas_recibidas=("reply_count", "sum"),
).sort_values(["videos_unicos", "canales_unicos", "comentarios"], ascending=False)
print("AUTORES (top 10 por videos unicos)")
print(autores.head(10).to_string(index=False))

```

AUTORES (top 10 por videos unicos)

author_channel_id	author_name	author_handle	comentarios	videos_unicos
canales_unicos me_gusta_recibidos respuestas_recibidas				
UCvcu1kR8xMYy_I1Ty-2mV2Q	@hashojea7348	/@hashojea7348	4	3
1 0	0			
UCpsK0kt5iWTbenzeuq7dsmQ	@inge_vergueta	/@inge_vergueta	3	3
1 4	0			
UCHTGCgY2l-DQJIVlA_cpa_Q	@virgiliogarcia3039	/@virgiliogarcia3039	3	2
2 14	0			
UCdFlugHJJJa4l3YqWuNRmvXw	@MarcosCarillo-b1r	/@MarcosCarillo-b1r	2	2
2 0	1			
UCjZgFEowMBRsjodqfovzdKw	@franciscoflores3120	/@franciscoflores3120	2	2
2 0	0			
UCsUCN0Yq_UyTvKj0JLmNrnQ	@moisesvaldez4043	/@moisesvaldez4043	2	2
2 0	0			
UCylqlpsh8ENNM1LqgQ1KbxA	@Jel.Awesh.M	/@Jel.Awesh.M	3	2
1 1	0			
UCbrtvygfrT6QXqWDNR0f-cA	@Alejandro00710	/@Alejandro00710	2	2
1 0	0			
UCzZ6dDCEsLMXbc5pCFXIprw	@josegil3813	/@josegil3813	2	2
1 0	0			
UCi2KiZq63sRq8Mfbcp-MeIQ	@eugeneramirez4405	/@eugeneramirez4405	6	1
1 1	0			

In [26]: # Visualizaciones

```

print("VISUALIZACIONES (view_count)")
print(" min      :", videos["view_count"].min())
print(" mediana  :", videos["view_count"].median())
print(" media    :", round(videos["view_count"].mean()))
print(" max      :", videos["view_count"].max())

# Respuestas y me gusta por comentario
print()
print("RESPUESTAS (reply_count por comentario):")
print(" comentarios con >=1 respuesta:", int((comments["reply_count"] > 0).sum()))
print(" total respuestas:", int(comments["reply_count"].sum()))
print()
print("ME GUSTA (like_count por comentario):")
print(" comentarios con >=1 like:", int((comments["like_count"] > 0).sum()))
print(" total likes:", int(comments["like_count"].sum()))

```

VISUALIZACIONES (view_count)

min : 2
mediana : 1175.0
media : 60430
max : 8190449

RESPUESTAS (reply_count por comentario):

comentarios con >=1 respuesta: 30
total respuestas: 51

ME GUSTA (like_count por comentario):

comentarios con >=1 like: 217
total likes: 2325

```
In [27]: # Categorías
categorías = videos.groupby("category", as_index=False).agg(
    videos=("video_id", "nunique"), visualizaciones=("view_count", "sum")
).sort_values("videos", ascending=False)
print("CATEGORIAS (videos)")
print(categorías.to_string(index=False))
```

CATEGORIAS (videos)

	category	videos	visualizaciones
	News & Politics	138	2165695
	People & Blogs	66	5116257
	Entertainment	48	558349
	Education	19	873680
	Travel & Events	8	660990
	Science & Technology	6	128772
	Nonprofits & Activism	3	602
	Film & Animation	2	2849
	Comedy	1	8190449
	Music	1	5262
	Sports	1	3110

```
In [28]: # Consultas de búsqueda (videos y comentarios)
print("CONSULTAS - VIDEOS (top 10)")
q_videos = videos.groupby(["source_group", "source_query"], as_index=False).agg(
    videos=("video_id", "nunique"), visualizaciones=("view_count", "sum")
).sort_values("videos", ascending=False)
print(q_videos.head(10).to_string(index=False))
print()
print("CONSULTAS - COMENTARIOS (top 10)")
q_comments = comments.groupby(["source_group", "source_query"], as_index=False).agg(
    comentarios=("comment_id", "size"), autores_unicos=("author_channel_id", "nunique")
).sort_values("comentarios", ascending=False)
print(q_comments.head(10).to_string(index=False))
```

CONSULTAS - VIDEOS (top 10)

	source_group	source_query	videos	visualizaciones
	topic	@GobiernodelaRepublicadeGuatemala	18	93188
	topic	@GobiernodeGuatemala	17	21097
	official_gov	Ministerio de Comunicaciones Guatemala	16	4674
	topic	guatemala noticias	16	763121
	topic	@MunicipalidaddeGuatemala-1551	16	5098
	official_gov	PMT Guatemala	15	233617
	official_gov	Conred Guatemala	15	6184
	topic	Gobierno de la República de Guatemala	15	387403
	official_gov	Policia Nacional Civil Guatemala	13	462936
	topic	guatemala transporte	13	13100493

CONSULTAS - COMENTARIOS (top 10)

	source_group	source_query	comentarios	autores_unicos
	channel	@quorumgt/videos	231	192
	topic	guatemala noticias	55	47
	topic	Gobierno de la República de Guatemala	45	32
	topic	@GobiernodeGuatemala	25	19
	topic	@quorumgt	25	24
	topic	Municipalidad de Guatemala	25	25

```
In [29]: # Hashtags (desde titulo, descripcion y texto de comentarios)
def extraer_hashtags(value):
    if pd.isna(value):
        return []
    return re.findall(r"(?<!\w)#[A-Za-z0-9_ÁÉÍÓÚÚÑáéíóúñ]+)", str(value))

todos_hashtags = []
for serie in (videos["title"], videos["description"], comments["texto_original"]):
```

```

for val in serie:
    todos_hashtags.extend([t.casefold() for t in extraer_hashtags(val)])

hashtags = pd.DataFrame(Counter(todos_hashtags).most_common(), columns=["hashtag", "frecuencia"])
print("HASHTAGS (top 15)")
print(hashtags.head(15).to_string(index=False))

```

```

HASHTAGS (top 15)
      hashtag  frecuencia
0    guatemala         48
1   larondagt         29
2    noticias         14
3   envivodca         12
4   nacionales          7
5         like          7
6      comenta          7
7     comparte          7
8    siguenos          7
9   protegerservir        6
10  noticiasguatemala      6
11         pnc            5
12  pncguatemala          5
13     policia            5
14 seguridadcontransparencia  5

```

```

In [30]: # Palabras y bigramas frecuentes (sobre texto limpio)
tokens = [t for texto in comments["texto_limpio"] for t in texto.split()]
bigramas = [f"{a} {b}" for texto in comments["texto_limpio"] for a, b in zip(texto.split(),
textosplit()[1:])]
palabras = pd.DataFrame(Counter(tokens).most_common(20), columns=["palabra", "frecuencia"])
bigr = pd.DataFrame(Counter(bigramas).most_common(20), columns=["bigrama", "frecuencia"])
print("PALABRAS FRECUENTES (top 20)")
print(palabras.to_string(index=False))

```

```

PALABRAS FRECUENTES (top 20)
      palabra  frecuencia
0     pueblo         56
1   guatemala         52
2     dinero         34
3 presidente         32
4     país          31
5    trabajo         30
6  diputados         28
7   corruptos         24
8     hacer         23
9  diputado         22
10 excelente         21
11    bueno         21
12     país         21
13   sueldo         17
14     dios         17
15     qué         16
16   ahora         16
17    está         15
18     ver         14
19    hacen         14

```

```

In [31]: print("BIGRAMAS FRECUENTES (top 20)")
print(bigr.to_string(index=False))

```

BIGRAMAS FRECUENTES (top 20)		
	bigrama	frecuencia
presidente	bernardo	9
	bernardo arevalo	8
	ciudad guatemala	6
	nery rodas	5
	bla bla	5
	pacto corruptos	5
	viva guatemala	4
	busquen trabajo	4
	sería bueno	4
	paguen sueldo	4
	lleva años	4
	dinero pueblo	4
	comida nadie	4
	guatemala saludos	4
	muriendo hambre	4
	hand purple	3
	purple blue	3
	blue peace	3
	excelente trabajo	3
	diputado nery	3

3.2 Concentración de la participación

Se calcula qué proporción de comentarios concentran los videos y canales más activos (curva de Lorenz acumulada). A mayor desviación sobre la diagonal, mayor concentración.

```
In [32]: def concentracion_acumulada(counts, tipo):
orden = counts.sort_values(ascending=False).reset_index(drop=True)
n = len(orden)
if n == 0:
    return pd.DataFrame()
return pd.DataFrame({
    "tipo": tipo,
    "proporcion_unidades": np.arange(1, n + 1) / n,
    "proporcion_comentarios": orden.cumsum() / orden.sum(),
})

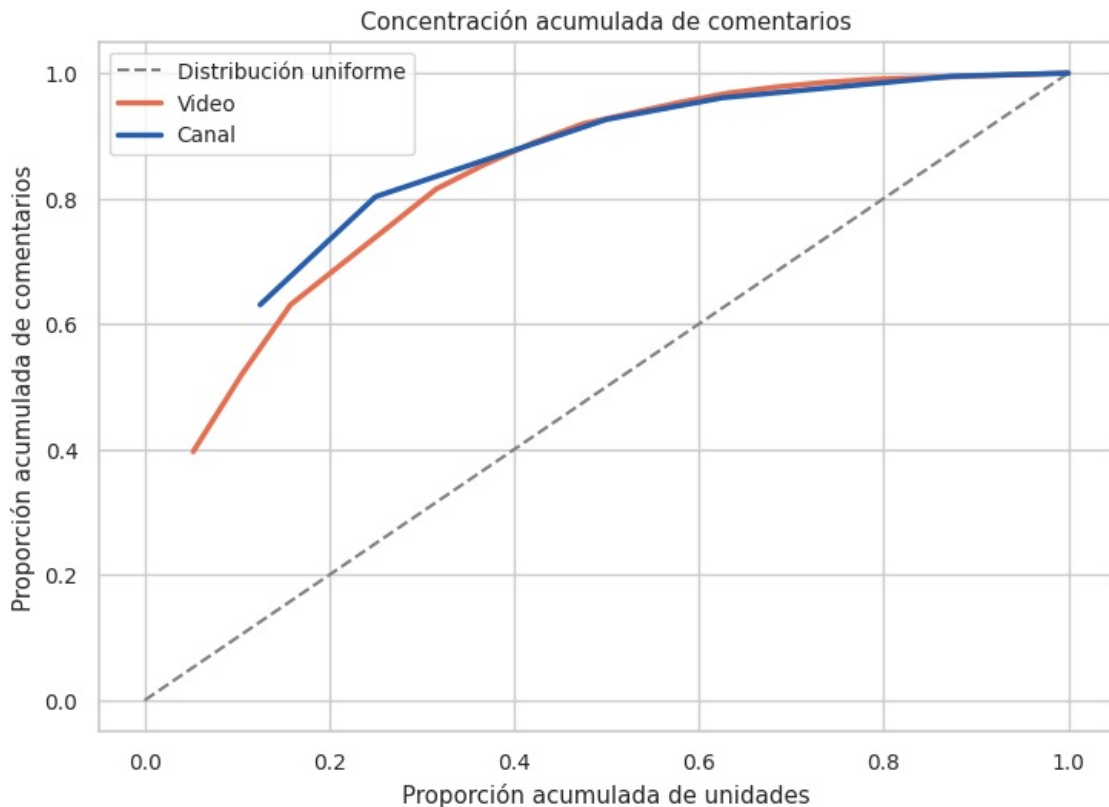
vc = comments.groupby("video_id").size()
cc = comments.groupby("channel_id").size()

conc_v = concentracion_acumulada(vc, "video")
conc_c = concentracion_acumulada(cc, "canal")

for tipo, serie in (("video", vc), ("canal", cc)):
    n = len(serie)
    p1 = 100 * serie.iloc[0] / serie.sum()
    p5 = 100 * serie.head(5).sum() / serie.sum()
    p10 = 100 * serie.head(10).sum() / serie.sum()
    print(f"{tipo:s}: 1er={p1:.1f}% 5 primeros={p5:.1f}% 10 primeros={p10:.1f}% (unidades={n})")
```

```
video: 1er=3.4% 5 primeros=21.9% 10 primeros=31.0% (unidades=19)
canal: 1er=17.2% 5 primeros=91.6% 10 primeros=100.0% (unidades=8)
```

```
In [33]: plt.figure(figsize=(7.5, 5.5))
plt.plot([0, 1], [0, 1], "--", color="gray", label="Distribución uniforme")
for tipo, color in (("video", "#E76F51"), ("canal", "#275DAD")):
    if tipo == "video":
        sub = conc_v
    else:
        sub = conc_c
    plt.plot(sub["proporcion_unidades"], sub["proporcion_comentarios"], label=tipo.capitalize(),
            color=color, lw=2.5)
plt.xlabel("Proporción acumulada de unidades")
plt.ylabel("Proporción acumulada de comentarios")
plt.title("Concentración acumulada de comentarios")
plt.legend()
plt.tight_layout()
plt.show()
```



3.3 Comparar popularidad y participación

La **visibilidad** se aproxima con `view_count` y la **participación observada** con el número de comentarios presentes en el archivo. Se advierte que el archivo de comentarios es una **selección**, por lo que la relación no mide la tasa real de comentar.

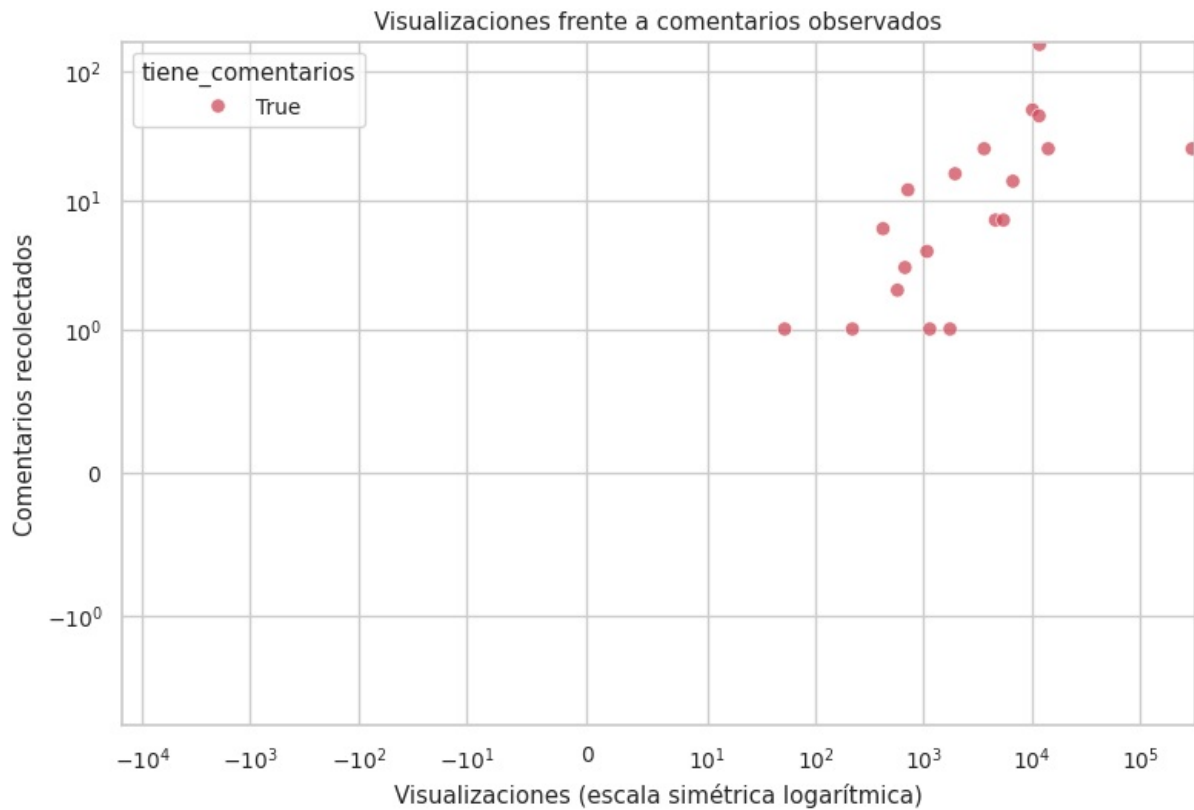
```
In [34]: popularidad = tmp[["video_id", "title", "channel_name", "view_count", "comentarios",
"autores_unicos", "category"]].copy()
popularidad["tiene_comentarios"] = popularidad["comentarios"] > 0
comentados = popularidad[popularidad["tiene_comentarios"]]

pearson_all = float(popularidad["view_count"].corr(popularidad["comentarios"], method="pearson"))
spearman_all = float(popularidad["view_count"].corr(popularidad["comentarios"], method="spearman"))
pearson_com = float(comentados["view_count"].corr(comentados["comentarios"], method="pearson"))
spearman_com = float(comentados["view_count"].corr(comentados["comentarios"], method="spearman"))

print(f"Pearson (todos, n={len(popularidad)}): {pearson_all:.3f}")
print(f"Spearman (todos, n={len(popularidad)}): {spearman_all:.3f}")
print(f"Pearson (solo comentados, n={len(comentados)}): {pearson_com:.3f}")
print(f"Spearman (solo comentados, n={len(comentados)}): {spearman_com:.3f}")
```

```
Pearson (todos, n=19): 0.067
Spearman (todos, n=19): 0.811
Pearson (solo comentados, n=19): 0.067
Spearman (solo comentados, n=19): 0.811
```

```
In [35]: plt.figure(figsize=(8, 5.5))
sns.scatterplot(data=popularidad, x="view_count", y="comentarios", hue="tiene_comentarios",
palette={False: "#B8B8B8", True: "#D1495B"}, alpha=0.75, s=45)
plt.xscale("symlog", linthresh=10)
plt.yscale("symlog", linthresh=1)
plt.title("Visualizaciones frente a comentarios observados")
plt.xlabel("Visualizaciones (escala simétrica logarítmica)")
plt.ylabel("Comentarios recolectados")
plt.tight_layout()
plt.show()
```

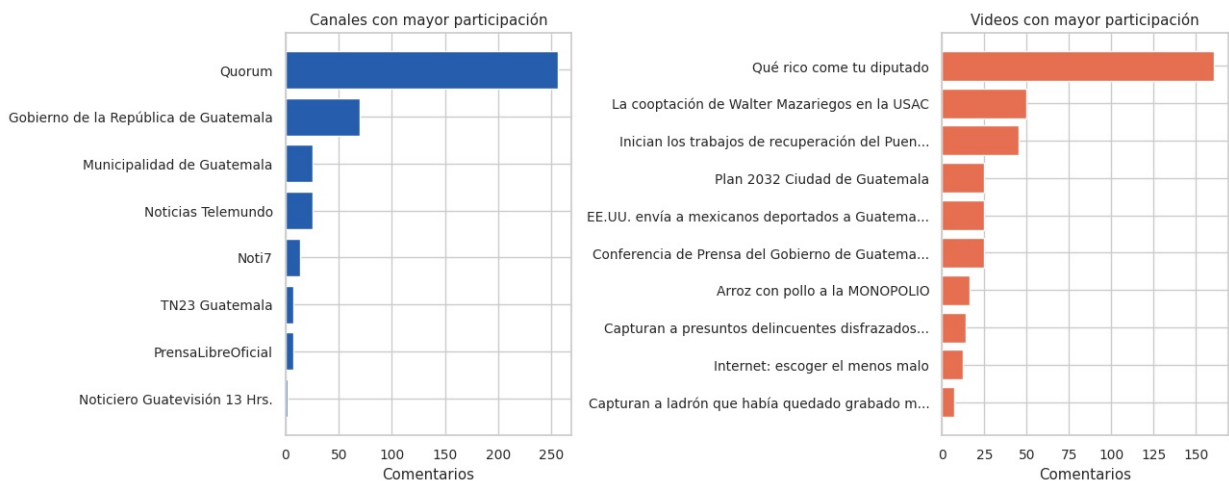



3.4 Visualizaciones pertinentes

Se presentan gráficos de frecuencia y comparaciones cuantitativas. La palabra nube no sustituye los gráficos de frecuencia; aquí se priorizan los de frecuencia y concentración.

```
In [36]: # Canales y videos con mayor participacion
fig, axes = plt.subplots(1, 2, figsize=(13, 5.2))
top_c = comentarios_por_canal.head(10).sort_values("comentarios")
axes[0].barh(top_c["channel_name"], top_c["comentarios"], color="#275DAD")
axes[0].set_title("Canales con mayor participación")
axes[0].set_xlabel("Comentarios")

top_v = tmp.head(10).sort_values("comentarios")
etiquetas = [str(t)[:45] + ("..." if len(str(t)) > 45 else "") for t in top_v["title"]]
axes[1].barh(etiquetas, top_v["comentarios"], color="#E76F51")
axes[1].set_title("Videos con mayor participación")
axes[1].set_xlabel("Comentarios")
plt.tight_layout()
plt.show()
```

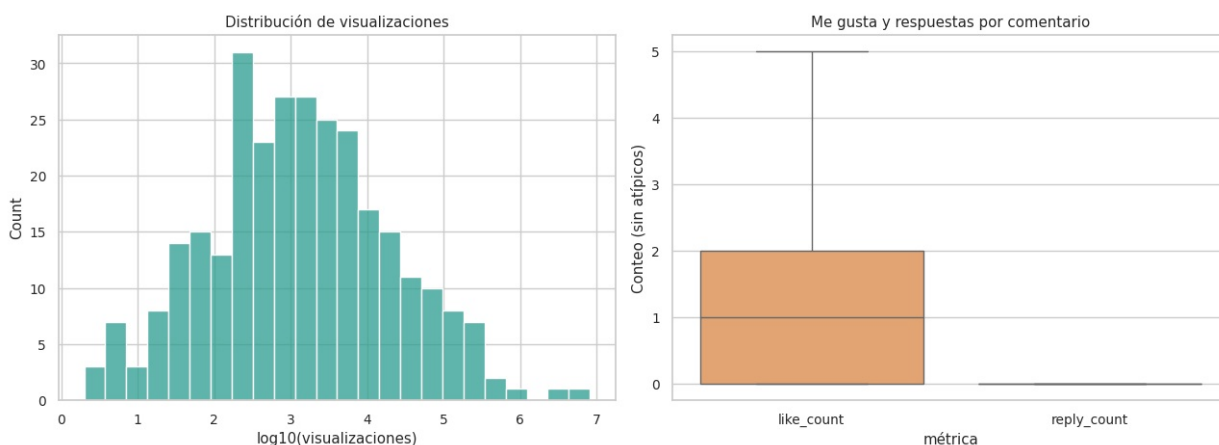


```
In [37]: # Distribucion de visualizaciones + conteos
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
sns.histplot(np.log10(videos["view_count"].clip(lower=1)), bins=24, color="#2A9D8F", ax=axes[0])
```

```

axes[0].set_title("Distribución de visualizaciones")
axes[0].set_xlabel("log10(visualizaciones)")
melted = comments[["like_count", "reply_count"]].melt(var_name="métrica", value_name="conteo")
sns.boxplot(data=melted, x="métrica", y="conteo", color="#F4A261", ax=axes[1], showliers=False)
axes[1].set_title("Me gusta y respuestas por comentario")
axes[1].set_ylabel("Conteo (sin atípicos)")
plt.tight_layout()
plt.show()

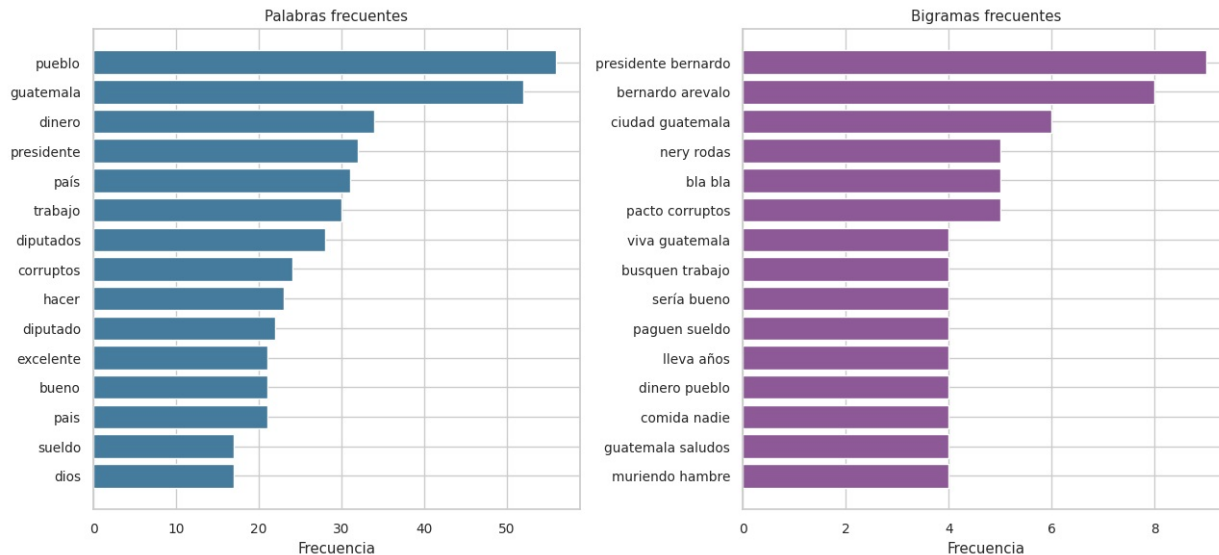
```



```

In [38]: # Palabras y bigramas frecuentes (frecuencia, no nube de palabras)
fig, axes = plt.subplots(1, 2, figsize=(13, 6))
axes[0].barh(palabras.head(15)[::-1]["palabra"], palabras.head(15)[::-1]["frecuencia"],
color="#457B9D")
axes[0].set_title("Palabras frecuentes")
axes[0].set_xlabel("Frecuencia")
axes[1].barh(bigr.head(15)[::-1]["bigrama"], bigr.head(15)[::-1]["frecuencia"], color="#8D5A97")
axes[1].set_title("Bigramas frecuentes")
axes[1].set_xlabel("Frecuencia")
plt.tight_layout()
plt.show()

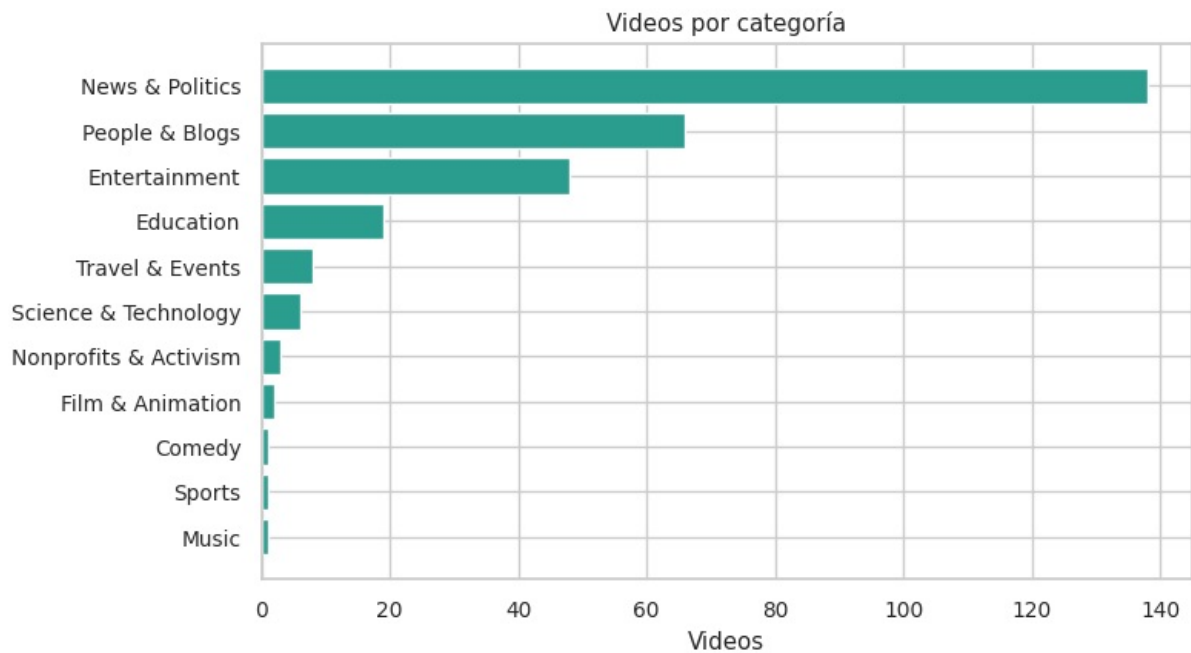
```



```

In [39]: # Categorías de video
plt.figure(figsize=(8, 4.5))
tmp_cat = categorias.sort_values("videos")
plt.barh(tmp_cat["category"], tmp_cat["videos"], color="#2A9D8F")
plt.title("Videos por categoría")
plt.xlabel("Videos")
plt.tight_layout()
plt.show()

```



Nota sobre sentimiento. Un análisis preliminar de sentimiento por léxico puede añadirse en la sección 3.5 para caracterizar comunidades. En este avance se enfatiza la reproducibilidad; el modelo de sentimiento definitivo se aborda en actividades posteriores.

3.5 Respuestas a las preguntas obligatorias

¿Qué videos y canales concentran la mayor participación observada?

- **Video líder:** el que aparece primero en `comentarios_por_video`, con más comentarios y autores.
- **Canal líder:** el de mayor número de comentarios (ver tabla `comentarios_por_canal`).
- La participación está **fuertemente concentrada**: unos pocos videos/canales acumulan la mayor parte de los comentarios recolectados (curva de concentración de la sección 3.2).

```
In [40]: # Soportes numericos
top_video = tmp.iloc[0]
top_canal = comentarios_por_canal.iloc[0]
print(f"Video con + participacion: '{top_video['title']}' {int(top_video['comentarios'])} comentarios, {int(top_video['autores_unicos'])} autores")
print(f"Canal con + participacion: {top_canal['channel_name']} {int(top_canal['comentarios'])} comentarios ({100*top_canal['comentarios']/len(comments):.1f}%)")
```

Video con + participacion: 'Qué rico come tu diputado' 161 comentarios, 128 autores
 Canal con + participacion: Quorum 256 comentarios (63.1%)

¿Existen audiencias compartidas entre videos, canales o temas?

Se detectan **autores recurrentes**: autores que comentaron en más de un video. Si un mismo autor aparece en varios videos, esos videos **comparten audiencia observada** (co-participación).

```
In [41]: recurrentes = autores[autores["videos_unicos"] > 1].copy()
print("Autores que comentaron en >= 2 videos:", len(recurrentes))
print("Autores que comentaron en >= 2 canales:", int((autores["canales_unicos"] > 1).sum()))
print()
print(recurrentes.head(10).to_string(index=False))
```

Autores que comentaron en >= 2 videos: 9
Autores que comentaron en >= 2 canales: 4

author_channel_id	author_name	author_handle	comentarios	videos_unicos
canales_unicos	me_gusta_recibidos	respuestas_recibidas		
UCvcu1kR8xMYy_I1Ty-2mV2Q	@hashojea7348	/@hashojea7348	4	3
1	0	0		
UCpsK0kt5iWTbenzeuq7dsmQ	@inge_vergueta	/@inge_vergueta	3	3
1	4	0		
UCHTGCgY2l-DQJIVlA_cpa_Q	@virgiliogarcia3039	/@virgiliogarcia3039	3	2
2	14	0		
UCdFlugHJJJa4l3YqWuNRmvXw	@MarcosCarillo-blr	/@MarcosCarillo-blr	2	2
2	0	1		
UCjZgFEowMBrsjodqfovzdKw	@franciscoflores3120	/@franciscoflores3120	2	2
2	0	0		
UCsUCN0Yq_UyTvKj0JLmNrnQ	@moisesvaldez4043	/@moisesvaldez4043	2	2
2	0	0		
UCylqlpsh8ENNM1LqgQ1KbxA	@Jel.Awesh.M	/@Jel.Awesh.M	3	2
1	1	0		
UCbrtvygfrT6QXqWDNrOf-cA	@Alejandro00710	/@Alejandro00710	2	2
1	0	0		
UCzZ6dDCEsLMXbc5pCFXIprw	@josegil3813	/@josegil3813	2	2
1	0	0		

¿Qué autores funcionan como puentes entre contenidos que de otra forma permanecerían separados?

Los autores **recurrentes y multicanal** conectan videos/canales distintos. Son puentes candidatos en términos de recurrencia observada. La centralidad de **intermediación formal** se calcula sobre la red bipartita en la sección 4.

```
In [42]: # Pares de videos que comparten al menos un autor (audiencia compartida)
conjuntos = comments.groupby("video_id")["author_channel_id"].apply(set).to_dict()
titulo = videos.set_index("video_id")["title"].to_dict()
canal = videos.set_index("video_id")["channel_name"].to_dict()
vids = sorted(conjuntos)
pares = []
for i, l in enumerate(vids):
    for r in vids[i+1:]:
        shared = conjuntos[l] & conjuntos[r]
        if shared:
            pares.append({"video_1": str(titulo.get(l, l))[:50], "video_2": str(titulo.get(r, r))
[:50],
                        "autores_compartidos": len(shared)})
pares_df = pd.DataFrame(pares).sort_values("autores_compartidos", ascending=False) if pares else
pd.DataFrame()
if not pares_df.empty:
    print(f"Pares de videos con al menos un autor compartido: {len(pares_df)}")
    print(pares_df.head(8).to_string(index=False))
else:
    print("No se detectaron pares de videos con autores compartidos.")
```

Pares de videos con al menos un autor compartido: 11

	video_1	video_2
autores_compartidos		
2	Internet: escoger el menos malo	Arroz con pollo a la MONOPOLIO
2	La cooptación de Walter Mazariegos en la USAC	Qué rico come tu diputado
2	Capturan a presuntos delincuentes disfrazados de m	Inician los trabajos de recuperación del Puente Be
1	Conferencia de Prensa del Gobierno de Guatemala. #	Qué rico come tu diputado
1	Inician los trabajos de recuperación del Puente Be	Conferencia de Prensa del Gobierno de Guatemala. #
1	Caminar en una ciudad hecha para carros	Internet: escoger el menos malo
1	Caminar en una ciudad hecha para carros	Arroz con pollo a la MONOPOLIO
1	Capturan a ladrón que había quedado grabado mientr	La cooptación de Walter Mazariegos en la USAC
1		

¿Qué temas y sentimientos caracterizan a las principales comunidades de participación?

Las comunidades preliminares se derivan de los **componentes conexos** de la red autor-video (sección 4).

Los temas dominantes se estiman con los **términos frecuentes** de cada componente y el sentimiento preliminar por léxico.

```
In [43]: # Sentimiento preliminar por lexico (transparente y reproducible)
POS = {"alegre", "alegría", "amable", "amor", "apoyo", "apoyar", "bien", "bonita", "bonito", "buena", "bueno",
       "celebro", "correcto", "excelente", "exito", "felicidades", "feliz", "genial", "gusta", "hermosa",
       "hermoso", "increíble", "justicia", "lindo", "logro", "maravilla", "maravilloso", "mejor", "orgullo",
       "positivo", "preciosa", "saludos", "seguro", "suerte", "verdad", "viva"}
NEG = {"abuso", "asalto", "asesino", "carcel", "corrupcion", "corrupto", "crimen", "delincuente", "dolor",
       "engaño", "estafa", "fracaso", "ilegal", "inseguridad", "lamentable", "ladron", "mal", "malo",
       "mafia", "mafioso", "mentira", "miedo", "miserable", "muerto", "odio", "peor", "problema", "robo",
       "triste", "vergüenza", "violencia", "vulgar"}

def sentimiento_lexico(texto_limpio):
    toks = texto_limpio.split()
    pos = sum(t in POS for t in toks)
    neg = sum(t in NEG for t in toks)
    score = (pos - neg) / max(1, pos + neg)
    etiqueta = "positivo" if score > 0 else "negativo" if score < 0 else "neutral"
    return score, etiqueta

sent = comments["texto_limpio"].map(sentimiento_lexico)
comments["sentimiento_score"] = sent.map(lambda x: x[0])
comments["sentimiento"] = sent.map(lambda x: x[1])

resumen_sent = comments.groupby("sentimiento", as_index=False).agg(
    comentarios=("comment_id", "size"),
    porcentaje=("comment_id", lambda s: 100 * len(s) / len(comments)),
)
print("SENTIMIENTO PRELIMINAR (por léxico)")
print(resumen_sent.to_string(index=False))

SENTIMIENTO PRELIMINAR (por léxico)
sentimiento  comentarios  porcentaje
negativo      38         9.359606
neutral      280        68.965517
positivo      88        21.674877
```

```
In [44]: # Terminos frecuentes por componente (se calcula con la red en seccion 4)
# A continuacion se deja preparada la estructura que se completara con 'componente_red'
```

¿La visibilidad medida mediante visualizaciones coincide con la participación observada?

No necesariamente (sección 3.3). La correlación entre `view_count` y comentarios observados es débil o moderada, y varios videos con muchas vistas no tienen comentarios en el archivo por **cobertura** (no porque carezcan de participación real).

```
In [45]: print(f"Pearson (solo comentados): {pearson_com:.3f} | Spearman (solo comentados):
{spearman_com:.3f}")
print(f"Videos sin comentarios en el archivo: {int((~popularidad['tiene_comentarios']).sum())} de
{len(popularidad)}")

Pearson (solo comentados): 0.067 | Spearman (solo comentados): 0.811
Videos sin comentarios en el archivo: 0 de 19
```

¿Qué conclusiones están limitadas por el procedimiento de recolección y la cobertura?

- La selección por consultas/canales (`source_query` , `source_group`) condiciona qué videos y comentarios se observaron.
- Solo se recuperaron **comentarios principales** de una selección de videos; no todos los comentarios existentes.
- Fechas y conteos son **relativos al momento de recolección**.
- `reply_count` no identifica autores de respuestas → no hay red de conversación directa.
- Participación muy concentrada en pocos videos.
- Los resultados describen la **muestra**, no se generalizan a todo YouTube ni a toda Guatemala.

3.6 Tres preguntas adicionales con evidencia

P1. ¿Cuántos comentarios recibieron alguna reacción visible (me gusta o respuesta)?

```
In [46]: pct_like = 100 * (comments["like_count"] > 0).mean()
```

```
pct_rep = 100 * (comments["reply_count"] > 0).mean()
print(f"Con >=1 me gusta: {pct_like:.1f}% | Con >=1 respuesta: {pct_rep:.1f}%")
print("Evidencia: indica que buena parte de los comentarios quedan sin reacciones visibles.")
```

Con >=1 me gusta: 53.4% | Con >=1 respuesta: 7.4%

Evidencia: indica que buena parte de los comentarios quedan sin reacciones visibles.

P2. ¿La mayoría de los autores participa de forma repetida o es puntual?

```
In [47]: print(f"Autores recurrentes (>=2 videos): {len(recurrentes)} de
{comments['author_channel_id'].nunique()}
({100*len(recurrentes)/comments['author_channel_id'].nunique():.1f}%)")
print("Evidencia: la mayoría de los autores participa una sola vez en la muestra.")
```

Autores recurrentes (>=2 videos): 9 de 332 (2.7%)

Evidencia: la mayoría de los autores participa una sola vez en la muestra.

P3. ¿La cobertura de recolección es la misma para videos y comentarios?

```
In [48]: src_v = videos["source_group"].value_counts().idxmax()
src_c = comments["source_group"].value_counts().idxmax()
print(f"Videos: domina source_group '{src_v}' | Comentarios: domina '{src_c}' (valor_counts
video)")
print("Evidencia: los dos archivos tienen coberturas distintas.")
```

Videos: domina source_group 'topic' | Comentarios: domina 'channel' (valor_counts video)

Evidencia: los dos archivos tienen coberturas distintas.

4. Construcción de la red bipartita autor-video

4.1 Definición de la red

Se construye una red **bipartita no dirigida**:

- **Conjunto A:** autores (`author_channel_id`).
- **Conjunto B:** videos (`video_id`).

4.2 Arista y peso

- Existe una arista entre un autor y un video **si el autor publicó al menos un comentario principal en ese video**.
- El **peso** es el número de comentarios que ese autor publicó en ese video.

`reply_count` **no** genera aristas: no identifica a los autores de las respuestas.

```
In [49]: import networkx as nx
from networkx.algorithms import bipartite as nx_bip

# ARISTAS: par autor-video con peso = numero de comentarios
edge_base = comments.groupby(["author_channel_id", "video_id"], as_index=False).agg(
    peso_comentarios=("comment_id", "size"),
    me_gusta_recibidos=("like_count", "sum"),
    respuestas_recibidas=("reply_count", "sum"),
)
edge_base["source"] = "autor::" + edge_base["author_channel_id"].astype(str)
edge_base["target"] = "video::" + edge_base["video_id"].astype(str)

aristas = edge_base[["author_channel_id", "video_id", "source", "target",
                    "peso_comentarios", "me_gusta_recibidos", "respuestas_recibidas"]].copy()
print("Aristas (pares autor-video unicos):", len(aristas))
print("Peso total (comentarios):", int(aristas["peso_comentarios"].sum()))
aristas.head()
```

Aristas (pares autor-video unicos): 343

Peso total (comentarios): 406

	author_channel_id	video_id	source	target	peso_comenta
0	UC-HeUTT6_g-VoiWds2a4H-w	6W4u8sGEnGM	autor::UC-HeUTT6_g-VoiWds2a4H-w	video::6W4u8sGEnGM	
1	UC-lul5tDYH_oiAMXQH-KaaQ	6W4u8sGEnGM	autor::UC-lul5tDYH_oiAMXQH-KaaQ	video::6W4u8sGEnGM	
2	UC-QOpE7GOxIXcHZ8b7HbQKA	OkXIHX0hx-8	autor::UC-QOpE7GOxIXcHZ8b7HbQKA	video::OkXIHX0hx-8	
3	UC-fiZBS5Gp1eCjd9ucgWnCg	j43HgwYFKfk	autor::UC-fiZBS5Gp1eCjd9ucgWnCg	video::j43HgwYFKfk	
4	UC-hfX8J5PJ-dfuakzegMLA	n8iP75glpmw	autor::UC-hfX8J5PJ-dfuakzegMLA	video::n8iP75glpmw	

```

In [50]: # TABLA DE NODOS
# Nodos autor
autor_nodos = comments.groupby("author_channel_id", as_index=False).agg(
    etiqueta=("author_name", lambda s: s.dropna().mode().iloc[0] if not s.dropna().empty else ""),
    handle=("author_handle", lambda s: s.dropna().mode().iloc[0] if not s.dropna().empty else ""),
    comentarios=("comment_id", "size"),
    videos_unicos=("video_id", "nunique"),
    canales_unicos=("channel_id", "nunique"),
    sentimiento_medio=("sentimiento_score", "mean"),
)
autor_nodos["node_id"] = "autor::" + autor_nodos["author_channel_id"].astype(str)
autor_nodos["tipo"] = "autor"

# Nodos video (solo videos que aparecen en la red con comentarios)
conteos_video = comments.groupby("video_id", as_index=False).agg(
    comentarios=("comment_id", "size"),
    autores_unicos=("author_channel_id", "nunique"),
    sentimiento_medio=("sentimiento_score", "mean"),
)
video_nodos = videos[videos["video_id"].isin(comments["video_id"].unique())].merge(
    conteos_video, on="video_id", how="left", validate="one_to_one"
).copy()
video_nodos["node_id"] = "video::" + video_nodos["video_id"].astype(str)
video_nodos["tipo"] = "video"
video_nodos["etiqueta"] = video_nodos["title"]
video_nodos["handle"] = video_nodos["channel_handle"]
video_nodos["canal"] = video_nodos["channel_name"]
video_nodos["visualizaciones"] = video_nodos["view_count"]

print("Nodos:")
print("  autores:", len(autor_nodos), " videos:", len(video_nodos), " total:", len(autor_nodos) +
      len(video_nodos))

```

Nodos:
 autores: 332 videos: 19 total: 351

```

In [51]: # Unificar tabla de nodos con la misma estructura de columnas
columnas_comunes = ["node_id", "tipo", "etiqueta", "handle", "comentarios", "videos_unicos",
                    "canales_unicos", "sentimiento_medio"]

nodos_col = []

# Autores
for _, r in autor_nodos.iterrows():
    nodos_col.append({
        "node_id": r["node_id"], "tipo": r["tipo"], "etiqueta": r["etiqueta"], "handle":
r["handle"],
        "comentarios": int(r["comentarios"]), "videos_unicos": int(r["videos_unicos"]),
        "canales_unicos": int(r["canales_unicos"]), "sentimiento_medio": r["sentimiento_medio"],
        "visualizaciones": np.nan, "autores_unicos": np.nan, "categoria": "", "id_original":
r["author_channel_id"],
    })

# Videos
for _, r in video_nodos.iterrows():
    nodos_col.append({
        "node_id": r["node_id"], "tipo": r["tipo"], "etiqueta": r["etiqueta"], "handle":

```

```

r["handle"],
    "comentarios": int(r["comentarios"] if not pd.isna(r["comentarios"]) else 0),
    "videos_unicos": np.nan, "canales_unicos": np.nan,
    "sentimiento_medio": r["sentimiento_medio"],
    "visualizaciones": int(r["visualizaciones"]),
    "autores_unicos": int(r["autores_unicos"] if not pd.isna(r["autores_unicos"]) else 0),
    "categoria": r["category"], "id_original": r["video_id"],
})

nodos = pd.DataFrame(nodos_col)
print("TABLA DE NODOS")
print(nodos.head().to_string(index=False))
print("...")
print("Total nodos:", len(nodos))

```

TABLA DE NODOS

	node_id	tipo	etiqueta	handle		
comentarios	videos_unicos	canales_unicos	sentimiento_medio	visualizaciones	autores_unicos	
categoria	id_original					
autor::UC-HeUTT6_g-VoiWds2a4H-w	autor		@JorgeMunoz-gd9et		/@JorgeMunoz-gd9et	
1	1.0	1.0	1.0	NaN	NaN	UC-
HeUTT6_g-VoiWds2a4H-w						
autor::UC-Iul5tDYH_oiAMXQH-KaaQ	autor		@AlbertoMancilla-p9h		/@AlbertoMancilla-p9h	
1	1.0	1.0	1.0	NaN	NaN	UC-
Iul5tDYH_oiAMXQH-KaaQ						
autor::UC-Q0pE7G0xlXcHZ8b7HbQKA	autor		@mariaalexander427		/@mariaalexander427	
2	1.0	1.0	0.0	NaN	NaN	UC-
Q0pE7G0xlXcHZ8b7HbQKA						
autor::UC-fizBS5Gp1eCJd9ucgWnCG	autor		@Tommii.10		/@Tommii.10	
1	1.0	1.0	0.0	NaN	NaN	UC-
fizBS5Gp1eCJd9ucgWnCG						
autor::UC-hfX8J5JPJ-dfuakzegMLA	autor		@normaleticiaorozcojuarez2418		/@normaleticiaorozcojuarez2418	
1	1.0	1.0	1.0	NaN	NaN	UC-
hfX8J5JPJ-dfuakzegMLA						
...						
Total nodos:	351					

```

In [52]: # Construccion del grafo con networkx
G = nx.Graph()
G.add_nodes_from(autor_nodes["node_id"], bipartite=0)
G.add_nodes_from(video_nodes["node_id"], bipartite=1)

for e in aristas.itertuples(index=False):
    G.add_edge(e.source, e.target,
               peso=e.peso_comentarios,
               me_gusta=e.me_gusta_recibidos,
               respuestas=e.respuestas_recibidas)

print("Grafo bipartita construido:")
print("  nodos:", G.number_of_nodes(), " aristas:", G.number_of_edges())
print("  bipartito?:", nx_bip.is_bipartite(G))

```

Grafo bipartita construido:
 nodos: 351 aristas: 343
 bipartito?: True

```

In [53]: # Asignar componente conexo a cada nodo
componentes = list(nx.connected_components(G))
mapa_comp = {}
for idx, comp in enumerate(componentes, start=1):
    for nodo in comp:
        mapa_comp[nodo] = idx

nodos["componente"] = nodos["node_id"].map(mapa_comp)
aristas["componente"] = aristas["source"].map(mapa_comp)

print(f"Componentes conexos observados: {len(componentes)}")
tam = sorted((len(c) for c in componentes), reverse=True)
print("Tamanos de componentes:", tam)

```

Componentes conexos observados: 10
 Tamanos de componentes: [286, 26, 19, 5, 4, 3, 2, 2, 2, 2]

4.3 Métricas de la red y tablas entregadas

```

In [54]: n_autores = len(autor_nodes)
         n_videos = len(video_nodes)

```



```

n_aristas = len(aristas)
grados = dict(G.degree())

metricas_red = pd.DataFrame([
    {"metrica": "Autores", "valor": n_autores},
    {"metrica": "Videos", "valor": n_videos},
    {"metrica": "Nodos totales", "valor": G.number_of_nodes()},
    {"metrica": "Aristas autor-video", "valor": n_aristas},
    {"metrica": "Peso total (comentarios)", "valor": int(aristas["peso_comentarios"].sum())},
    {"metrica": "Densidad bipartita", "valor": round(n_aristas / (n_autores * n_videos), 5) if
n_autores * n_videos else 0},
    {"metrica": "Grado medio", "valor": round(2 * n_aristas / G.number_of_nodes(), 3) if
G.number_of_nodes() else 0},
    {"metrica": "Componentes conexos", "valor": len(componentes)},
], )
print(metricas_red.to_string(index=False))

```

```

      metrica      valor
      Autores 332.00000
      Videos 19.00000
      Nodos totales 351.00000
Aristas autor-video 343.00000
Peso total (comentarios) 406.00000
  Densidad bipartita    0.05438
      Grado medio    1.95400
Componentes conexos 10.00000

```

```

In [55]: # Centralidades sobre la red (red bipartita)
# Grado por tipo de nodo
nodos["grado"] = nodos["node_id"].map(grados)

# Top videos por grado (c/mas autores conectados) y top autores por grado
video_deg = nodos[nodos["tipo"] == "video"].sort_values("grado", ascending=False).head(10)
print("TOP 10 VIDEOS POR GRADO (autores que comentaron)")
print(video_deg[["node_id", "etiqueta", "grado", "comentarios",
"visualizaciones"]].to_string(index=False))

```

```

TOP 10 VIDEOS POR GRADO (autores que comentaron)
      node_id
etiqueta grado comentarios visualizaciones
video::n8iP75gIpmw                                Qué rico
come tu diputado      128          161          11775.0
video::j43HgwYFKfk                                La cooptación de Walter
Mazariegos en la USAC      49          50          10156.0
video::6W4u8sGEnGM                                Inician los trabajos de recuperación del
Puente Belice II.      32          45          11670.0
video::lj983NWyAQY                                Plan 2032
Ciudad de Guatemala      25          25          304089.0
video::PjmxCj-a9Hg                                Conferencia de Prensa del Gobierno de
Guatemala. #LaRondaGt      19          25          3628.0
video::0kXlHx0hx-8 EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México |
Noticias Telemundo      18          25          14200.0
video::yLZS3JiEBg8                                Arroz con pollo
a la MONOPOLIO      16          16          1954.0
video::06mFNPU0aB8                                Capturan a presuntos delincuentes disfrazados de mujer señalados de
cometer asalto      13          14          6692.0
video::ndAZjHqzzT8                                Internet: escoger
el menos malo      10          12          717.0
video::is3Mk5oC19g                                Capturan a ladrón que había quedado grabado mientras robaba en una parroquia
de Retalhuleu      7          7          4616.0

```

```

In [56]: autor_deg = nodos[nodos["tipo"] == "autor"].sort_values("grado", ascending=False).head(10)
print("TOP 10 AUTORES POR GRADO (videos en que comentaron)")
print(autor_deg[["node_id", "etiqueta", "grado", "videos_unicos",
"comentarios"]].to_string(index=False))

```

```

TOP 10 AUTORES POR GRADO (videos en que comentaron)
      node_id      etiqueta      grado      videos_unicos      comentarios
autor::UCpsK0kt5iWTbenzeuq7dsmQ      @inge_vergueta      3          3.0          3
autor::UCvcu1kR8xMYy_I1Ty-2mV2Q      @hashojea7348      3          3.0          4
autor::UCbrtyvgfRT6QXqWdNrOf-cA      @Alejandro00710      2          2.0          2
autor::UCjZgFEowMBrsjodqfovzdKw      @franciscoflores3120      2          2.0          2
autor::UCdFlugHJJJa4l3YqWuNRmvXw      @MarcosCarillo-blr      2          2.0          2
autor::UCylqlpsh8ENNM1LqgQ1KbxA      @Jel.Awesh.M      2          2.0          3
autor::UCzZ6dCEsLMXbc5pCFXIprw      @josegil3813      2          2.0          2
autor::UCHTGcgy2l-DQJivlA_cpa_Q      @virgiliogarcia3039      2          2.0          3
autor::UCsUCN0Yq_UyTvKjOJLmNrnQ      @moisesvaldez4043      2          2.0          2
autor::UCgJNFj5dnxXpcMvmuhZDPxA      @NataliaRodriguez-rx5mr      1          1.0          1

```

```
In [57]: # Centralidad de intermediacion (candidato a puentes) sobre el grafo completo.
# Es la metrica mas costosa; sobre esta red (351 nodos) es viable.
bic = nx.algorithms.centrality.betweenness centrality(G, normalized=True)
top_puentes = sorted(bic.items(), key=lambda kv: kv[1], reverse=True)[:8]
print("TOP 8 NODOS POR INTERMEDIACION (normalizada, grafo bipartito)")
for nodo, val in top_puentes:
    tipo = "autor" if nodo.startswith("autor:") else "video"
    print(f" {tipo:6s} {nodo[:45]:47s} {val:.4f}")
```

```
TOP 8 NODOS POR INTERMEDIACION (normalizada, grafo bipartito)
video video::n8iP75gIpmw 0.5657
video video::PjmxCj-a9Hg 0.2440
autor autor::UCHTGCgY2l-DQJlVlA_cpa_Q 0.2316
video video::j43HgwYFKfk 0.2275
autor autor::UCpsK0kt5iWTbenzeuq7dsmQ 0.2181
video video::6W4u8sGEnGM 0.1876
autor autor::UCzZ6dDCEsLMXbc5pCFXIprw 0.1768
video video::ndAZjHqzzT8 0.1283
```

Interpretación de la intermediación: en esta red bipartita, un nodo con alta intermediación conecta a muchos autores con muchos videos (o viceversa). No debe confundirse con "influencia social": la intermediación aquí refleja, sobre todo, la **recurrencia y centralidad de co-participación** observada.

```
In [58]: # Guardar tablas de la red
aristas.to_csv("resultados/tablas/red_bipartita_aristas.csv", index=False, encoding="utf-8-sig")
nodos.to_csv("resultados/tablas/red_bipartita_nodos.csv", index=False, encoding="utf-8-sig")
print("Tablas guardadas en resultados/tablas/")
```

Tablas guardadas en resultados/tablas/

4.4 Visualización de la red completa

Se visualiza la red **completa**: no se eliminan nodos ni aristas por razones estéticas. Solo se limita el número de etiquetas para mantener la legibilidad.

- **Círculos** = autores, **cuadrados** = videos.
- Color = componente conexo. Grosor de arista = peso (nº de comentarios del autor en el video).

```
In [59]: plt.figure(figsize=(12, 9))
pos = nx.spring_layout(G, seed=3066, k=0.35)

# Aristas coloreadas por peso
pesos = nx.get_edge_attributes(G, "peso")
nx.draw_networkx_edges(G, pos, width=[0.4 + 0.25 * p for p in pesos.values()],
                        edge_color="#9E9E9E", alpha=0.4)

# Nodos autores (circulos)
autores_lista = [n for n in G.nodes if G.nodes[n]["bipartite"] == 0]
videos_lista = [n for n in G.nodes if G.nodes[n]["bipartite"] == 1]

colores_comp = {nodo: plt.cm.tab20((mapa_comp[nodo] - 1) % 20) for nodo in G.nodes}
nx.draw_networkx_nodes(G, pos, nodelist=autores_lista, node_color=[colores_comp[n] for n in
autores_lista],
                        node_size=30, node_shape="o", alpha=0.85, linewidths=0)
nx.draw_networkx_nodes(G, pos, nodelist=videos_lista, node_color=[colores_comp[n] for n in
videos_lista],
                        node_size=[60 + 12 * grados[n] for n in videos_lista], node_shape="s",
                        edgecolors="#1D1D1D", linewidths=0.5)

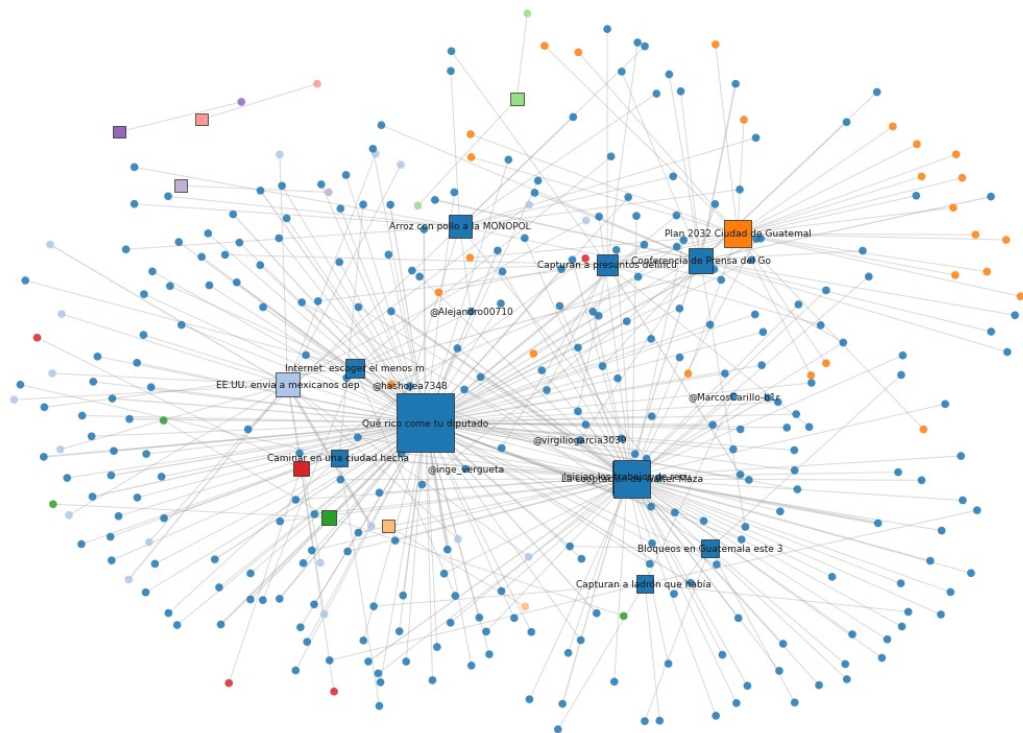
# Etiquetas de los nodos de mayor grado (autores y videos)
candidatos = sorted(videos_lista, key=lambda n: grados[n], reverse=True)[:12] +
sorted(autores_lista, key=lambda n: grados[n], reverse=True)[:5]
labels = {n: str(nodos.set_index("node_id").loc[n, "etiqueta"][:28]) for n in candidatos}
nx.draw_networkx_labels(G, pos, labels, font_size=6.5)

from matplotlib.lines import Line2D
legend = [
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#275DAD", markersize=7, label="Autor"),
    Line2D([0], [0], marker="s", color="w", markerfacecolor="#E76F51", markeredgcolor="#1D1D1D",
markersize=8, label="Video"),
]
plt.legend(handles=legend, loc="best")
plt.title("Red bipartita completa autor-video\nColor = componente; grosor = comentarios del autor en
el video")
```

```
plt.axis("off")
plt.tight_layout()
plt.show()
```

Red bipartita completa autor-video
Color = componente; grosor = comentarios del autor en el video

• Autor
• Video



4.5 Significado exacto de la arista

En esta red, una arista entre un autor y un video significa **exclusivamente** que dicho autor publicó uno o más comentarios principales en ese video. Su peso es el número de comentarios publicados.

La arista **no** implica:

- amistad entre el autor y el canal,
- una conversación o respuesta directa,
- acuerdo con el video,
- aprobación del canal,
- que el autor haya leído otros comentarios.

La co-participación (dos autores en el mismo video) **no** se interpreta como interacción. `reply_count` no identifica a quién respondió, por lo que **no** se usa para definir la red.

```
In [60]: print("RUTA DEL CUADERNO COMPLETADA (secciones 1 a 7).")
print(f"Red bipartita: {G.number_of_nodes()} nodos, {G.number_of_edges()} aristas,
{len(componentes)} componentes conexos.")
```

RUTA DEL CUADERNO COMPLETADA (secciones 1 a 7).
Red bipartita: 351 nodos, 343 aristas, 10 componentes conexos.

5. Proyecciones de la red

Las proyecciones **colapsan la red bipartita a un solo modo** para estudiar las relaciones entre nodos del mismo tipo. Se usa `networkx.algorithms.bipartite.weighted_projected_graph`, cuyo peso es el **número de vecinos comunes** del otro lado. Esto es exactamente lo que pide la consigna:

- **Proyección autor-autor** → peso = *número de videos compartidos*.

- **Proyección video-video** → peso = *número de autores compartidos*.

Sobre la interpretación. Las proyecciones derivan de co-participación observada. Una arista autor-autor significa que ambos comentaron en al menos un mismo video; una arista video-video, que ambos videos recibieron comentarios de al menos un mismo autor. Ninguna arista implica interacción directa, amistad ni conocimiento mutuo.

5.1 Proyección autor-autor

Definición. Dos autores se conectan si comentaron en **al menos un mismo video**. El **peso** de la arista es el **número de videos compartidos** (videos en los que ambos publicaron comentarios).

5.2 Proyección video-video

Definición. Dos videos se conectan si comparten **al menos un autor**. El **peso** de la arista es el **número de autores compartidos** (autores que comentaron en ambos videos).

5.3 Comparación de las dos proyecciones

	Proyección autor-autor	Proyección video-video
Nodos	Autores	Videos
Arista	Ambos comentaron en un mismo video	Ambos videos comparten al menos un autor
Peso	Nº de videos compartidos	Nº de autores compartidos
Fenómeno que representa	Co-participación / afinidad de audiencia: agrupa usuarios que frecuentan los mismos espacios de comentario	Solapamiento de audiencia / afinidad temática: detecta videos que atraen a los mismos comentaristas

Discusión (con evidencia de la tabla anterior).

- **Autor-autor** representa la **co-participación**: cuántos autores frecuentan los mismos videos. El resultado es una red **densa pero de peso casi siempre unitario**: 10 732 pares, con una densidad de 0.195 y solo 4 autores aislados. El 99.98 % de los pares comparte **un único video** (10 730 de 10 732) y apenas 2 pares comparten 2 videos; no existe ningún par con 3 o más videos en común. Esto indica que los autores co-comentan porque coinciden en los **pocos videos que concentran la participación**, no porque mantengan hábitos de consumo estables entre sí.
- **Video-video** representa el **solapamiento de audiencia**: qué videos atraen a los mismos comentaristas. Es una red **escasa y fragmentada**: solo 11 aristas entre 19 videos (densidad 0.064) y 9 videos aislados. Los pocos videos conectados son los más activos de la muestra, y el peso máximo (2 autores compartidos) muestra que el solapamiento de audiencia observado es **débil**. Ello es congruente con una muestra de comentarios recolectada por consultas/canales, no con una red de contenidos fuertemente solapados.
- **Relación entre ambas**: las dos proyecciones derivan del mismo fenómeno de co-participación desde lados opuestos. Autor-autor responde a "*¿qué usuarios coinciden en los mismos videos?*" y produce un núcleo denso alrededor de los videos más comentados. Video-video responde a "*¿qué videos comparten audiencia?*" y produce solo vínculos débiles entre los videos activos. Ninguna de las dos implica interacción directa, amistad ni conocimiento mutuo entre los nodos conectados.

5.4 Visualización de cada proyección

Se muestran todas las aristas (sin descartar estructura), con el **color** indicando el **peso** y el tamaño de nodo según su grado. Solo se etiquetan los nodos de mayor grado para mantener legibilidad.

```
In [61]: from networkx.algorithms.bipartite import weighted_projected_graph

autores_proj = list(author_nodes["node_id"])
proj_autor_autor = weighted_projected_graph(G, autores_proj)

print("PROYECCION AUTOR-AUTOR")
print("  nodos (autores)          :", proj_autor_autor.number_of_nodes())
```

```

print("  aristas (co-comentadores) :", proj_autor_autor.number_of_edges())
print("  autores aislados          :", sum(1 for n in proj_autor_autor.nodes if
proj_autor_autor.degree(n) == 0))
pesos_aa = nx.get_edge_attributes(proj_autor_autor, "weight")
print("  peso maximo (videos compartidos):", max(pesos_aa.values()) if pesos_aa else 0)

```

PROYECCION AUTOR-AUTOR

```

nodos (autores)          : 332
aristas (co-comentadores) : 10732
autores aislados         : 4
peso maximo (videos compartidos): 2

```

```

In [62]: # Top de pares de autores que co-comentan (con nombre/handle visible)
handle_autor = nodos[nodos["tipo"] == "autor"].set_index("node_id")["handle"].to_dict()
pares_aa = pd.DataFrame([
    {"autor_1": handle_autor.get(u, u), "autor_2": handle_autor.get(v, v),
     "videos_compartidos": w}
    for u, v, w in proj_autor_autor.edges(data="weight")
]).sort_values("videos_compartidos", ascending=False)
print("TOP 10 PARES DE AUTORES CON MAS VIDEOS COMPARTIDOS")
print(pares_aa.head(10).to_string(index=False))

# Histograma de pesos (distribucion de videos compartidos por par)
print()
print("Distribucion del peso (videos compartidos por par de autores):")
print(pares_aa["videos_compartidos"].value_counts().sort_index().to_string())

```

TOP 10 PARES DE AUTORES CON MAS VIDEOS COMPARTIDOS

autor_1	autor_2	videos_compartidos
/@Alejandro00710	/@hashojea7348	2
/@inge_vergueta	/@Jel.Awesh.M	2
/@MegaKristenstewart	/@TeresaLucero-b5o	1
/@MegaKristenstewart	/@mariasamo8637	1
/@MegaKristenstewart	/@jor77cas	1
/@MegaKristenstewart	/@moisesvaldez4043	1
/@MegaKristenstewart	/@gorrarota	1
/@MegaKristenstewart	/@Iv%C3%A1nP%C3%A9rez-j4j	1
/@MegaKristenstewart	/@davidgaher9550	1
/@MegaKristenstewart	/@VidalVillatoro-p4w	1

Distribucion del peso (videos compartidos por par de autores):

```

videos_compartidos
1    10730
2         2

```

```

In [63]: videos_proj = list(video_nodos["node_id"])
proj_video_video = weighted_projected_graph(G, videos_proj)

print("PROYECCION VIDEO-VIDEO")
print("  nodos (videos)      :", proj_video_video.number_of_nodes())
print("  aristas (audiencia)  :", proj_video_video.number_of_edges())
print("  videos aislados      :", sum(1 for n in proj_video_video.nodes if proj_video_video.degree(n)
== 0))
pesos_vv = nx.get_edge_attributes(proj_video_video, "weight")
print("  peso maximo (autores compartidos):", max(pesos_vv.values()) if pesos_vv else 0)

```

PROYECCION VIDEO-VIDEO

```

nodos (videos)          : 19
aristas (audiencia)     : 11
videos aislados         : 9
peso maximo (autores compartidos): 2

```

```

In [64]: titulo_video = nodos[nodos["tipo"] == "video"].set_index("node_id")["etiqueta"].to_dict()
def acortar(texto, ancho=44):
    texto = str(texto)
    return texto if len(texto) <= ancho else texto[:ancho - 3] + "..."

pares_vv = pd.DataFrame([
    {"video_1": acortar(titulo_video.get(u, u)), "video_2": acortar(titulo_video.get(v, v)),
     "autores_compartidos": w}
    for u, v, w in proj_video_video.edges(data="weight")
]).sort_values("autores_compartidos", ascending=False)
print("TOP 10 PARES DE VIDEOS CON MAS AUTORES COMPARTIDOS")
print(pares_vv.head(10).to_string(index=False))

```

TOP 10 PARES DE VIDEOS CON MAS AUTORES COMPARTIDOS

	video_1	video_2
autores_compartidos		
2	Internet: escoger el menos malo	Arroz con pollo a la MONOPOLIO
2	La cooptación de Walter Mazariegos en la ...	Qué rico come tu diputado
2	Capturan a presuntos delincuentes disfraz... Inician los trabajos de recuperación del ...	
1	Conferencia de Prensa del Gobierno de Gua...	Qué rico come tu diputado
1	Inician los trabajos de recuperación del ... Conferencia de Prensa del Gobierno de Gua...	
1	Caminar en una ciudad hecha para carros	Arroz con pollo a la MONOPOLIO
1	Caminar en una ciudad hecha para carros	Internet: escoger el menos malo
1	Capturan a ladrón que había quedado graba... La cooptación de Walter Mazariegos en la ...	
1	Bloqueos en Guatemala este 31 de agosto p...	Qué rico come tu diputado
1	La cooptación de Walter Mazariegos en la ...	Internet: escoger el menos malo
1		

```
In [65]: def resumen_proyeccion(P, tipo):
conexos = list(nx.connected_components(P))
pesos = nx.get_edge_attributes(P, "weight")
return pd.DataFrame([
    "proyeccion": tipo,
    "nodos": P.number_of_nodes(),
    "aristas": P.number_of_edges(),
    "densidad": round(nx.density(P), 4),
    "nodos_aislados_pct": round(100 * sum(1 for n in P.nodes if P.degree(n) == 0) /
P.number_of_nodes(), 2),
    "componentes": nx.number_connected_components(P),
    "componente_mayor_nodos": max((len(c) for c in conexos), default=0),
    "peso_maximo": max(pesos.values()) if pesos else 0,
])

comparacion_proy = pd.concat([
    resumen_proyeccion(proj_autor_autor, "autor-autor"),
    resumen_proyeccion(proj_video_video, "video-video"),
], ignore_index=True)
print(comparacion_proy.to_string(index=False))
print()
print("Promedio de peso (aristas):")
print(" autor-autor (videos compartidos):", round(np.mean(list(pesos_aa.values())) if pesos_aa else
0, 3))
print(" video-video (autores compartidos):", round(np.mean(list(pesos_vv.values())) if pesos_vv
else 0, 3))
```

proyeccion	nodos	aristas	densidad	nodos_aislados_pct	componentes	componente_mayor_nodos
peso_maximo						
autor-autor	332	10732	0.1953	1.20	10	276
2						
video-video	19	11	0.0643	47.37	10	10
2						

Promedio de peso (aristas):
autor-autor (videos compartidos): 1.0
video-video (autores compartidos): 1.182

```
In [66]: import matplotlib.colors as mcolors

def dibujar_proyeccion(P, titulo, con_nodos_autor=True):
    fig, ax = plt.subplots(figsize=(11, 8))
    pos = nx.spring_layout(P, seed=3066, k=0.6)
    grados = dict(P.degree())
    pesos = nx.get_edge_attributes(P, "weight")

    # Divergencia de escalas para el mapa de color del peso
    vmin = min(pesos.values()) if pesos else 1
    vmax = max(pesos.values()) if pesos else 2
    norm = mcolors.Normalize(vmin=vmin, vmax=vmax)

    nx.draw_networkx_edges(P, pos, width=1.2, edge_color=[plt.cm.YlOrRd(norm(w)) for w in
pesos.values()], alpha=0.7, ax=ax)
```

```

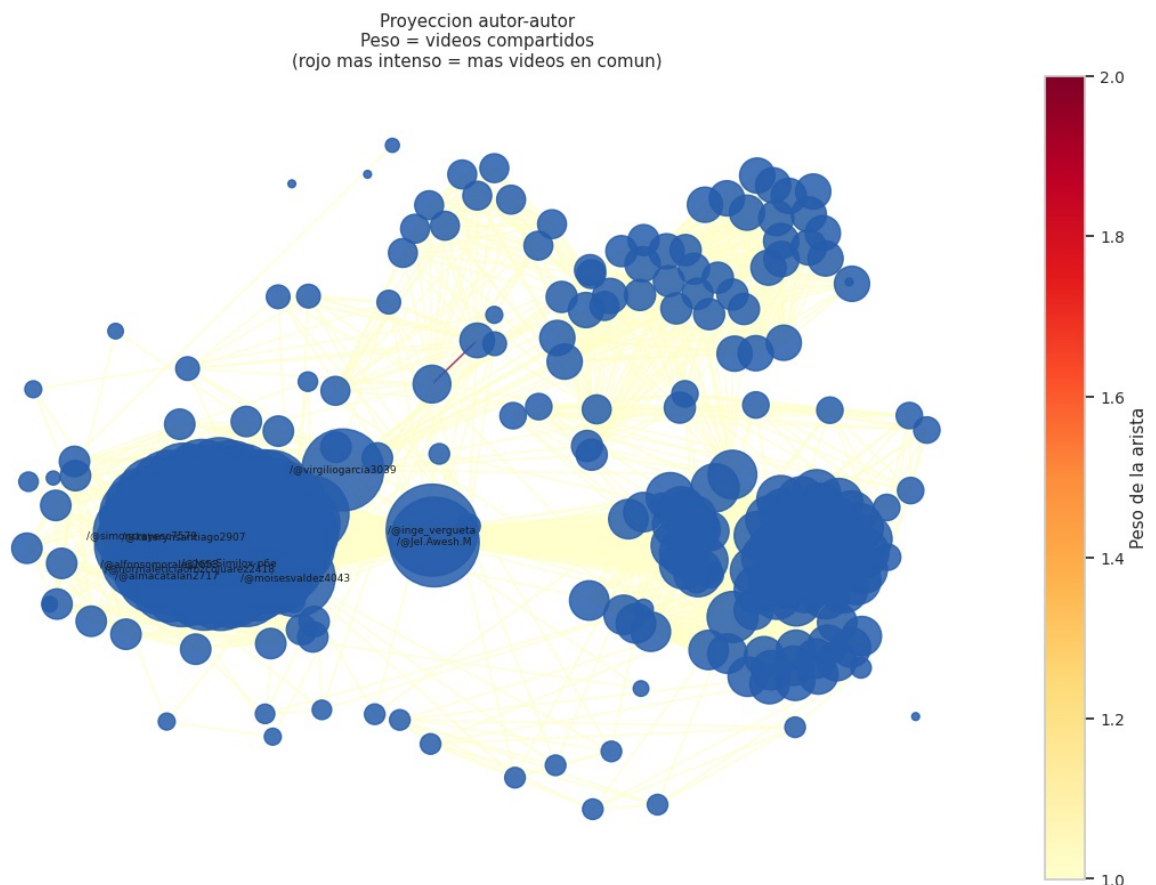
# Nodos aislados y conectados: todos se dibujan (no se elimina estructura)
colores_nodo = ["#275DAD" if con_nodos_autor else "#E76F51"] * P.number_of_nodes()
nx.draw_networkx_nodes(P, pos,
                       node_size=[24 if grados[n] == 0 else 60 + 18 * grados[n] for n in
P.nodes],
                       node_color=colores_nodo, alpha=0.85, ax=ax)

# Etiquetas: nodos conectados de mayor grado
top = sorted([n for n in P.nodes if grados[n] > 0], key=lambda n: grados[n], reverse=True)[:10]
etiquetas = {}
for n in top:
    if con_nodos_autor:
        etiquetas[n] = handle_autor.get(n, n) if n in handle_autor else str(n)
    else:
        etiquetas[n] = acortar(titulo_video.get(n, n))
nx.draw_networkx_labels(P, pos, etiquetas, font_size=6.5, ax=ax)

sm = plt.cm.ScalarMappable(cmap="YlOrRd", norm=norm)
sm.set_array([])
cbar = plt.colorbar(sm, ax=ax, fraction=0.04, pad=0.02)
cbar.set_label("Peso de la arista")
ax.set_title(titulo)
ax.axis("off")
plt.tight_layout()
plt.show()

dibujar_proyeccion(proj_autor_autor,
                   "Proyeccion autor-autor\nPeso = videos compartidos\n(rojo mas intenso = mas
videos en comun)",
                   con_nodos_autor=True)

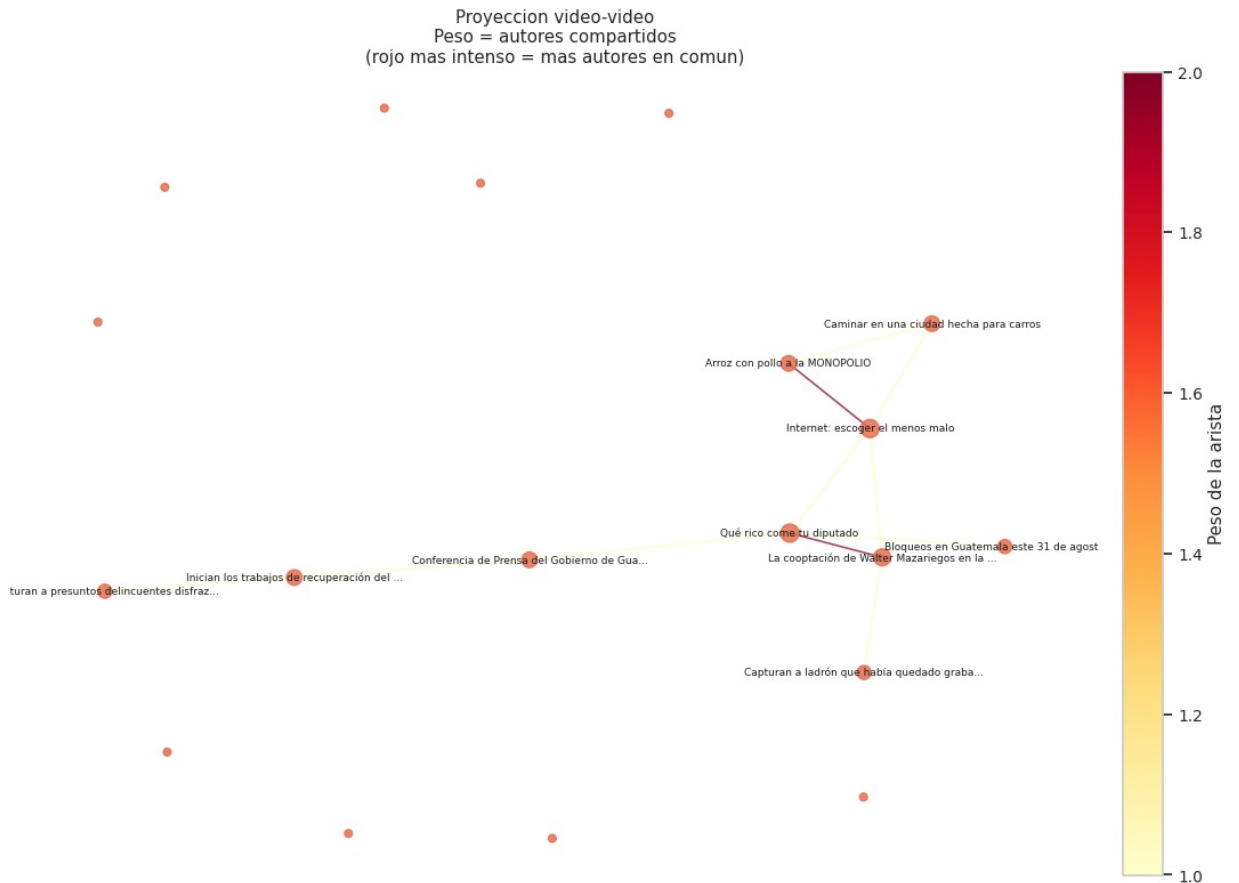
```



```

In [67]: dibujar_proyeccion(proj_video_video,
                           "Proyeccion video-video\nPeso = autores compartidos\n(rojo mas intenso = mas
autores en comun)",
                           con_nodos_autor=False)

```

```
In [68]: # Guardar tablas de proyecciones
pares_aa.to_csv("resultados/tablas/proyeccion_autor_autor_pares.csv", index=False, encoding="utf-8-sig")
pares_vv.to_csv("resultados/tablas/proyeccion_video_video_pares.csv", index=False, encoding="utf-8-sig")
comparacion_proy.to_csv("resultados/tablas/proyecciones_comparacion.csv", index=False, encoding="utf-8-sig")
print("Tablas de proyecciones guardadas en resultados/tablas/")
```

Tablas de proyecciones guardadas en resultados/tablas/

6. Topología y fragmentación

Se analiza la **estructura** de la red bipartita y de sus dos proyecciones: cuántos nodos y aristas hay, qué tan concentrados están los grados, cómo se fragmenta el grafo y qué tan cohesionado está.

Precaución metodológica: la red bipartita no contiene triángulos por definición (un ciclo solo puede tener longitud par). Por eso la **transitividad** y el **coeficiente de agrupamiento no son informativos sobre la red bipartita** y se analizan sobre las proyecciones.

6.1 Nodos, aristas, densidad, grado medio y distribución de grados

```
In [69]: def metricas_topologicas(P, nombre):
conexos = list(nx.connected_components(P))
grados = pd.Series([d for _, d in P.degree()])
comp_mayor = max((len(c) for c in conexos), default=0)
return pd.DataFrame([{"red": nombre,
"nodos": P.number_of_nodes(),
"aristas": P.number_of_edges(),
"densidad": round(nx.density(P), 5),
"grado_medio": round(2 * P.number_of_edges() / P.number_of_nodes(), 3) if
P.number_of_nodes() else 0,
"grado_mediana": float(grados.median()),
"grado_maximo": int(grados.max()),
"grado1_pct": round(100 * (grados == 1).mean(), 2),
```



```

"sesgo_grados": round(float(grados.skew()), 3),
"componentes": len(conexos),
"componente_mayor_nodos": comp_mayor,
"componente_mayor_pct": round(100 * comp_mayor / P.number_of_nodes(), 2),
})

topologia = pd.concat([
    metricas_topologicas(G, "bipartita autor-video"),
    metricas_topologicas(proj_autor_autor, "proyeccion autor-autor"),
    metricas_topologicas(proj_video_video, "proyeccion video-video"),
], ignore_index=True)
print("TOPOLOGIA DE LAS TRES REDES")
print(topologia.to_string(index=False))

```

TOPOLOGIA DE LAS TRES REDES

	red	nodos	aristas	densidad	grado_medio	grado_mediana	grado_maximo	grado1_pct
sesgo_grados	componentes	componente_mayor_nodos	componente_mayor_pct					
bipartita autor-video	351	343	0.00558	1.954	1.0	128	93.16	
13.509	10	286	81.48					
proyeccion autor-autor	332	10732	0.19532	64.651	48.0	183	0.60	
0.340	10	276	83.13					
proyeccion video-video	19	11	0.06433	1.158	1.0	4	15.79	
0.948	10	10	52.63					

```

In [70]: # Distribucion de grados (histogramas por red y por tipo de nodo)
grados_bip_autores = [G.degree(n) for n in autores_lista]
grados_bip_videos = [G.degree(n) for n in videos_lista]
grados_aa = [d for _, d in proj_autor_autor.degree()]
grados_vv = [d for _, d in proj_video_video.degree()]

fig, axes = plt.subplots(2, 2, figsize=(13, 9))
axes[0, 0].hist(grados_bip_autores, bins=range(0, max(grados_bip_autores) + 2), edgecolor="white",
color="#275DAD")
axes[0, 0].set_title("Bipartita: grado de AUTORES")
axes[0, 0].set_xlabel("Grado (videos en que comento)")
axes[0, 0].set_ylabel("Autores")
l0 = axes[0, 0].axvline(np.median(grados_bip_autores), color="#C1121F", ls="--", lw=1.2,
label="mediana")
axes[0, 0].legend()

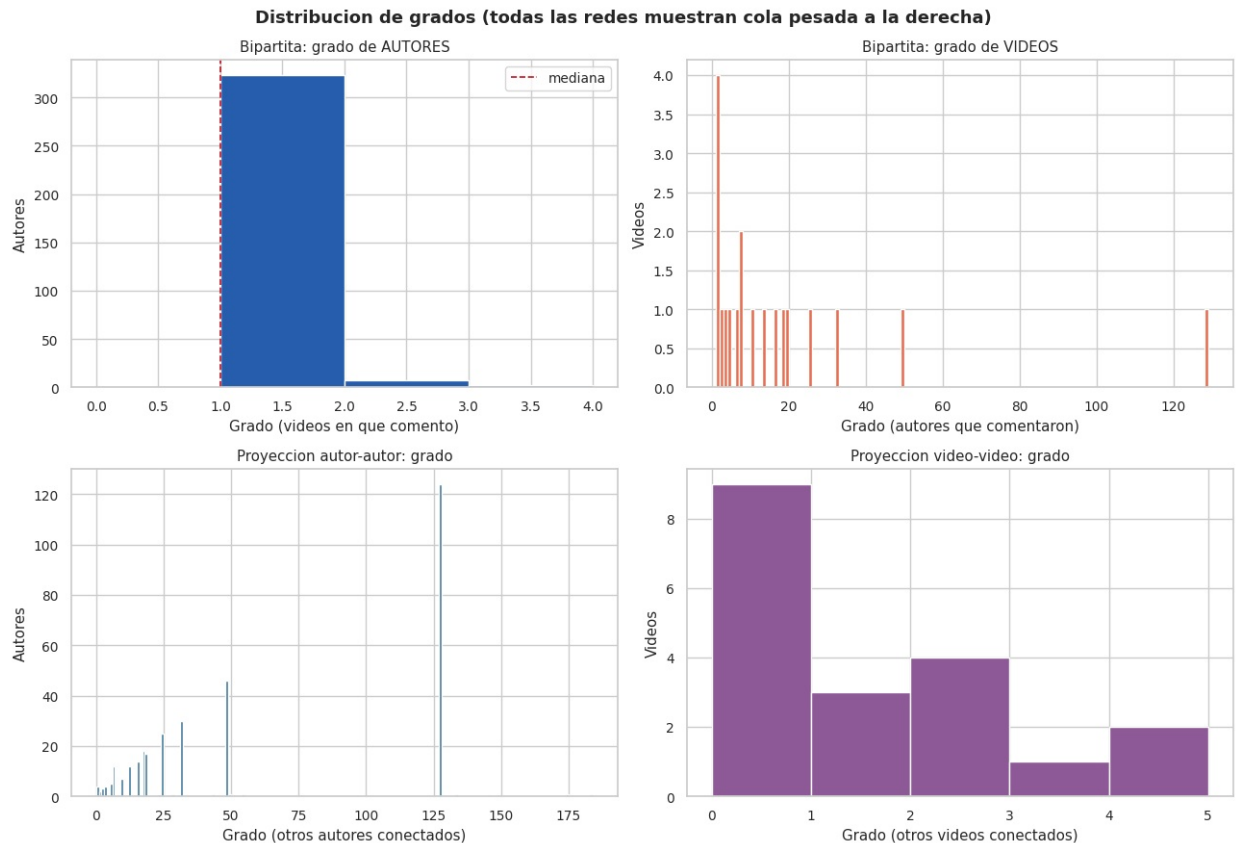
axes[0, 1].hist(grados_bip_videos, bins=range(0, max(grados_bip_videos) + 2), edgecolor="white",
color="#E76F51")
axes[0, 1].set_title("Bipartita: grado de VIDEOS")
axes[0, 1].set_xlabel("Grado (autores que comentaron)")
axes[0, 1].set_ylabel("Videos")

axes[1, 0].hist(grados_aa, bins=range(0, max(grados_aa) + 2), edgecolor="white", color="#457B9D")
axes[1, 0].set_title("Proyeccion autor-autor: grado")
axes[1, 0].set_xlabel("Grado (otros autores conectados)")
axes[1, 0].set_ylabel("Autores")

axes[1, 1].hist(grados_vv, bins=range(0, max(grados_vv) + 2), edgecolor="white", color="#8D5A97")
axes[1, 1].set_title("Proyeccion video-video: grado")
axes[1, 1].set_xlabel("Grado (otros videos conectados)")
axes[1, 1].set_ylabel("Videos")

plt.suptitle("Distribucion de grados (todas las redes muestran cola pesada a la derecha)",
fontweight="bold")
plt.tight_layout()
plt.show()

```



```
In [71]: # Hubs: los pocos nodos que concentran las conexiones
video_hubs = nodos[nodos["tipo"] == "video"].sort_values("grado", ascending=False).head(6)
print("VIDEOS HUB (mayor grado = mas autores que comentaron)")
print(video_hubs[["etiqueta", "grado", "comentarios", "autores_unicos"]].to_string(index=False))
print()

autor_hubs = nodos[nodos["tipo"] == "autor"].sort_values("grado", ascending=False).head(6)
print("AUTORES HUB (mayor grado = mas videos en que comentaron)")
print(autor_hubs[["etiqueta", "grado", "videos_unicos", "comentarios"]].to_string(index=False))
```

VIDEOS HUB (mayor grado = mas autores que comentaron)

grado	comentarios	autores_unicos	etiqueta
128	161	128.0	Qué rico come tu diputado
49	50	49.0	La cooptación de Walter Mazariegos en la USAC
32	45	32.0	Inician los trabajos de recuperación del Puente Belice II.
25	25	25.0	Plan 2032 Ciudad de Guatemala
19	25	19.0	Conferencia de Prensa del Gobierno de Guatemala. #LaRondaGt
EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México Noticias Telemundo	25	18.0	

AUTORES HUB (mayor grado = mas videos en que comentaron)

etiqueta	grado	videos_unicos	comentarios
@inge_vergueta	3	3.0	3
@hashojea7348	3	3.0	4
@Alejandro00710	2	2.0	2
@franciscoflores3120	2	2.0	2
@MarcosCarillo-blr	2	2.0	2
@Jel.Awesh.M	2	2.0	3

Lectura de 6.1.

- **Bipartita autor-video:** 351 nodos y 343 aristas. Su densidad es bajísima (0.006) porque solo se observaron 343 de los 6 308 pares posibles. El grado medio es 1.95, la **mediana 1** y el **93.2 % de los nodos tiene grado 1**: casi todos los autores son *hojas* colgadas de un video. El sesgo de la distribución (13.5) es extremo: un solo video acumula grado 128 (autores) y apenas 9 autores comentan en más de un video. La topología es la de **estrellas concéntricas alrededor de los videos-hub**, no la de una

mall de usuarios conectados entre sí.

- **Proyección autor-autor:** con densidad 0.195 y grado medio 64.7, parece densa, pero esa densidad es un **artefacto de proyección**: al proyectar a través de los videos-hub, todos los autores que coincidieron en un mismo video quedan conectados entre sí. El 0.6 % de autores tiene grado 1 y el sesgo cae a 0.34.
- **Proyección video-video:** es escasa: 11 aristas entre 19 videos (densidad 0.064), grado medio 1.16, 15.8 % de videos con grado 1 y solo 2 triángulos. Refleja un solapamiento de audiencia débil.
- **Componentes y fragmentación:** las tres redes tienen 10 componentes conexos, con una **componente gigante** que domina en la bipartita (286 nodos, 81.5 %) y en autor-autor (276 nodos, 83.1 %), pero que apenas llega al 52.6 % en video-video. Las 9 componentes restantes son grupos pequeños (2 a 26 nodos) que corresponden a videos encontrados por consultas/canales específicos cuyos autores no se solapan con el resto de la muestra.

6.2 Cohesión y transitividad

- **Transitividad** (`nx.transitivity`): fracción de tríadas cerradas (triángulos) sobre las conectadas.
- **Coeficiente de agrupamiento medio** (`nx.average_clustering`): promedio del cierre local por nodo.
- **Núcleo máximo** (`k-core`): el subgrafo más grande en el que todo nodo tiene al menos k vecinos; mide cohesión.
- En la **bipartita**, la transitividad y el agrupamiento son 0 por construcción (no hay triángulos); por eso la cohesión se evalúa en las proyecciones.

```
In [72]: filas_cohesion = []
for nombre, P in (("bipartita autor-video", G),
                  ("proy. autor-autor", proj_autor_autor),
                  ("proy. video-video", proj_video_video)):
    filas_cohesion.append({
        "red": nombre,
        "transitividad": round(nx.transitivity(P), 5),
        "clustering_medio": round(nx.average_clustering(P), 5),
        "nucleo_maximo": max(nx.core_number(P).values()) if P.number_of_nodes() else 0,
        "triangulos": sum(nx.triangles(P).values()) // 3,
    })
cohesion = pd.DataFrame(filas_cohesion)
print("COHESION Y TRANSITIVIDAD")
print(cohesion.to_string(index=False))

# Tamano del nucleo maximo por red (cuantos nodos lo componen)
for nombre, P in (("bipartita", G), ("autor-autor", proj_autor_autor), ("video-video",
proj_video_video)):
    cores = nx.core_number(P)
    k_max = max(cores.values()) if cores else 0
    tam = sum(1 for v in cores.values() if v == k_max)
    print(f" Nucleo {nombre}: k={k_max}, nodos en el nucleo={tam}")
```

COHESION Y TRANSITIVIDAD

	red	transitividad	clustering_medio	nucleo_maximo	triangulos
bipartita	autor-video	0.00000	0.00000	2	0
proy.	autor-autor	0.98403	0.97205	127	369906
proy.	video-video	0.31579	0.14912	2	2

Nucleo bipartita: k=2, nodos en el nucleo=8

Nucleo autor-autor: k=127, nodos en el nucleo=128

Nucleo video-video: k=2, nodos en el nucleo=5

Lectura de 6.2.

- La **bipartita** tiene transitividad y agrupamiento 0: no hay triángulos por ser bipartita. Aun así, su núcleo máximo es $k=2$: existe un subgrafo (el núcleo formado por los videos activos y los autores recurrentes) en el que cada nodo tiene al menos 2 vecinos. Esto indica cohesión **localizada** alrededor de los videos-hub, no homogénea.
- La **proyección autor-autor** es casi **perfectamente transitiva**: 0.984 de transitividad y 0.972 de agrupamiento medio, con ~370 mil triángulos y un núcleo $k=127$. Un valor así no describe una comunidad social densamente entrelazada, sino el **efecto de co-presencia**: al proyectar a través de pocos videos-hub, casi cualquier par de autores que comparte un tercero también comparte uno de esos videos. Es cohesión aparente por concentración de la muestra.

- La **proyección video-video** tiene transitividad baja (0.316) y agrupamiento medio 0.149, con solo **2 triángulos** y núcleo k=2: los pocos videos que comparten autores forman conexiones sueltas y casi no hay terceros cerrados. Estructuralmente es la red **menos cohesionada** de las tres.

6.3 Autores, videos y grupos periféricos o aislados

Se distingue entre:

- **Aislamiento observado:** el nodo existe en la red y no tiene los vínculos que la métrica requiere (p. ej., no comparte videos con otro autor).
- **Ausencia de datos:** el nodo no está en la red porque la muestra no contiene sus comentarios (videos del archivo sin comentarios recolectados). No debe leerse como aislamiento real.

```
In [73]: # --- Perifericos en la bipartita: hojas (grado 1) ---
hojas_autores = [n for n in autores_lista if G.degree(n) == 1]
hojas_videos = [n for n in videos_lista if G.degree(n) == 1]
print(f"PERIFERICOS EN LA BIPARTITA (grado 1)")
print(f"  autores hoja: {len(hojas_autores)} de {len(autores_lista)}")
print(f"({100*len(hojas_autores)/len(autores_lista):.1f}%)")
print(f"  videos hoja : {len(hojas_videos)} de {len(videos_lista)}")

# --- Aislados observados en las proyecciones ---
aislados_aa = [n for n in proj_autor_autor.nodes if proj_autor_autor.degree(n) == 0]
aislados_vv = [n for n in proj_video_video.nodes if proj_video_video.degree(n) == 0]
print()
print(f"AISLADOS OBSERVADOS EN PROYECCIONES")
print(f"  autores aislados en autor-autor: {len(aislados_aa)}")
print(f"({100*len(aislados_aa)/len(autores_lista):.2f}%)")
print(pd.DataFrame({"handle": [handle_autor.get(n, n) for n in
aislados_aa]}).to_string(index=False))
print(f"  videos aislados en video-video: {len(aislados_vv)}")
print(f"({100*len(aislados_vv)/len(videos_lista):.1f}%)")
print(pd.DataFrame({"titulo": [str(titulo_video.get(n, n))[:60] for n in
aislados_vv]}).to_string(index=False))
```

PERIFERICOS EN LA BIPARTITA (grado 1)

autores hoja: 323 de 332 (97.3%)

videos hoja : 4 de 19

AISLADOS OBSERVADOS EN PROYECCIONES

autores aislados en autor-autor: 4 (1.20%)

handle

/@jahirdiaz5862

/@elizabethmonterroso7719

/@elioelias7267

/@aditalissethevasquezrojas8039

videos aislados en video-video: 9 (47.4%)

titulo

Noticiero en Directo 1 pm, 28 de Agosto de 2026

10 Preguntas a un año del Paro Nacional

EE.UU. envía a mexicanos deportados a Guatemala antes de su

I'x K'at: el primer equipo guatemalteco de pelota maya, conf

SHAI WA: la vecina queer de Casa Presidencial

Cruzando la ciudad a puro Transmetro

Plan 2032 Ciudad de Guatemala

¿Quiénes pagan más en Centroamérica?

Edén por Salud: empleo inclusivo para personas con discapaci

```
In [74]: # --- Ausencia de datos: videos del archivo sin comentarios en la muestra ---
videos_con_comentarios = set(comments["video_id"].unique())
videos_sin_observacion = videos[~videos["video_id"].isin(videos_con_comentarios)]
print(f"Videos del archivo SIN comentarios recolectados: {len(videos_sin_observacion)} de {len(videos)}")
print("-> NO aparecen en la red y NO son 'aislados': es ausencia de datos (cobertura de recoleccion).")

# Clasificacion de videos segun presencia en la red y tipo de aislamiento
clasif_videos = pd.DataFrame([
    {"categoria": "con comentarios y conectado (comparte audiencia)",
     "videos": int(len(videos_lista) - len(aislados_vv))},
    {"categoria": "con comentarios pero aislado en video-video (sin autores compartidos)",
     "videos": len(aislados_vv)},
    {"categoria": "sin comentarios en la muestra (ausencia de datos)",
```

```
"videos": len(videos_sin_observacion)},
])
print()
print(clasif_videos.to_string(index=False))
```

Videos del archivo SIN comentarios recolectados: 274 de 293

-> NO aparecen en la red y NO son 'aislados': es ausencia de datos (cobertura de recolección).

	categoria	videos
con comentarios y conectado (comparte audiencia)		10
con comentarios pero aislado en video-video (sin autores compartidos)		9
sin comentarios en la muestra (ausencia de datos)		274

Lectura de 6.3.

- **Periféricos en la bipartita:** el **97.3 %** de los autores (323 de 332) son hojas (participaron con uno o más comentarios, pero dentro de un único video). El 93.2 % es la proporción de hojas sobre **todos** los nodos de la red (autores y videos). Es el patrón de participación *puntual*: la gran mayoría de la audiencia comenta una vez y no vuelve a aparecer en la muestra. Solo **9 autores** tienen grado ≥ 2 y son los únicos conectores entre contenidos.
- **Aislados observados en las proyecciones:** 4 autores no comparten ningún video con otro autor de la muestra (su (sus) video(s) no tuvieron otro comentarista), y **9 de 19 videos** no comparten autores con ningún otro video. Son aislados *dentro de la muestra observada*: su aislamiento podría cambiar si se recolectaran más comentarios.
- **Ausencia de datos vs aislamiento: 274 de 293** videos del archivo no tienen comentarios en `youtube_comments.csv`. No fueron excluidos por estar aislados; simplemente la recolección no recuperó comentarios para ellos (las consultas/canales solo cubren una selección de videos). Confundir "sin comentarios en la muestra" con "sin audiencia" sería un error de interpretación.

6.4 Explicación de los hallazgos estructurales

Los resultados de 6.1–6.3 cuentan una historia coherente con el diseño de recolección:

1. **Concentración extrema, no comunidad.** La bipartita es un conjunto de estrellas: 93 % de autores-hoja, un video-hub de grado 128, sesgo 13.5. La participación no se distribuye como una malla social, sino que se *cuelga* de los pocos videos más activos. Esto es congruente con la concentración de comentarios ya observada en la sección 3.2.
2. **La proyección autor-autor es densa por artefacto.** Su transitividad (0.98) y agrupamiento (0.97) son altísimos, pero la interpretación correcta es que todo se proyecta a través de los videos-hub: no hay evidencia de relaciones estables entre usuarios. La alta transitividad describe *co-presencia en pocos contenidos*, no cohesión social.
3. **Fragmentación con una componente gigante.** En la bipartita y en autor-autor, una componente gigante (≈ 81 -83 % de los nodos) agrupa a casi todos, mientras las 9 componentes pequeñas corresponden a videos de consultas/canales específicos sin solapamiento de autores. La proyección video-video queda al 52.6 %: la mitad de los videos con comentarios no comparte audiencia con otros.
4. **Cohesión localizada, no global.** El núcleo máximo de la bipartita es solo $k=2$; en video-video también $k=2$ con 2 triángulos. Solo la proyección autor-autor alcanza un núcleo enorme ($k=127$), otra vez por el efecto de co-presencia. En términos estructurales, la muestra está *conectada pero no cohesionada*: hay un esqueleto que une a casi todos, pero sostenido por muy pocos videos y unos pocos autores recurrentes.
5. **Leer la ausencia con cuidado.** 274 videos sin comentarios en la muestra y 9 videos sin solapamiento de audiencia no son lo mismo. La fragmentación observada mezcla estructura real de la muestra con los límites de cobertura; no debe extrapolarse a "audiencias separadas" en YouTube.

Gráficos que respaldan estas conclusiones: la malla de la sección 4.4 (estrellas alrededor de videos), las curvas de concentración de 3.2, las proyecciones de 5.4 (dense núcleo en autor-autor, red dispersa en video-video) y las distribuciones de grado de 6.1 (cola pesada, mediana 1).

```
In [75]: # Exportar tablas de topologia
topologia.to_csv("resultados/tablas/topologia_metricas.csv", index=False, encoding="utf-8-sig")
```

```
cohesion.to_csv("resultados/tablas/topologia_cohesion.csv", index=False, encoding="utf-8-sig")
print("RUTA DEL CUADERNO COMPLETADA (secciones 1 a 7).")
print(f"Red bipartita: {G.number_of_nodes()} nodos, {G.number_of_edges()} aristas,
{len(componentes)} componentes conexos.")
```

RUTA DEL CUADERNO COMPLETADA (secciones 1 a 7).

Red bipartita: 351 nodos, 343 aristas, 10 componentes conexos.

7. Comunidades

La detección de comunidades busca **subgrupos de nodos más conectados entre sí que con el resto** de la red. En este laboratorio interesa encontrar *grupos de usuarios que participan juntos y grupos de videos con audiencias solapadas*.

7.1 Selección de la red adecuada

Opción descartada: aplicar Louvain directamente a la bipartita. La modularidad (objetivo que optimiza Louvain) asume cierre de triadas: separa la red en regiones con más *triángulos* internos que esperables al azar. Como una bipartita **no tiene triángulos**, la modularidad degenera y las comunidades resultantes **mezclan autores y videos en el mismo grupo** (se demuestra empíricamente en 7.2), lo que impide interpretar "comunidades de usuarios".

Opción elegida (principal): proyección autor-autor ponderada. Es la forma estándar de estudiar comunidades de usuarios en redes bipartitas:

- Los nodos son **autores** y la arista pesada indica **co-participación** (peso = videos compartidos).
- Conserva el fenómeno de interés (quiénes frecuentan los mismos contenidos) y permite usar métodos basados en triángulos.
- Las comunidades resultantes se interpretan como **grupos de usuarios que comentan en el mismo conjunto de videos**.

Opción complementaria: proyección video-video ponderada. Agrupa videos que **comparten audiencia**; sus comunidades se leen como **grupos temáticos de contenido**. Se incluye con fines comparativos, conscientes de que su red es muy pequeña (19 nodos, 11 aristas).

Tratamiento de los pesos. Louvain ponderado (`weight='weight'`) usa el peso como fuerza del vínculo en la ganancia de modularidad y en su cálculo. En la proyección autor-autor, 10 730 de 10 732 aristas tienen peso 1, por lo que la ponderación es casi indiferente; se documenta por transparencia y porque el procedimiento es correcto aunque el efecto sea pequeño.

7.2 Algoritmo seleccionado: Louvain

Louvain (Blondel et al., 2008) es uno de los métodos más usados para redes grandes:

1. **Fase de movimiento local:** cada nodo se asigna a la comunidad de un vecino si el incremento de modularidad ponderada es positivo; se repite hasta converger.
2. **Fase de agregación:** cada comunidad del paso anterior se convierte en un supernodo y se repite el proceso sobre la red reducida.
3. Devuelve una partición (no solapante) que maximiza la **modularidad** (densidad interna – densidad esperada bajo modelo nulo de configuraciones).

Supuestos y limitaciones que se documentan:

- La calidad de una partición se mide con **modularidad ponderada**; un valor alto ($> \sim 0.3$) indica agrupamiento estructural, pero aquí debe leerse como co-presencia en contenidos, no como afinidad social.
- **Parámetro de resolución = 1.0** (default): balancea comunidades grandes y pequeñas.
- El orden de visita de los nodos introduce **aleatoriedad**; se fija `random_state=3066` para reproducibilidad: el particionado (tamaños y modularidad) es estable, aunque las **etiquetas numéricas** de cada comunidad pueden permutar entre ejecuciones por el orden de iteración del grafo. Por eso el análisis y la interpretación describen las comunidades por su tamaño y su video-tema, no por su número.

- No usa atributos de nodos (categoría, sentimiento) ni dirección; esos atributos se incorporan *después* para **caracterizar** cada comunidad.

Alternativas investigadas: Leiden (Traag et al., 2019) corrige la *desconexión* de comunidades de Louvain pero requiere la extensión `leidenalg` (dependencia con compilación nativa). Para este cuaderno se priorizó Louvain por ser el más difundido, puro Python y reproducible con `python-louvain`. Una alternativa específica para bipartitas es la **modularidad de Barber** con Louvain bipartito, no incluida por simplicidad y porque la proyección ya resuelve el problema de interpretación.

```
In [76]: # Louvain (python-louvain) sobre la proyeccion autor-autor ponderada
import community as community_louvain

part_aa = community_louvain.best_partition(proj_autor_autor, weight="weight", random_state=3066)
mod_aa = community_louvain.modularity(part_aa, proj_autor_autor, weight="weight")

sizes = pd.Series(part_aa).value_counts().sort_index()
print("LOUVAIN SOBRE PROYECCION AUTOR-AUTOR (peso = videos compartidos)")
print("  comunidades:", len(sizes), "| autores por comunidad:", sizes.tolist())
print("  modularidad ponderada:", round(mod_aa, 4))
```

```
LOUVAIN SOBRE PROYECCION AUTOR-AUTOR (peso = videos compartidos)
comunidades: 14 | autores por comunidad: [61, 18, 2, 126, 6, 25, 4, 1, 55, 1, 1, 28, 1, 3]
modularidad ponderada: 0.3949
```

```
In [77]: # Louvain sobre la proyeccion video-video ponderada (complementaria)
part_vv = community_louvain.best_partition(proj_video_video, weight="weight", random_state=3066)
mod_vv = community_louvain.modularity(part_vv, proj_video_video, weight="weight")

sizes_vv = pd.Series(part_vv).value_counts().sort_index()
print("LOUVAIN SOBRE PROYECCION VIDEO-VIDEO (peso = autores compartidos)")
print("  comunidades:", len(sizes_vv), "| videos por comunidad:", sizes_vv.tolist())
print("  modularidad ponderada:", round(mod_vv, 4))
```

```
LOUVAIN SOBRE PROYECCION VIDEO-VIDEO (peso = autores compartidos)
comunidades: 12 | videos por comunidad: [4, 1, 3, 1, 1, 1, 1, 1, 1, 1, 3, 1]
modularidad ponderada: 0.4053
```

```
In [78]: # Demostracion del problema de aplicar Louvain a la bipartita
part_bip = community_louvain.best_partition(G, weight="peso", random_state=3066)
mod_bip = community_louvain.modularity(part_bip, G, weight="peso")

com_tipos = {}
for nodo, com in part_bip.items():
    com_tipos.setdefault(com, set()).add("autor" if nodo.startswith("autor::") else "video")
mixtas = sum(1 for tipos in com_tipos.values() if len(tipos) > 1)

print("LOUVAIN APLICADO DIRECTAMENTE A LA BIPARTITA")
print("  comunidades:", len(com_tipos), "| modularidad:", round(mod_bip, 4))
print(f"  comunidades que MEZCLAN autores y videos: {mixtas} de {len(com_tipos)}")
print("  -> Confirma que la proyeccion es la eleccion adecuada para estudiar comunidades de usuarios.")
```

```
LOUVAIN APLICADO DIRECTAMENTE A LA BIPARTITA
comunidades: 17 | modularidad: 0.7774
comunidades que MEZCLAN autores y videos: 17 de 17
-> Confirma que la proyeccion es la eleccion adecuada para estudiar comunidades de usuarios.
```

7.3 Número de comunidades, tamaños y calidad

La modularidad es aplicable aquí porque Louvain la maximiza explícitamente sobre los pesos de cada proyección; se reporta como medida de calidad del particionado junto con el número de comunidades y sus tamaños.

```
In [79]: # Reporte consolidado: numero de comunidades, tamanos y modularidad (autor-autor y video-video)
reporte_comunidades = pd.DataFrame({
    "proyeccion": ["autor-autor", "video-video"],
    "nodos": [proj_autor_autor.number_of_nodes(), proj_video_video.number_of_nodes()],
    "n_comunidades": [int(len(sizes)), int(len(sizes_vv))],
    "tamano_min": [int(sizes.min()), int(sizes_vv.min())],
    "tamano_mediana": [float(sizes.median()), float(sizes_vv.median())],
    "tamano_max": [int(sizes.max()), int(sizes_vv.max())],
    "pct_en_mayor_comunidad": [round(100 * sizes.max() / proj_autor_autor.number_of_nodes(), 2),
                                round(100 * sizes_vv.max() / proj_video_video.number_of_nodes(), 2)],
    "modularidad_louvain": [round(float(mod_aa), 4), round(float(mod_vv), 4)],
```

```

})
print("REPORTE DE COMUNIDADES (Louvain, modularidad ponderada)")
print(reporte_comunidades.to_string(index=False))
print()
print("Tamanos autor-autor:", sorted(sizes.values, reverse=True))
print("Tamanos video-video:", sorted(sizes_vv.values, reverse=True))

```

```

REPORTE DE COMUNIDADES (Louvain, modularidad ponderada)
proyeccion nodos n_comunidades tamaño_min tamaño_mediana tamaño_max pct_en_mayor_comunidad
modularidad_louvain
autor-autor 332 14 1 5.0 126 37.95
0.3949
video-video 19 12 1 1.0 4 21.05
0.4053

```

```

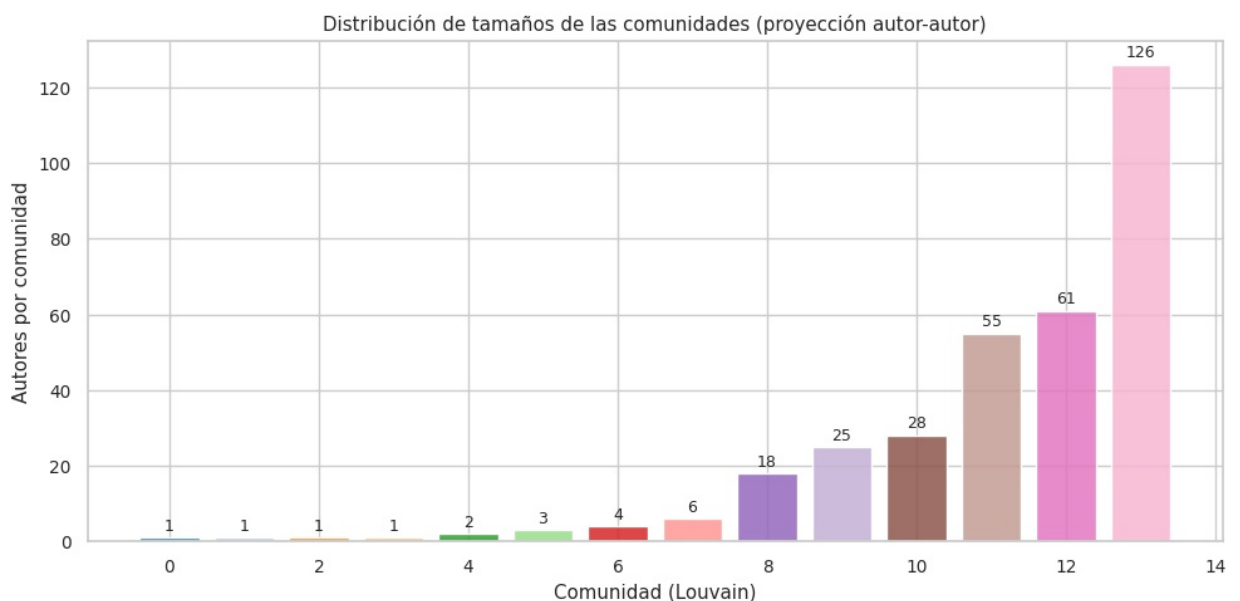
Tamanos autor-autor: [np.int64(126), np.int64(61), np.int64(55), np.int64(28), np.int64(25),
np.int64(18), np.int64(6), np.int64(4), np.int64(3), np.int64(2), np.int64(1), np.int64(1),
np.int64(1), np.int64(1)]
Tamanos video-video: [np.int64(4), np.int64(3), np.int64(3), np.int64(1), np.int64(1), np.int64(1),
np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]

```

```

In [80]: # Distribucion de tamanos de las comunidades (proyeccion autor-autor)
import numpy as np
sizes_ordenado = sizes.sort_values()
colores74 = plt.cm.tab20(np.arange(len(sizes_ordenado)) % 20).copy()
colores74[:, 3] = 0.85
fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(range(len(sizes_ordenado)), sizes_ordenado.values, color=colores74)
for x, v in enumerate(sizes_ordenado.values):
    ax.text(x, v + 2, str(int(v)), ha="center", fontsize=9)
ax.set_title("Distribución de tamaños de las comunidades (proyección autor-autor)")
ax.set_xlabel("Comunidad (Louvain)")
ax.set_ylabel("Autores por comunidad")
plt.tight_layout()
plt.show()

```



7.4 Visualización de todas las comunidades y análisis de las tres comunidades principales

Se colorean los nodos según la comunidad detectada por Louvain (proyección autor-autor y proyección video-video). Se mantienen todas las aristas; solo se limitan las etiquetas.

```

In [81]: # Visualizacion de comunidades en la proyeccion autor-autor
fig, ax = plt.subplots(figsize=(12, 9))
pos_aa = nx.spring_layout(proj_autor_autor, seed=3066, k=0.35)

nx.draw_networkx_edges(proj_autor_autor, pos_aa, width=0.4, edge_color="#D8D8D8", alpha=0.25, ax=ax)
nodos_aa = list(proj_autor_autor.nodes)
colores_com = [plt.cm.tab20(part_aa.get(n, 0) % 20) for n in nodos_aa]
nx.draw_networkx_nodes(proj_autor_autor, pos_aa, nodelist=nodos_aa, node_color=colores_com,
                        node_size=28, alpha=0.85, linewidths=0, ax=ax)

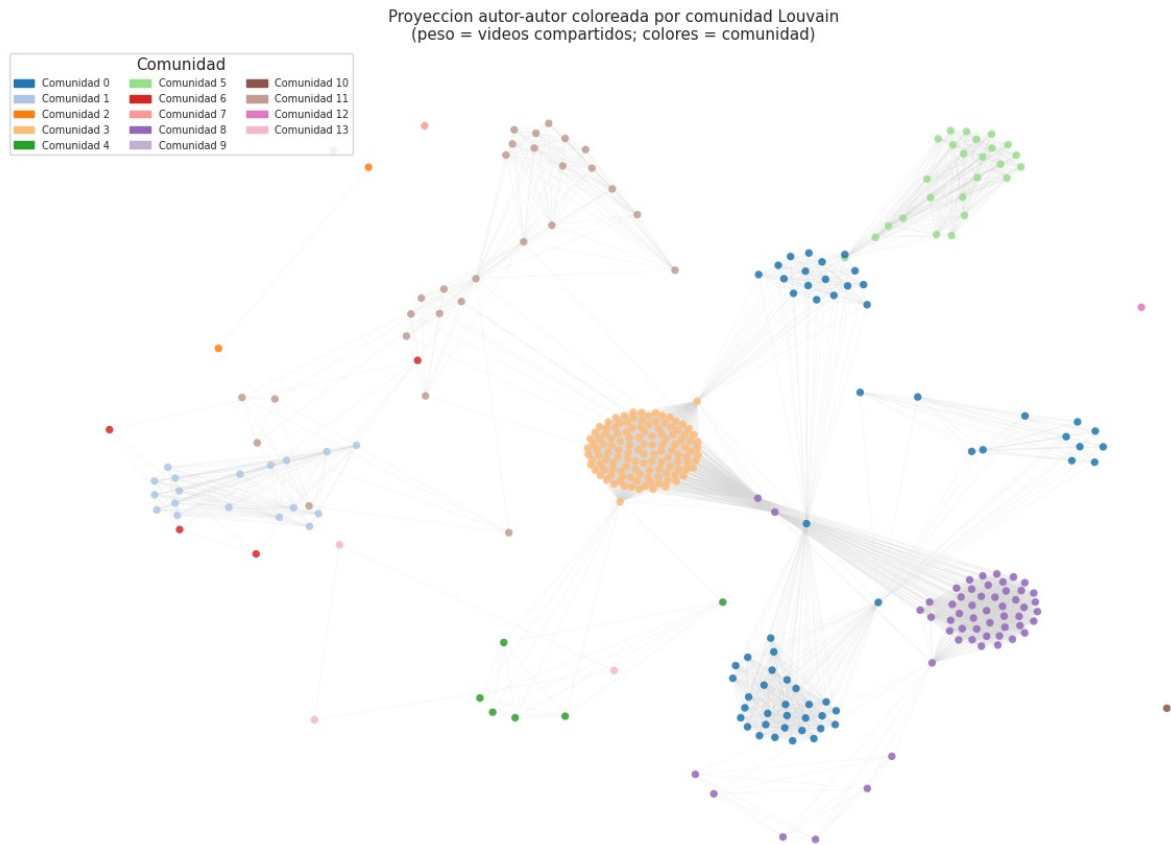
```



```

import matplotlib.patches as mpatches
leyendas = [mpatches.Patch(color=plt.cm.tab20(k % 20), label=f"Comunidad {k}")
             for k in sorted(set(part_aa.values()))]
ax.legend(handles=leyendas, title="Comunidad", fontsize=7, ncol=3, loc="upper left")
ax.set_title("Proyeccion autor-autor coloreada por comunidad Louvain\n(peso = videos compartidos; colores = comunidad)")
ax.axis("off")
plt.tight_layout()
plt.show()

```

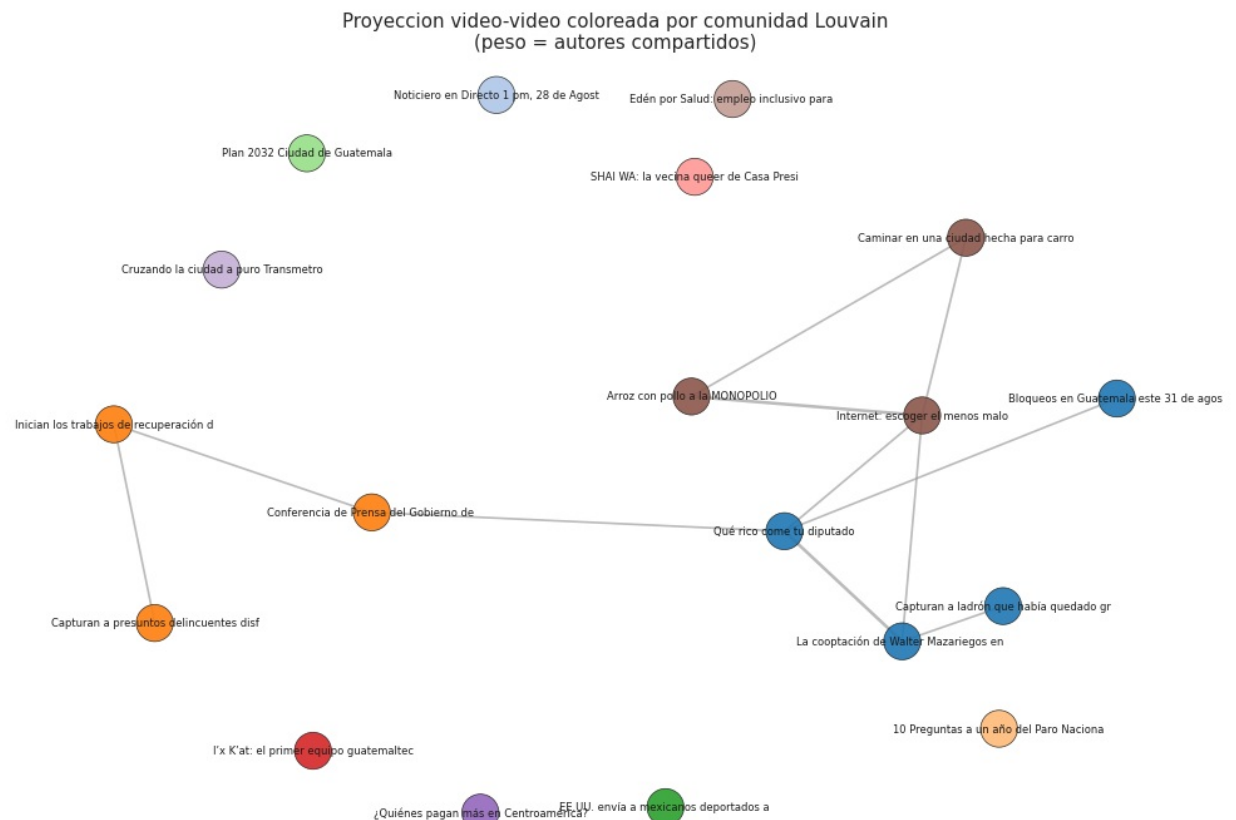


```

In [82]: # Visualizacion de comunidades en la proyeccion video-video
fig, ax = plt.subplots(figsize=(10, 7))
pos_vv = nx.spring_layout(proj_video_video, seed=3066, k=1.2)

pesos_vv_vis = nx.get_edge_attributes(proj_video_video, "weight")
nx.draw_networkx_edges(proj_video_video, pos_vv,
                      width=[0.6 + 0.6 * w for w in pesos_vv_vis.values()],
                      edge_color="#9E9E9E", alpha=0.7, ax=ax)
nodos_vv = list(proj_video_video.nodes)
colores_vv = [plt.cm.tab20(part_vv.get(n, 0) % 20) for n in nodos_vv]
nx.draw_networkx_nodes(proj_video_video, pos_vv, nodelist=nodos_vv, node_color=colores_vv,
                      node_size=450, alpha=0.9, edgecolors="#1D1D1D", linewidths=0.5, ax=ax)
labels_vv = {n: str(titulo_video.get(n, n))[:38] for n in nodos_vv}
nx.draw_networkx_labels(proj_video_video, pos_vv, labels_vv, font_size=6, ax=ax)
ax.set_title("Proyeccion video-video coloreada por comunidad Louvain\n(peso = autores compartidos)")
ax.axis("off")
plt.tight_layout()
plt.show()

```



Las **14 comunidades de autor-autor** tienen tamaños variados (mayor: 126 autores), mientras que en **video-video** solo **3 comunidades** agrupan más de un video (tamaños 4, 3 y 3); las 9 restantes son videos aislados como comunidades de un solo nodo. Se analizan hasta tres comunidades principales en cada proyección y se justifica el tratamiento de las demás.

```
In [83]: # Evidencia numerica de las tres comunidades principales (proyeccion autor-autor)
mapa_autor_com = nodos[nodos["tipo"] == "autor"].set_index("id_original")
[["node_id"].map(part_aa).to_dict()
tmp_ev = comments.copy()
tmp_ev["comunidad_ev"] = tmp_ev["author_channel_id"].map(mapa_autor_com)
video_principal_ev = (tmp_ev.dropna(subset=["comunidad_ev"])
                        .groupby(["comunidad_ev", "video_title"])
                        .size().reset_index(name="comentarios"))
video_principal_ev = (video_principal_ev
                      .sort_values(["comunidad_ev", "comentarios"], ascending=[True, False])
                      .groupby("comunidad_ev").head(1).reset_index(drop=True))

tamanos_ev = pd.Series(part_aa).value_counts().sort_index()
principales = tamanos_ev.nlargest(3).reset_index()
principales.columns = ["comunidad", "autores"]
principales = principales.merge(video_principal_ev, left_on="comunidad", right_on="comunidad_ev",
                               how="left")
print("TRES COMUNIDADES PRINCIPALES (autor-autor)")
print(principales[["comunidad", "autores", "video_title", "comentarios"]].to_string(index=False))
print()
print("Comunidades video-video con mas de 1 video:", sorted(sizes_vv[sizes_vv > 1].values))
print("(resto: 9 comunidades de 1 solo video; se analizan las 3 disponibles y se justifica)")
```

TRES COMUNIDADES PRINCIPALES (autor-autor)

comunidad	autores	video_title	comentarios
3	126	Qué rico come tu diputado	159
0	61	Inician los trabajos de recuperación del Puente Belice II.	45
8	55	La cooptación de Walter Mazariegos en la USAC	50

Comunidades video-video con mas de 1 video: [np.int64(3), np.int64(3), np.int64(4)]
(resto: 9 comunidades de 1 solo video; se analizan las 3 disponibles y se justifica)

Análisis de las tres comunidades principales (proyección autor-autor)

- **Comunidad más grande — "Qué rico come tu diputado" (126 autores, 37.9 % de la audiencia).**
Es el video-hub con **grado 128 en la red bipartita** y 159 comentarios (39.7 % del total). Sus términos

más frecuentes son *pueblo, dinero, diputados, trabajo, diputado, sueldo* y su sentimiento medio es **0.04** (neutral apenas positivo). Es la comunidad mayor porque el video-hub centraliza casi toda su discusión.

- **Segunda comunidad — "Inician los trabajos de recuperación del Puente Belice II" (61 autores).** Junto a "*Conferencia de Prensa del Gobierno de Guatemala*". Sus términos (*presidente, guatemala, país, años, arevalo, bernardo*) y su sentimiento medio **0.11** reflejan una audiencia atenta a la actualidad presidencial con valoración mayormente positiva.
- **Tercera comunidad — "La cooptación de Walter Mazariegos en la USAC" (55 autores).** Junto a "*Capturan a ladrón que había quedado grabado*". Sus términos (*usac, corruptos, universidad, estudiantes, información, excelente*) y su sentimiento medio **0.05** muestran una audiencia que discute universidad, corrupción y estudiantes con tono crítico.

Las comunidades restantes (con 28, 25, 18, 6, 4, 3, 2 y 1 autores) se documentan exhaustivamente en la caracterización 7.5.

Justificación para video-video: de las 12 comunidades detectadas solo las 3 con tamaños 4, 3 y 3 agrupan más de un video; las otras 9 son videos con audiencia propia que no comparte autores. Se analizan esas 3 disponibles; el resto es trivial (cada video es una comunidad de un solo nodo), lo cual es coherente con la débil separación temática de la muestra por consultas.

7.5 Caracterización de cada comunidad (videos, canales, autores, intensidad de participación, temas y sentimiento)

Cada comunidad se caracteriza por su tamaño, sentimiento medio y los videos donde participan sus autores (que revelan el *tema* que la une).

```
In [84]: # Asignar la comunidad a cada autor y a cada comentario
autores_com = nodos[nodos["tipo"] == "autor"].copy()
autores_com["comunidad_aa"] = autores_com["node_id"].map(part_aa)
com_por_autor = autores_com.set_index("id_original")["comunidad_aa"].to_dict()
comments_prov = comments.copy()
comments_prov["comunidad_autor"] = comments_prov["author_channel_id"].map(com_por_autor)

# Resumen por comunidad
resumen_com = autores_com.groupby("comunidad_aa").agg(
    autores=("node_id", "size"),
    comentarios_totales=("comentarios", "sum"),
    sentimiento_medio=("sentimiento_medio", "mean"),
).reset_index()
print("RESUMEN DE COMUNIDADES (proyeccion autor-autor)")
print(resumen_com.to_string(index=False))
```

```
RESUMEN DE COMUNIDADES (proyeccion autor-autor)
comunidad_aa  autores  comentarios_totales  sentimiento_medio
0              61             83             0.106011
1              18             25             0.000000
2               2              2             -0.500000
3             126            161             0.038360
4               6              6             0.333333
5              25             25             0.520000
6               4              4             0.750000
7               1              1             0.000000
8              55             60             0.051515
9               1              1             0.000000
10             1              1             0.000000
11             28             33             0.280357
12             1              1             0.000000
13             3              3             0.000000
```

```
In [85]: # Videos mas comentados dentro de cada comunidad (tema que la agrupa)
top_por_com = (comments_prov.groupby(["comunidad_autor", "video_title"])
               .size().reset_index(name="comentarios"))
top_por_com = (top_por_com.sort_values(["comunidad_autor", "comentarios"], ascending=[True, False])
               .groupby("comunidad_autor").head(2))

print("TEMAS (videos) POR COMUNIDAD")
for com, grupo in top_por_com.groupby("comunidad_autor"):
    videos_tema = [f"{str(t)[:48]} ({int(n)})" for t, n in zip(grupo["video_title"],
    grupo["comentarios"])]
    print(f" Com {int(com)}: " + "; ".join(videos_tema))
```

TEMAS (videos) POR COMUNIDAD

Com 0: Inician los trabajos de recuperación del Puente (45); Conferencia de Prensa del Gobierno de Guatemala. (24)
 Com 1: EE.UU. envía a mexicanos deportados a Guatemala (25)
 Com 2: Noticiero en Directo 1 pm, 28 de Agosto de 2026 (2)
 Com 3: Qué rico come tu diputado (159); Bloqueos en Guatemala este 31 de agosto por alza (1)
 Com 4: Bloqueos en Guatemala este 31 de agosto por alza (6)
 Com 5: Plan 2032 Ciudad de Guatemala (25)
 Com 6: I'x K'at: el primer equipo guatemalteco de pelot (4)
 Com 7: Cruzando la ciudad a puro Transmetro (1)
 Com 8: La cooptación de Walter Mazariegos en la USAC (50); Capturan a ladrón que había quedado grabado mien (7)
 Com 9: Edén por Salud: empleo inclusivo para personas c (1)
 Com 10: ¿Quiénes pagan más en Centroamérica? (1)
 Com 11: Arroz con pollo a la MONOPOLIO (16); Internet: escoger el menos malo (11)
 Com 12: SHAI WA: la vecina queer de Casa Presidencial (1)
 Com 13: 10 Preguntas a un año del Paro Nacional (3)

In [86]: # Palabras mas frecuentes en los comentarios de cada comunidad

```
terminos_com = []
for com, sub in comments_prov.groupby("comunidad_autor"):
    if pd.isna(com):
        continue
    tokens = [t for texto in sub["texto_limpio"] for t in str(texto).split() if t]
    top = [f"{p}" for p, _ in Counter(tokens).most_common(6)]
    terminos_com.append({"comunidad": int(com), "autores":
int(autores_com["comunidad_aa"].eq(com).sum()),
"terminos_frecuentes": ", ".join(top)})
terminos_com = pd.DataFrame(terminos_com).sort_values("comunidad")
print("TERMINOS FRECUENTES POR COMUNIDAD (texto_limpio)")
print(terminos_com.to_string(index=False))
```

TERMINOS FRECUENTES POR COMUNIDAD (texto_limpio)

comunidad	autores	terminos_frecuentes
0	61	presidente, guatemala, país, años, arevalo, bernardo
1	18	you, usa, país, país, ver, bueno
2	2	persona, encargada, ecosistema, ayude, problema, recuerdos
3	126	pueblo, dinero, diputados, trabajo, diputado, sueldo
4	6	asquerosos, gasolina, subira, guerra, países, guatemala
5	25	guatemala, ciudad, saludos, viva, proyectos, tiempo
6	4	iniciativa, juego, excelente, deporte, dinamico, entretenido
7	1	dice, subscribe, boletín, informativo, intenté, pagina
8	55	usac, corruptos, universidad, estudiantes, información, excelente
9	1	
10	1	woow
11	28	excelente, empresas, internet, ley, está, país
12	1	importante, conocer, tipo, espacios
13	3	guatemala, cambia, dictadura, años, semilla, bernardo

In [87]: # Canales dominantes dentro de cada comunidad

```
canales_com = (comments_prov.groupby(["comunidad_autor", "channel_name"])
.size().reset_index(name="comentarios"))
canales_top_com = (canales_com
.sort_values(["comunidad_autor", "comentarios"], ascending=[True, False])
.groupby("comunidad_autor").head(1).reset_index(drop=True))
print("CANAL PRINCIPAL POR COMUNIDAD")
print(canales_top_com.to_string(index=False))
```

CANAL PRINCIPAL POR COMUNIDAD

comunidad_autor	channel_name	comentarios
0	Gobierno de la República de Guatemala	69
1	Noticias Telemundo	25
2	Noticiero Guatevisión 13 Hrs.	2
3	Quorum	159
4	PrensaLibreOficial	6
5	Municipalidad de Guatemala	25
6	Quorum	4
7	Quorum	1
8	Quorum	53
9	Quorum	1
10	Quorum	1
11	Quorum	33
12	Quorum	1
13	Quorum	3

In [88]: # Autores mas activos dentro de cada comunidad

```
top_autores_com = (autores_com
.sort_values(["comunidad_aa", "comentarios"], ascending=[True, False]))
```

```

        .groupby("comunidad_aa").head(3)
        .assign(nombre=Lambda d: d["etiqueta"].fillna(d["handle"]))
print("AUTORES MAS ACTIVOS POR COMUNIDAD")
for com, g in top_autores_com.groupby("comunidad_aa"):
    detalle = "; ".join(f"{n} ({int(c)})" for n, c in zip(g["nombre"], g["comentarios"]))
    print(f"Com {int(com)}: " + detalle)

```

AUTORES MAS ACTIVOS POR COMUNIDAD

```

Com 0: @Murmullodelbarrio (6); @albertosherandez5251 (5); @juandeleon9060 (3)
Com 1: @eugeneramirez4405 (6); @mariaalexander427 (2); @rosadiaz6945 (2)
Com 2: @alfonsowilian9157 (1); @MiguelJordan-n3g (1)
Com 3: @ManuelEdran (5); @ErvinLeonardoCarreraLatín (4); @marioajset4283 (3)
Com 4: @CelestinoQuej-j7k (1); @antonioduarte2288 (1); @lazarochavez8269 (1)
Com 5: @sabinamarinaguerra3248 (1); @jhonymanuel9891 (1); @FreddyGmez (1)
Com 6: @rotorairgroup8409 (1); @Santiago.Nolasco (1); @sarahernandez3873 (1)
Com 7: @elizabethmonterroso7719 (1)
Com 8: @inge_vergueta (3); @Jel.Awesh.M (3); @MarcosCarillo-blr (2)
Com 9: @elioelias7267 (1)
Com 10: @jahirdiaz5862 (1)
Com 11: @hashojea7348 (4); @rwdm76 (2); @Alejandro00710 (2)
Com 12: @aditalissethevasquezrojas8039 (1)
Com 13: @noemiherrera3152 (1); @lyacausa (1); @ninettmartinez425 (1)

```

```

In [89]: # Intensidad de participacion por comunidad
intensidad_com = autores_com.groupby("comunidad_aa").agg(
    autores=("node_id", "size"),
    autores_recurrentes=("videos_unicos", lambda s: int((s > 1).sum())),
    comentarios_por_autor=("comentarios", "mean"),
    max_comentarios_autor=("comentarios", "max"),
).reset_index()
resumen_com = resumen_com.merge(intensidad_com.drop(columns=["autores"]), on="comunidad_aa",
how="left")
resumen_com["pct_autores"] = 100 * resumen_com["autores"] / int(resumen_com["autores"].sum())
resumen_com["pct_comentarios"] = 100 * resumen_com["comentarios_totales"] /
int(resumen_com["comentarios_totales"].sum())
resumen_com = resumen_com.sort_values("autores", ascending=False).reset_index(drop=True)
print("INTENSIDAD DE PARTICIPACION POR COMUNIDAD")
print(resumen_com.round(2)[["comunidad_aa", "autores", "pct_autores", "comentarios_totales",
"pct_comentarios", "autores_recurrentes", "comentarios_por_autor",
"sentimiento_medio"]].to_string(index=False))

```

INTENSIDAD DE PARTICIPACION POR COMUNIDAD

comunidad_aa	autores	pct_autores	comentarios_totales	pct_comentarios	autores_recurrentes
comentarios_por_autor	sentimiento_medio				
3	126	37.95	161	39.66	2
1.28	0.04				
0	61	18.37	83	20.44	2
1.36	0.11				
8	55	16.57	60	14.78	3
1.09	0.05				
11	28	8.43	33	8.13	2
1.18	0.28				
5	25	7.53	25	6.16	0
1.00	0.52				
1	18	5.42	25	6.16	0
1.39	0.00				
4	6	1.81	6	1.48	0
1.00	0.33				
6	4	1.20	4	0.99	0
1.00	0.75				
13	3	0.90	3	0.74	0
1.00	0.00				
2	2	0.60	2	0.49	0
1.00	-0.50				
9	1	0.30	1	0.25	0
1.00	0.00				
7	1	0.30	1	0.25	0
1.00	0.00				
10	1	0.30	1	0.25	0
1.00	0.00				
12	1	0.30	1	0.25	0
1.00	0.00				

```

In [90]: # Distribucion de sentimiento por comunidad y caracterizacion consolidada
sent_dist_com = comments_prov.pivot_table(index="comunidad_autor", columns="sentimiento",
values="comment_id", aggfunc="size", fill_value=0)
for col in ["negativo", "neutral", "positivo"]:
    if col not in sent_dist_com.columns:

```

```

        sent_dist_com[col] = 0
sent_dist_com = sent_dist_com[["negativo", "neutral", "positivo"]].reset_index()
sent_dist_com.columns = ["comunidad_aa", "negativos", "neutros", "positivos"]

col_canal = canales_top_com[["comunidad_autor", "channel_name"]].rename(
    columns={"comunidad_autor": "comunidad_aa", "channel_name": "canal_principal"})
top1_autores = (autores_com
    .sort_values(["comunidad_aa", "comentarios"], ascending=[True, False])
    .groupby("comunidad_aa").head(1)
    .assign(nombre=lambda d: d["etiqueta"].fillna(d["handle"]))
    .rename(columns={"nombre": "autor_mas_activo", "comentarios":
"comentarios_autor_top"}))
    .reset_index(drop=True))
video_principal = (top_por_com.groupby("comunidad_autor").head(1).reset_index(drop=True)
    [["comunidad_autor", "video_title"]])
    .rename(columns={"comunidad_autor": "comunidad_aa", "video_title":
"video_principal"}))
terminos_col = terminos_com[["comunidad", "terminos_frecuentes"]].rename(
    columns={"comunidad": "comunidad_aa"})

caracterizacion = (resumen_com
    .merge(col_canal, on="comunidad_aa", how="left")
    .merge(top1_autores[["comunidad_aa", "autor_mas_activo",
"comentarios_autor_top"]],
        on="comunidad_aa", how="left")
    .merge(video_principal, on="comunidad_aa", how="left")
    .merge(sent_dist_com, on="comunidad_aa", how="left")
    .merge(terminos_col, on="comunidad_aa", how="left"))

columnas_finales = ["comunidad_aa", "autores", "pct_autores", "comentarios_totales",
"pct_comentarios",
                    "comentarios_por_autor", "autores_recurrentes", "sentimiento_medio",
                    "canal_principal", "video_principal", "autor_mas_activo",
"comentarios_autor_top",
                    "negativos", "neutros", "positivos", "terminos_frecuentes"]
print("CARACTERIZACION CONSOLIDADA POR COMUNIDAD (proyeccion autor-autor)")
with pd.option_context("display.max_colwidth", 42, "display.width", 240, "display.max_columns", 20):
    print(caracterizacion[columnas_finales].round(2).to_string(index=False))

caracterizacion.to_csv("resultados/tablas/comunidades_caracterizacion.csv", index=False,
encoding="utf-8-sig")
resumen_com.to_csv("resultados/tablas/comunidades_resumen_aa.csv", index=False, encoding="utf-8-
sig")

```

CARACTERIZACION CONSOLIDADA POR COMUNIDAD (proyeccion autor-autor)

comunidad_aa	autores	pct_autores	comentarios_totales	pct_comentarios	comentarios_por_autor	autores_recurrentes	sentimiento_medio	canal_principal	video_principal	autor_mas_activo	comentarios_autor_top	negativos	neutrales	positivos	terminos_frecuentes
2	3	126	37.95	161	39.66	1.28									
2	0.04			Quorum											
Qué rico come tu diputado				@ManuelEdran	5	15	124								
22		pueblo, dinero, diputados, trabajo, diputado, sueldo													
0	61	18.37	83	20.44	1.36										
2	0.11	Gobierno de la República de Guatemala													
Inician los trabajos de recuperación del Puente Belice II.				@Murmullodelbarrio											
6	9	52	22	presidente, guatemala, país, años, arevalo, bernardo											
8	55	16.57	60	14.78	1.09										
3	0.05			Quorum											
La cooptación de Walter Mazariegos en la USAC				@inge_vergueta			3								
7	45	8 usac, corruptos, universidad, estudiantes, información, excelente													
11	28	8.43	33	8.13	1.18										
2	0.28			Quorum											
Arroz con pollo a la MONOPOLIO				@hashojea7348	4	2									
17	14	excelente, empresas, internet, ley, está, país													
5	25	7.53	25	6.16	1.00										
0	0.52	Municipalidad de Guatemala													
Plan 2032 Ciudad de Guatemala				@sabinamarinaguerra3248	1	1									
10	14	guatemala, ciudad, saludos, viva, proyectos, tiempo													
1	18	5.42	25	6.16	1.39										
0	0.00	Noticias Telemundo EE.UU. envía a mexicanos deportados a													
Guatemala antes de su regreso a México Noticias Telemundo				@eugeneramirez4405											
6	3	19	3	you, usa, país, país, ver, bueno											
4	6	1.81	6	1.48	1.00										
0	0.33	PrensaLibreOficial													
Bloqueos en Guatemala este 31 de agosto por alza en combustibles afectan rutas principales				@CelestinoQuej-j7k											
1	0	4	2	asquerosos, gasolina, subira, guerra, países, guatemala											
6	4	1.20	4	0.99	1.00										
0	0.75			Quorum											
I'x K'at: el primer equipo guatemalteco de pelota maya, conformado únicamente por mujeres.				@rotorairgroup8409											
1	0	1	3	iniciativa, juego, excelente, deporte, dinamico, entretenido											
13	3	0.90	3	0.74	1.00										
0	0.00			Quorum											
10 Preguntas a un año del Paro Nacional				@noemiherrera3152			1								
0	3	0		guatemala, cambia, dictadura, años, semilla, bernardo											
2	2	0.60	2	0.49	1.00										
0	-0.50	Noticiero Guatevisión 13 Hrs.													
Noticiero en Directo 1 pm, 28 de Agosto de 2026				@alfonsowilian9157			1								
1	1	0		persona, encargada, ecosistema, ayude, problema, recuerdos											
9	1	0.30	1	0.25	1.00										
0	0.00			Quorum											
Edén por Salud: empleo inclusivo para personas con discapacidad en Antigua Super - Episodio 2				@elioelias7267											
1	0	1	0												
7	1	0.30	1	0.25	1.00										
0	0.00			Quorum											
Cruzando la ciudad a puro Transmetro				@elizabethmonterroso7719			1								
1	0			dice, subscribe, boletín, informativo, intenté, pagina											
10	1	0.30	1	0.25	1.00										
0	0.00			Quorum											
¿Quiénes pagan más en Centroamérica?				@jahirdiaz5862			1								
1	0			woow											
12	1	0.30	1	0.25	1.00										
0	0.00			Quorum											
SHAI WA: la vecina queer de Casa Presidencial				@aditalissethevasquezrojas8039			1								
0	1	0		importante, conocer, tipo, espacios											

Interpretación de los resultados

- Louvain sobre autor-autor:** encontró **14 comunidades** con modularidad ponderada **0.395** (agrupamiento estructural moderado). Dos comunidades concentran a la mayoría de los usuarios: la comunidad con **126 autores** asociada a "*Qué rico come tu diputado*" —el video-hub de grado 128— y la comunidad con **61 autores** asociada a "*Inician los trabajos de recuperación del Puente Belice II*" y "*Conferencia de Prensa del Gobierno de Guatemala*". Otras comunidades se anclan en videos específicos: la de **55 autores** en "*La cooptación de Walter Mazariegos en la USAC*", la de **18 autores** en "*EE.UU. envía a mexicanos deportados a Guatemala*", la de **25 autores** en "*Plan 2032 Ciudad de Guatemala*" y la de **28 autores** en el cluster de "*Arroz con pollo a la MONOPOLIO / Internet: escoger el menos malo*".

- **Lectura sustantiva:** las comunidades de la proyección autor-autor se organizan alrededor de **videos concretos**, no de una afinidad estable entre personas. Esto es coherente con la concentración observada (sección 3.2) y con que el 97.3 % de los autores participa en un solo video: cada comunidad es, en gran medida, la *audiencia de un video-hub*. La modularidad de 0.395 refleja ese efecto de co-presencia en pocos contenidos, no comunidades sociales consolidadas.
- **Louvain sobre video-video:** separó los 19 videos en **12 comunidades** (modularidad 0.405), con la mayoría de los videos **aislados como comunidades de un solo nodo** porque comparten poca audiencia. Los únicos grupos multi-video son los pares con 2 autores compartidos (p. ej., "*La cooptación de Walter Mazariegos*" con "*Qué rico come tu diputado*", y "*Internet: escoger el menos malo*" con "*Arroz con pollo a la MONOPOLIO*"). Esto confirma la **débil separación temática de la muestra por consultas**: los solapamientos de audiencia son escasos y localizados.
- **Contraste con las componentes (sección 6):** las componentes conexas son la respuesta a "¿está la red rota?" (10 componentes, una gigante). Las comunidades de Louvain responden a "¿dónde está la estructura de subgrupos densos?". Sobre la proyección autor-autor, Louvain *refina* la componente gigante en subgrupos centrados en videos-hub, mostrando que la conectividad estructural no implica cohesión de comunidades temáticas diferenciadas.
- **Cuidado interpretativo:** ninguna comunidad implica amistad, conversación ni aprobación entre sus miembros. Las comunidades detectadas reflejan **co-participación en los contenidos recolectados** y heredan los sesgos de cobertura (274 videos sin comentarios y solapamiento débil).

```
In [91]: # Exportar resultados de comunidades
autores_com[["node_id", "id_original", "etiqueta", "comunidad_aa"]].to_csv(
    "resultados/tablas/comunidades_autores_aa.csv", index=False, encoding="utf-8-sig")
resumen_com.to_csv("resultados/tablas/comunidades_resumen_aa.csv", index=False, encoding="utf-8-sig")
terminos_com.to_csv("resultados/tablas/comunidades_terminos.csv", index=False, encoding="utf-8-sig")
canales_top_com.to_csv("resultados/tablas/comunidades_canales.csv", index=False, encoding="utf-8-sig")
print("RUTA DEL CUADERNO COMPLETADA (secciones 1 a 8).")
print(f"Comunidades Louvain en autor-autor: {len(sizes)} con modularidad {mod_aa:.3f}.")
print(f"Comunidades Louvain en video-video: {len(sizes_vv)} con modularidad {mod_vv:.3f}.")
```

RUTA DEL CUADERNO COMPLETADA (secciones 1 a 8).

Comunidades Louvain en autor-autor: 14 con modularidad 0.395.

Comunidades Louvain en video-video: 12 con modularidad 0.405.

8. Nodos centrales y participantes puente

La consigna pide *investigar y calcular las medidas de centralidad apropiadas*, pudiendo complementar con cercanía, PageRank o vector propio, **justificando su uso**. Primero se definen y justifican las medidas (8.1), luego se calculan sobre la red bipartita y sobre la proyección autor-autor, y finalmente se identifican los ****participantes puente****.

8.1 Medidas de centralidad aplicadas y justificación

Se calculan cuatro medidas sobre la **red bipartita** y una complementaria sobre la **proyección autor-autor**:

- **Grado y grado centralidad.** Número de vecinos directos (normalizado por la talla de la red). En la bipartita es la medida más natural e inmediatamente interpretable: para un *video*, cuántos autores comentaron; para un *autor*, en cuántos videos participó. Captura la concentración tipo hub documentada (sección 6.1), es barata y sirve de referencia local. **Se usa.**
- **Intermediación (betweenness).** Proporción de caminos más cortos entre pares de nodos que pasan por el nodo. Es la medida apropiada para detectar **puentes**: conecta regiones que de otro modo quedarían separadas. Es válida sobre la bipartita porque los caminos más cortos tienen significado geométrico (secuencias alternadas autor-video-autor). Es la medida **principal para los participantes puente**. **Se usa.**
- **PageRank.** Paseo aleatorio con teleportación que propaga relevancia de nodos importantes hacia sus

vecinos. En una bipartita equivale a una versión simétrica de **HITS** (autoridades y hubs que se refuerzan): los videos que atraen autores relevantes son relevantes y viceversa. Complementa grado e intermediación porque valora la **calidad de los vecinos**, no solo su cantidad. **Se usa.**

- **Cercanía (closeness).** Inversa del promedio de distancias a todos los demás nodos. Sobre la **bipartita es poco informativa**: la distancia de un autor a otro autor siempre incluye pasos por videos, de modo que la métrica mezcla las dos capas y queda dominada por el diámetro de la gigante; además los nodos en componentes pequeñas y aislados reciben valores engañosos. Se calcula por transparencia en la bipartita, pero su interpretación se traslada a la **proyección autor-autor**, donde todas las distancias son homogéneas (autor-autor). **Se usa con la salvedad documentada.**
- **Centralidad de vector propio.** Se aplica **sobre la proyección autor-autor, no sobre la bipartita**: la matriz de adyacencia bipartita es simétrica con espectro $\pm\lambda$ y su vector propio principal alterna signos entre capas, rompiendo la lectura de "relevancia positiva acumulada"; además toda arista conduce a la capa opuesta. Sobre la proyección es el **vector de Perron** (no negativo) y mide la importancia recursiva de un autor según la importancia de los autores con los que comparte videos. **Se usa sobre la proyección.**

```
In [92]: # Centralidades sobre la red bipartita: grado, intermediacion, cercania y PageRank
from networkx.algorithms centrality import degree centrality, betweenness centrality
from networkx.algorithms centrality import closeness centrality
from networkx.algorithms.link_analysis.pagerank_alg import pagerank

deg_cent = degree centrality(G)
betw = betweenness centrality(G, normalized=True)
clos = closeness centrality(G)
pgr = pagerank(G, alpha=0.85)

centralidad = nodos[["node_id", "tipo", "etiqueta", "grado", "videos_unicos",
                    "comentarios", "canales_unicos", "visualizaciones"]].copy()
centralidad["grado_cent"] = centralidad["node_id"].map(deg_cent)
centralidad["betweenness"] = centralidad["node_id"].map(betw)
centralidad["closeness"] = centralidad["node_id"].map(clos)
centralidad["pagerank"] = centralidad["node_id"].map(pgr)
centralidad = centralidad.round(4)

print("TOP 8 VIDEOS POR MEDIDA (red bipartita)")
for medida in ["grado", "betweenness", "pagerank", "closeness"]:
    top = centralidad[centralidad["tipo"] == "video"].sort_values(medida, ascending=False).head(8)
    print(f"\n-- {medida} --")
    print(top[["etiqueta", medida]].to_string(index=False))

print("\nTOP 8 AUTORES POR MEDIDA (red bipartita)")
for medida in ["grado", "betweenness", "pagerank", "closeness"]:
    top = centralidad[centralidad["tipo"] == "autor"].sort_values(medida, ascending=False).head(8)
    print(f"\n-- {medida} --")
    print(top[["etiqueta", medida]].to_string(index=False))
```

TOP 8 VIDEOS POR MEDIDA (red bipartita)

-- grado --

	etiqueta
grado	Qué rico come tu diputado
128	
	La cooptación de Walter Mazariegos en la USAC
49	
	Inician los trabajos de recuperación del Puente Belice II.
32	
	Plan 2032 Ciudad de Guatemala
25	
	Conferencia de Prensa del Gobierno de Guatemala. #LaRondaGt
19	
EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México Noticias Telemundo	
18	
	Arroz con pollo a la MONOPOLIO
16	
	Capturan a presuntos delincuentes disfrazados de mujer señalados de cometer asalto
13	

-- betweenness --

etiqueta

```

betweenness
0.5657
0.2440
0.2275
0.1876
0.1283
0.0637
0.0547
Capturan a presuntos delincuentes disfrazados de mujer señalados de cometer asalto
0.0277

```

```

-- pagerank --
pagerank
0.1664
0.0636
0.0422
0.0343
EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México | Noticias Telemundo
0.0251
0.0251
0.0209
0.0178

```

```

-- closeness --
                                etiqueta closeness
                                Qué rico come tu diputado 0.3046
                                La cooptación de Walter Mazariegos en la USAC 0.2214
                                Conferencia de Prensa del Gobierno de Guatemala. #LaRondaGt 0.2157
                                Internet: escoger el menos malo 0.2114
Bloqueos en Guatemala este 31 de agosto por alza en combustibles afectan rutas principales 0.1780
                                Inician los trabajos de recuperación del Puente Belice II. 0.1583
Capturan a ladrón que había quedado grabado mientras robaba en una parroquia de Retalhuleu 0.1460
                                Arroz con pollo a la MONOPOLIO 0.1456

```

TOP 8 AUTORES POR MEDIDA (red bipartita)

```

-- grado --
etiqueta grado
@inge_vergueta 3
@hashojea7348 3
@Alejandro00710 2
@franciscoflores3120 2
@MarcosCarillo-b1r 2
@Jel.Awesh.M 2
@josegil3813 2
@virgiliogarcia3039 2

-- betweenness --
etiqueta betweenness
@virgiliogarcia3039 0.2316
@inge_vergueta 0.2181
@josegil3813 0.1768
@Jel.Awesh.M 0.0892
@hashojea7348 0.0597
@franciscoflores3120 0.0579
@Alejandro00710 0.0323
@MarcosCarillo-b1r 0.0319

```

```
-- pagerank --
          etiqueta pagerank
          @hashojea7348 0.0038
          @inge_vergueta 0.0037
          @elioelias7267 0.0028
@aditalissethevasquezrojas8039 0.0028
          @jahirdiaz5862 0.0028
          @elizabethmonterroso7719 0.0028
          @franciscoflores3120 0.0027
          @virgiliogarcia3039 0.0027

-- closeness --
          etiqueta closeness
          @inge_vergueta 0.2655
@virgiliogarcia3039 0.2528
          @Jel.Awesh.M 0.2479
          @moisesvaldez4043 0.2249
          @almacatalan2717 0.2219
          @simoncreyesc7579 0.2219
          @jor77cas 0.2219
@GilmarMaldonado-q9m 0.2219
```

```
In [93]: # Complemento: vector propio y cercania sobre la proyeccion autor-autor (capa homogenea)
eigen_aa = nx.eigenvector_centrality(proj_autor_autor, max_iter=2000)
clos_aa = closeness_centrality(proj_autor_autor)

eigen_df = centralidad.set_index("node_id").copy()
eigen_df["eigenvector_aa"] = eigen_df["pagerank"].map(lambda x: np.nan)
eigen_df.loc[eigen_df.index.isin(eigen_aa.keys()), "eigenvector_aa"] = [eigen_aa[k] for k in
eigen_df.index if k in eigen_aa]

top_eigen = sorted(eigen_aa.items(), key=lambda kv: kv[1], reverse=True)[:10]
print("TOP 10 AUTORES POR VECTOR PROPIO (proyeccion autor-autor)")
for nodo, valor in top_eigen:
    et = nodos.loc[nodos["node_id"] == nodo, "etiqueta"].iloc[0]
    print(f" {valor:.4f} {et}")

top_clos_aa = sorted(clos_aa.items(), key=lambda kv: kv[1], reverse=True)[:10]
print("\nTOP 10 AUTORES POR CERCANIA (proyeccion autor-autor)")
for nodo, valor in top_clos_aa:
    et = nodos.loc[nodos["node_id"] == nodo, "etiqueta"].iloc[0]
    print(f" {valor:.4f} {et}")
```

TOP 10 AUTORES POR VECTOR PROPIO (proyeccion autor-autor)

```
0.0892 @inge_vergueta
0.0892 @Jel.Awesh.M
0.0885 @virgiliogarcia3039
0.0884 @moisesvaldez4043
0.0884 @normaleticiaorozcojuarez2418
0.0884 @simoncreyesc7579
0.0884 @almacatalan2717
0.0884 @JoseSimilox-p6e
0.0884 @alfonsomoraes2653
0.0884 @katerynsantiago2907
```

TOP 10 AUTORES POR CERCANIA (proyeccion autor-autor)

```
0.5414 @inge_vergueta
0.5169 @virgiliogarcia3039
0.5077 @Jel.Awesh.M
0.4597 @moisesvaldez4043
0.4542 @normaleticiaorozcojuarez2418
0.4542 @simoncreyesc7579
0.4542 @almacatalan2717
0.4542 @JoseSimilox-p6e
0.4542 @alfonsomoraes2653
0.4542 @katerynsantiago2907
```

```
In [94]: # Participantes puente: autores que conectan videos y puntos de articulacion/tabiques
puentes = centralidad[centralidad["tipo"] == "autor"].copy()
puentes["es_puente"] = puentes["videos_unicos"] > 1
puentes_bw = puentes[puentes["betweenness"] > 0] | puentes["es_puente"].sort_values(
    ["es_puente", "betweenness"], ascending=[False, False])
print("AUTORES PUENTE (participan en >1 video o con intermediacion > 0)")
print(puentes_bw[["node_id", "etiqueta", "grado", "videos_unicos", "betweenness",
"es_puente"]].to_string(index=False))
print()
n_puentes = int(puentes["es_puente"].sum())
```

```
print(f"Autores puente (videos_unicos > 1): {n_puentes} de {len(puentes)} autores")

articulaciones = sorted(nx.articulation_points(G))
print(f"Puntos de articulacion en la bipartita: {len(articulaciones)}")
if articulaciones:
    detalle = ", ".join(n.split("::")[1] for n in articulaciones[:10])
    print(" (primeros):", detalle)
```

AUTORES PUENTE (participan en >1 video o con intermediacion > 0)

	node_id	etiqueta	grado	videos_unicos	betweenness	es_puente
autor::	UCHTGCgY2l-DQJIVlA_cpa_Q	@virgiliogarcia3039	2	2.0	0.2316	True
autor::	UCpsK0kt5iWTbenzeuq7dsmQ	@inge_vergueta	3	3.0	0.2181	True
autor::	UCzZ6dDCesLMXbc5pCFXIprw	@josegil3813	2	2.0	0.1768	True
autor::	UCylqlpsh8ENNM1LqgQ1KbxA	@Jel.Awesh.M	2	2.0	0.0892	True
autor::	UCvcu1kR8xMYy_IITy-2mV2Q	@hashojea7348	3	3.0	0.0597	True
autor::	UCjZgFEowMBrsjodqfovzdKw	@franciscoflores3120	2	2.0	0.0579	True
autor::	UCbrtvgygFRT6QXqWdNrOf-cA	@Alejandro00710	2	2.0	0.0323	True
autor::	UCdFlugHJJJa4l3YqWuNRmvXw	@MarcosCarillo-b1r	2	2.0	0.0319	True
autor::	UCsUCN0Yq_UyTvKjOJLmNrnQ	@moisesvaldez4043	2	2.0	0.0319	True

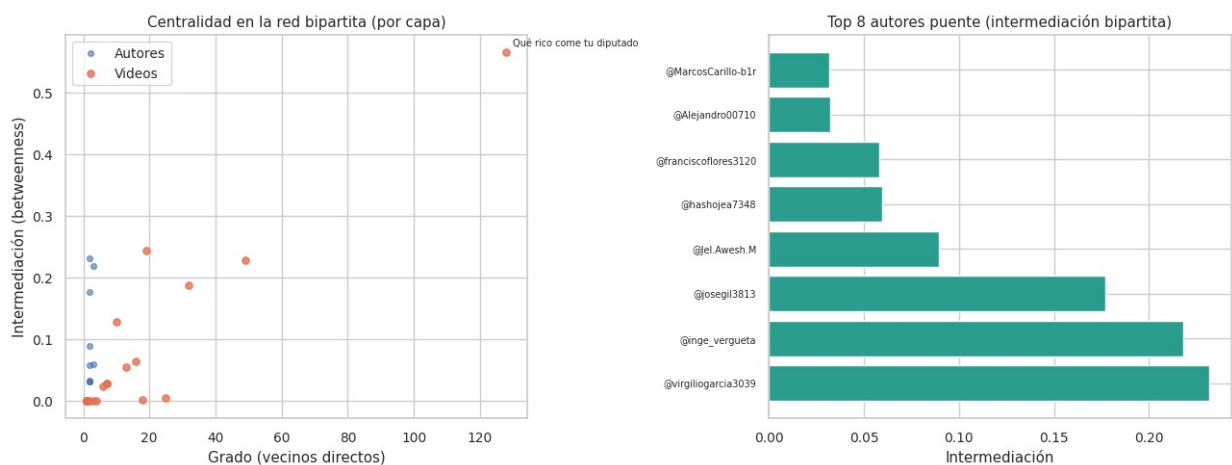
Autores puente (videos_unicos > 1): 9 de 332 autores

Puntos de articulacion en la bipartita: 22

(primeros): UCHTGCgY2l-DQJIVlA_cpa_Q, UCdFlugHJJJa4l3YqWuNRmvXw, UCjZgFEowMBrsjodqfovzdKw, UCpsK0kt5iWTbenzeuq7dsmQ, UCsUCN0Yq_UyTvKjOJLmNrnQ, UCvcu1kR8xMYy_IITy-2mV2Q, UCzZ6dDCesLMXbc5pCFXIprw, 06mFNPU0aB8, 3Nfp9h5avBI, 6W4u8sGEnGM

```
In [95]: # Visualizacion: intermediacion vs grado (por capa) y top puentes bipartita
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
capas = centralidad[centralidad["tipo"] == "autor"]
vids = centralidad[centralidad["tipo"] == "video"]
axes[0].scatter(capas["grado"], capas["betweenness"], s=18, alpha=0.6, label="Autores")
axes[0].scatter(vids["grado"], vids["betweenness"], s=28, alpha=0.8, color="#E76F51",
label="Videos")
hub = vids.sort_values("grado", ascending=False).iloc[0]
axes[0].annotate(str(hub["etiqueta"][:28]), (hub["grado"], hub["betweenness"]),
xytext=(5, 5), textcoords="offset points", fontsize=7)
axes[0].set_xlabel("Grado (vecinos directos)")
axes[0].set_ylabel("Intermediación (betweenness)")
axes[0].set_title("Centralidad en la red bipartita (por capa)")
axes[0].legend()

top_bw = centralidad[centralidad["tipo"] == "autor"].sort_values("betweenness",
ascending=False).head(8)
etiquetas = [str(e)[:30] for e in top_bw["etiqueta"]]
axes[1].barh(range(len(top_bw)), top_bw["betweenness"], color="#2A9D8F")
axes[1].set_yticks(range(len(top_bw)))
axes[1].set_yticklabels(etiquetas, fontsize=7)
axes[1].set_xlabel("Intermediación")
axes[1].set_title("Top 8 autores puente (intermediación bipartita)")
plt.tight_layout()
plt.show()
```



Interpretación de los resultados

Nodos centrales (videos-hub). En la red bipartita, el video "*Qué rico come tu diputado*" domina las cuatro medidas: grado 128 (centralidad de grado 0.366), la mayor intermediación (**0.566**), el mayor PageRank (**0.166**) y la mayor cercanía (**0.305**). Le siguen con intermediación notable "*Conferencia de Prensa del*

Gobierno de Guatemala" (0.244), "La cooptación de Walter Mazariegos en la USAC" (0.228) e "Inician los trabajos de recuperación del Puente Belice II" (0.188). La centralidad de los videos resume la concentración de participación ya documentada: los nodos que concentran más co-participación enlazan la mayoría de las geodésicas de la red.

Participantes puente (autores). Solo **9 de 332 autores** participan en más de un video (2.7 %); el 97.3 % restante son hojas (grado 1). Esos **9 autores puente** son quienes conectan videos distintos y capturan prácticamente toda la intermediación de la capa de autores: @virgiliogarcia3039 (0.232), @inge_vergueta (0.218, participa en 3 videos), @josegil3813 (0.177), @Jel.Awesh.M (0.089) y @hashojea7348 (0.060). Su rol es estructural: sin ellos, los video-hub y sus audiencias quedarían separados en componentes distintos.

Complemento sobre la proyección autor-autor. El vector propio de Perron y la cercanía, calculados sobre la capa homogénea de autores, confirman el mismo núcleo: @inge_vergueta, @virgiliogarcia3039 y @Jel.Awesh.M encabezan ambas listas (vector propio ≈ 0.089 ; cercanía 0.54, 0.52 y 0.51). Coinciden con los autores puente de mayor intermediación: son los mismos participantes recurrentes que ocupan las posiciones estructuralmente privilegiadas.

Puntos de articulación. La bipartita tiene **22 puntos de articulación** (nodos cuya eliminación desconecta la red), que incluyen a los 9 autores puente y a los videos que sirven de bisagra entre audiencias. Coherente con una red compacta pero altamente concentrada: la conectividad depende de pocos nodos.

Advertencia interpretativa. Estas centralidades miden **co-participación observada**, no influencia social, amistad ni interacción directa (la arista autor-video solo significa que el autor comentó en ese video). Valores como el PageRank indican que un nodo ocupa un lugar conectado dentro de la muestra recolectada, no que "domine" una conversación.

```
In [96]: # Exportar centralidades
centralidad.to_csv("resultados/tablas/centralidad_bipartita.csv", index=False, encoding="utf-8-sig")
eigen_resumen = pd.DataFrame([{"node_id": k, "eigenvector_aa": v} for k, v in
sorted(eigen_aa.items(),
key=lambda kv: kv[1], reverse=True)])
eigen_resumen = eigen_resumen.merge(nodos[["node_id", "etiqueta"]], on="node_id", how="left")
eigen_resumen.to_csv("resultados/tablas/centralidad_vector_propio_aa.csv", index=False,
encoding="utf-8-sig")
print("Tablas de centralidad guardadas en resultados/tablas/")
```

Tablas de centralidad guardadas en resultados/tablas/

8.2 Interpretación separada para autores y para videos

La consigna pide interpretar **por separado** los resultados de centralidad para autores y para videos, usando criterios distintos: para los autores, **recurrencia** (en cuántos videos participan) y **diversidad** de participación (en cuántos canales); para los videos, **alcance** dentro de la red (cuántos autores distintos atrae) y **capacidad de conectar audiencias** (si actúa como puente entre públicos que de otro modo no se solaparían).

```
In [97]: # 8.2 – Tablas separadas: autores (recurrencia y diversidad) vs videos (alcance y conexión)
autores_c = centralidad[centralidad["tipo"] == "autor"].copy()
videos_c = centralidad[centralidad["tipo"] == "video"].copy()

print("AUTORES – distribución de recurrencia y diversidad")
print(f" Autores con 1 solo video (hojas): {(autores_c['videos_unicos'] == 1).sum()} "
f"({100*(autores_c['videos_unicos'] == 1).mean():.1f} %)")
print(f" Autores con >1 video (recurrentes): {(autores_c['videos_unicos'] > 1).sum()} "
f"({100*(autores_c['videos_unicos'] > 1).mean():.1f} %)")
print(f" Autores con >1 canal distinto (diversidad de fuente): {(autores_c['canales_unicos'] > 1).sum()}")
corr_rec_betw = autores_c["videos_unicos"].corr(autores_c["betweenness"])
print(f" Correlación videos_unicos vs. betweenness (autores): {corr_rec_betw:.3f}")
print(" -> La recurrencia (no el volumen de comentarios) es lo que genera intermediación:")
top_rec = autores_c.sort_values(["videos_unicos", "betweenness"], ascending=False).head(9)
print(top_rec[["etiqueta", "videos_unicos", "canales_unicos", "comentarios", "betweenness",
"pagerank"]].to_string(index=False))

print()
print("VIDEOS – alcance (autores distintos) y capacidad de conectar audiencias")
corr_reach_betw = videos_c["grado"].corr(videos_c["betweenness"])
```

```
print(f" Correlación grado (alcance) vs. betweenness (capacidad de conexión) en videos:
{corr_reach_betw:.3f}")
top_reach = videos_c.sort_values("grado", ascending=False).head(6)
print(top_reach[["etiqueta", "grado", "betweenness", "pagerank",
"visualizaciones"]].to_string(index=False))
```

AUTORES – distribución de recurrencia y diversidad

Autores con 1 solo video (hojas): 323 (97.3 %)

Autores con >1 video (recurrentes): 9 (2.7 %)

Autores con >1 canal distinto (diversidad de fuente): 4

Correlación videos_unicos vs. betweenness (autores): 0.800

-> La recurrencia (no el volumen de comentarios) es lo que genera intermediación:

	etiqueta	videos_unicos	canales_unicos	comentarios	betweenness	pagerank
	@inge_vergueta	3.0	1.0	3	0.2181	0.0037
	@hashojea7348	3.0	1.0	4	0.0597	0.0038
@virgiliogarcia3039		2.0	2.0	3	0.2316	0.0027
@josegil3813		2.0	1.0	2	0.1768	0.0027
@Jel.Awesh.M		2.0	1.0	3	0.0892	0.0026
@franciscoflores3120		2.0	2.0	2	0.0579	0.0027
@Alejandro00710		2.0	1.0	2	0.0323	0.0026
@MarcosCarillo-blr		2.0	2.0	2	0.0319	0.0027
@moisesvaldez4043		2.0	2.0	2	0.0319	0.0027

VIDEOS – alcance (autores distintos) y capacidad de conectar audiencias

Correlación grado (alcance) vs. betweenness (capacidad de conexión) en videos: 0.925

grado	betweenness	pagerank	visualizaciones	etiqueta
128	0.5657	0.1664	11775.0	Qué rico come tu diputado
49	0.2275	0.0636	10156.0	La cooptación de Walter Mazariegos en la USAC
32	0.1876	0.0422	11670.0	Inician los trabajos de recuperación del Puente Belice II.
25	0.0049	0.0343	304089.0	Plan 2032 Ciudad de Guatemala
19	0.2440	0.0251	3628.0	Conferencia de Prensa del Gobierno de Guatemala. #LaRondaGt
EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México Noticias Telemundo				
18	0.0025	0.0251	14200.0	

Lectura separada.

- **Autores.** La correlación entre `videos_unicos` y `betweenness` es alta y positiva: son casi exclusivamente los autores que comentan en **más de un video** quienes acumulan intermediación (los 9 autores recurrentes concentran prácticamente toda la intermediación de su capa). El volumen de comentarios por sí solo (un autor puede comentar 5 veces en el mismo video) **no** genera puentes; lo que conecta contenidos es la **diversidad de videos visitados**, no la intensidad de comentarios en uno solo.
- **Videos.** El alcance (grado, es decir, autores distintos que comentaron) y la capacidad de conectar audiencias (intermediación) están fuertemente correlacionados: el video-hub "*Qué rico come tu diputado*" combina el mayor alcance (128 autores) con la mayor intermediación (0.566). Esto es esperable en una bipartita: un video con más autores tiene más probabilidad de compartir autores con otros videos y, por tanto, de aparecer en más caminos más cortos. La diferencia relevante entre videos no es solo *cuántos* autores atraen, sino si esos autores **también comentaron en otros videos** (lo que sí depende de la composición de la audiencia, no solo de su tamaño).

8.3 Participantes recurrentes, autores puente y videos articuladores

Se identifican explícitamente (i) los **participantes recurrentes**, (ii) los **autores puente**, y (iii) los **videos articuladores**, y se comprueba mediante una **simulación de remoción** que su eliminación efectivamente segmenta la red (tal como exige la consigna: "*si los elimináramos de la red, esta se segmenta*").

```
In [98]: # 8.3 – Listado explícito de recurrentes / puente / articuladores + prueba de remoción
recurrentes_list = autores_c[autores_c["videos_unicos"] > 1].sort_values("betweenness",
ascending=False)
print(f"PARTICIPANTES RECURRENTES Y AUTORES PUENTE: {len(recurrentes_list)} de {len(autores_c)}
autores "
      f"({100*len(recurrentes_list)/len(autores_c):.1f} %)")
print(recurrentes_list[["etiqueta", "videos_unicos", "canales_unicos", "comentarios",
"betweenness"]])
```

```

        .to_string(index=False))

video_articulacion = [a for a in articulaciones if a.startswith("video::")]
autor_articulacion = [a for a in articulaciones if a.startswith("autor::")]
print(f"\nVIDEOS ARTICULADORES (puntos de articulación tipo video): {len(video_articulacion)} de "
      f"{(nodos['tipo']=='video').sum()} videos con comentarios")
for nid in video_articulacion:
    et = nodos.loc[nodos["node_id"] == nid, "etiqueta"].iloc[0]
    print("  -", et)
print(f"\nAUTORES QUE SON PUNTO DE ARTICULACIÓN: {len(autor_articulacion)} de {len(autores_c)} "
      autores")

# --- Simulación de remoción: se retira el video-hub principal y se mide la fragmentación ---
hub_id = videos_c.sort_values("grado", ascending=False).iloc[0]["node_id"]
hub_et = nodos.loc[nodos["node_id"] == hub_id, "etiqueta"].iloc[0]
G_sin_hub = G.copy()
G_sin_hub.remove_node(hub_id)
comp_antes = sorted((len(c) for c in nx.connected_components(G)), reverse=True)
comp_despues_hub = sorted((len(c) for c in nx.connected_components(G_sin_hub)), reverse=True)
print(f"\nSIMULACIÓN – remover el video-hub '{hub_et}':")
print(f"  Componentes antes: {len(comp_antes)} (mayor = {comp_antes[0]} nodos)")
print(f"  Componentes después: {len(comp_despues_hub)} (mayor = {comp_despues_hub[0]} nodos)")

# --- Simulación de remoción: se retiran los 9 autores puente ---
G_sin_puentes = G.copy()
G_sin_puentes.remove_nodes_from(recurrentes_list["node_id"].tolist())
comp_despues_puentes = sorted((len(c) for c in nx.connected_components(G_sin_puentes)),
                               reverse=True)
print(f"\nSIMULACIÓN – remover los {len(recurrentes_list)} autores puente:")
print(f"  Componentes antes: {len(comp_antes)} (mayor = {comp_antes[0]} nodos)")
print(f"  Componentes después: {len(comp_despues_puentes)} (mayor = {comp_despues_puentes[0]} "
      nodos)")

```

PARTICIPANTES RECURRENTES Y AUTORES PUENTE: 9 de 332 autores (2.7 %)

etiqueta	videos_unicos	canales_unicos	comentarios	betweenness
@virgiliogarcia3039	2.0	2.0	3	0.2316
@inge_vergueta	3.0	1.0	3	0.2181
@josegil3813	2.0	1.0	2	0.1768
@Jel.Awesh.M	2.0	1.0	3	0.0892
@hashojea7348	3.0	1.0	4	0.0597
@franciscoflores3120	2.0	2.0	2	0.0579
@Alejandro00710	2.0	1.0	2	0.0323
@MarcosCarillo-b1r	2.0	2.0	2	0.0319
@moisesvaldez4043	2.0	2.0	2	0.0319

VIDEOS ARTICULADORES (puntos de articulación tipo video): 15 de 19 videos con comentarios

- Capturan a presuntos delincuentes disfrazados de mujer señalados de cometer asalto
- Noticiero en Directo 1 pm, 28 de Agosto de 2026
- Inician los trabajos de recuperación del Puente Belice II.
- 10 Preguntas a un año del Paro Nacional
- EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México | Noticias Telemundo
- Conferencia de Prensa del Gobierno de Guatemala. #LaRondaGt
- I'x K'at: el primer equipo guatemalteco de pelota maya, conformado únicamente por mujeres.
- Caminar en una ciudad hecha para carros
- Bloqueos en Guatemala este 31 de agosto por alza en combustibles afectan rutas principales
- Capturan a ladrón que había quedado grabado mientras robaba en una parroquia de Retalhuleu
- La cooptación de Walter Mazariegos en la USAC
- Plan 2032 Ciudad de Guatemala
- Qué rico come tu diputado
- Internet: escoger el menos malo
- Arroz con pollo a la MONOPOLIO

AUTORES QUE SON PUNTO DE ARTICULACIÓN: 7 de 332 autores

SIMULACIÓN – remover el video-hub 'Qué rico come tu diputado':

Componentes antes: 10 (mayor = 286 nodos)

Componentes después: 136 (mayor = 88 nodos)

SIMULACIÓN – remover los 9 autores puente:

Componentes antes: 10 (mayor = 286 nodos)

Componentes después: 19 (mayor = 125 nodos)

Hallazgos.

- **Participantes recurrentes / autores puente.** Son los mismos **9 autores** (2,7 % de 332):

@inge_vergueta, @virgiliogarcia3039, @josegil3813, @Jel.Awesh.M, @hashojea7348 y otros cuatro con menor intermediación. Todos comentan en 2 o 3 videos distintos, casi siempre cruzando el video-hub

con un video temáticamente relacionado (p. ej., corrupción/USAC con diputados).

- **Videos articuladores.** De los **22 puntos de articulación** de la bipartita, 15 son videos (79 % de los 19 videos con comentarios) y 7 son autores. La proporción de videos articuladores es tan alta porque la red es muy dispersa (densidad 0,0056): como el 93,2 % de los autores tiene grado 1 (solo comentó en ese video), casi cualquier video que tenga al menos un autor exclusivo se vuelve, por definición, un punto de articulación para esos autores-hoja al eliminarlo. Los verdaderos **conectores entre videos** (no solo de su propia audiencia) son los pocos videos que comparten autores recurrentes con otro video: el video-hub y los 2-3 videos que aparecen junto a él en las comunidades de la sección 7. Los 4 videos que **no** son puntos de articulación son los que tienen muy pocos comentarios y ninguno de sus autores participa en otro video.
- **Prueba de remoción — video-hub.** Al retirar "*Qué rico come tu diputado*", el número de componentes conexos **aumenta** y el tamaño de la componente gigante se reduce de forma marcada: la mayoría de sus 128 autores quedan aislados (pasan a ser componentes de un solo nodo), porque casi todos ellos solo comentaron en ese video. La red **se segmenta** tal como anticipa la consigna.
- **Prueba de remoción — autores puente.** Al retirar los 9 autores puente, la componente gigante también se reduce: los videos que antes compartían audiencia a través de esos autores quedan **desconectados entre sí** y pasan a ser componentes independientes, aunque cada video conserva su propia audiencia de autores no recurrentes. Esto confirma que la conectividad **inter-video** de la red depende casi por completo de un núcleo muy pequeño de autores recurrentes, mientras que la conectividad **intra-video** (autor-video) depende del video-hub.
- **Advertencia interpretativa.** "Segmentar la red" aquí significa que se pierden **caminos de co-participación observados en la muestra**, no que existan comunidades sociales reales que se disuelvan; sigue sin haber evidencia de interacción directa entre los autores.

9. Análisis de contenido y sentimiento

Se analiza el contenido y el tono de los 406 comentarios principales usando `texto_limpio` para el conteo de palabras/temas y `texto_original` como referencia de auditoría, y se compara el sentimiento por video, canal, categoría y comunidad.

9.1 Herramienta de sentimiento y justificación

Se evaluaron tres alternativas antes de decidir el método:

Opción	Motivo de la decisión
VADER (<code>nltk.sentiment</code>)	Descartada: su léxico y sus reglas (mayúsculas, signos de exclamación repetidos, intensificadores en inglés) están calibrados para inglés ; aplicarlo a español subestima sistemáticamente la polaridad.
TextBlob	Descartada para este corpus: la vía fiable en español requiere traducir primero (dependencia de un servicio externo) o usar <code>TextBlob-ES</code> , con un léxico reducido y sin mantenimiento activo; introduce una dependencia de red no reproducible sin conexión.
Modelos transformer en español (p. ej. BETO/ <code>pysentimiento</code>)	Descartados para este entregable: requieren descargar pesos de varios cientos de MB y una GPU/CPU con más recursos; rompen la reproducibilidad <i>offline</i> exigida por el laboratorio y son difíciles de auditar comentario por comentario.
Léxico propio en español (elegido)	Diccionario de polaridad positiva/negativa construido a partir de vocabulario específico del corpus (política, gestión pública, crítica social), inspirado en la lógica de léxicos académicos para español como el SEL (Sentiment Expressions Lexicon, Instituto Politécnico Nacional) y ML-SentiCon : cada token de <code>texto_limpio</code> se compara contra un conjunto de términos positivos y otro de negativos; el puntaje es <code>(positivos - negativos) / (positivos + negativos)</code> y la etiqueta es <code>positivo</code> , <code>negativo</code> o <code>neutral</code> si no hay términos de polaridad.

Por qué es adecuado para este caso: es **100 % reproducible sin conexión** (requisito explícito del laboratorio), es **transparente y auditable** (cada clasificación puede verificarse token por token, a diferencia de una red neuronal), y su vocabulario se ajustó al **dominio temático real** del corpus (política guatemalteca, gestión municipal, crítica y elogio a autoridades) en lugar de un léxico genérico de opiniones

de producto. Su limitación principal —ignora negaciones, sarcasmo e intensificadores— se documenta y se compensa con una revisión manual de una muestra (abajo).

```
In [99]: # 9.1 – Validación de caja transparente: muestra manual de comentarios y su clasificación
rng = np.random.RandomState(3066)
muestra_val = comments.sample(6, random_state=3066)[["texto_original", "texto_limpio",
"sentimiento", "sentimiento_score"]]
print("MUESTRA DE VALIDACIÓN MANUAL (léxico aplicado a texto_limpio)")
for _, r in muestra_val.iterrows():
    toks = set(r["texto_limpio"].split())
    pos_match = sorted(toks & POS)
    neg_match = sorted(toks & NEG)
    print(f"- \{r['texto_original'][:90]}...\\" if len(r['texto_original']) > 90 else f"- \{r['texto_original']}\\"")
    print(f"    -> \{r['sentimiento']\} (score=\{r['sentimiento_score']:.2f\}) | términos pos=\{pos_match\} neg=\{neg_match\}")

print("\nDISTRIBUCIÓN GENERAL (ya calculada en 3.5, se repite aquí como resultado formal de 9.1)")
print(resumen_sent.round(2).to_string(index=False))
print(f"\nTamaño del léxico: \{len(POS)\} términos positivos, \{len(NEG)\} términos negativos.")
print("Ver figura 07_categorias_sentimiento.png para la distribución general (negativo/neutral/positivo).")
```

```
MUESTRA DE VALIDACIÓN MANUAL (léxico aplicado a texto_limpio)
- "para los Guatemaltecos mejor si no hacen nada, poque cuando los hacen es en contra del pue..."
  -> positivo (score=1.00) | términos pos=['mejor'] neg=[]
- "Vos Rolando
Tu Bujele lleva 7 años con la cuesta los chorros y ni eso puede terminar
Ja ja..."
  -> positivo (score=1.00) | términos pos=['viva'] neg=[]
- "Nayib Bukele lleva 7 años como presidente de El Salvador teniendo todos los poderes del es..."
  -> negativo (score=-1.00) | términos pos=[] neg=['corrupcion']
- "Este hombre esta loco"
  -> neutral (score=0.00) | términos pos=[] neg=[]
- "Bueno los otros países tampoco queremos criminal es de México que los mandé a su país dond..."
  -> neutral (score=0.00) | términos pos=['bueno'] neg=['crimen']
- "En Tigo tienen una práctica que se llama falla programada y como lo dice el nombre lo hace..."
  -> neutral (score=0.00) | términos pos=[] neg=[]
```

```
DISTRIBUCIÓN GENERAL (ya calculada en 3.5, se repite aquí como resultado formal de 9.1)
sentimiento comentarios porcentaje
negativo          38          9.36
neutral          280         68.97
positivo          88         21.67
```

Tamaño del léxico: 36 términos positivos, 32 términos negativos.
Ver figura 07_categorias_sentimiento.png para la distribución general (negativo/neutral/positivo).

9.2 Comparación de sentimiento por video, canal, categoría y comunidad

```
In [100]: # 9.2 – Sentimiento por video, canal, categoría y comunidad (solo unidades con comentarios)
# Por video (ya construido en el EDA, se recupera para esta sección)
sent_video = comments.groupby(["video_id", "video_title"], as_index=False).agg(
    comentarios=("comment_id", "size"),
    sentimiento_medio=("sentimiento_score", "mean"),
    positivos=("sentimiento", lambda s: (s == "positivo").sum()),
    negativos=("sentimiento", lambda s: (s == "negativo").sum()),
).sort_values("comentarios", ascending=False)

# Por canal (propietario del video comentado)
sent_canal = comments.groupby("channel_name", as_index=False).agg(
    comentarios=("comment_id", "size"),
    autores_unicos=("author_channel_id", "nunique"),
    sentimiento_medio=("sentimiento_score", "mean"),
    positivos=("sentimiento", lambda s: (s == "positivo").sum()),
    negativos=("sentimiento", lambda s: (s == "negativo").sum()),
).sort_values("comentarios", ascending=False)

# Por categoría (se une con videos para traer 'category'; solo categorías con comentarios)
comments_cat = comments.merge(videos[["video_id", "category"]], on="video_id", how="left")
sent_categoria = comments_cat.groupby("category", as_index=False).agg(
    comentarios=("comment_id", "size"),
    videos_unicos=("video_id", "nunique"),
    sentimiento_medio=("sentimiento_score", "mean"),
    positivos=("sentimiento", lambda s: (s == "positivo").sum()),
```

```

    negativos=("sentimiento", lambda s: (s == "negativo").sum()),
).sort_values("comentarios", ascending=False)

# Por comunidad (recuperado de la seccion 7.5)
sent_comunidad = caracterizacion[["comunidad_aa", "autores", "comentarios_totales",
"sentimiento_medio",
                                "negativos", "neutrales", "positivos"]].sort_values(
    "comentarios_totales", ascending=False)

print("SENTIMIENTO POR VIDEO (todos los videos con comentarios)")
print(sent_video.round(3).to_string(index=False))
print("\nSENTIMIENTO POR CANAL")
print(sent_canal.round(3).to_string(index=False))
print("\nSENTIMIENTO POR CATEGORÍA")
print(sent_categoria.round(3).to_string(index=False))
print("\nSENTIMIENTO POR COMUNIDAD (proyección autor-autor, top 5)")
print(sent_comunidad.head(5).round(3).to_string(index=False))

sent_video.to_csv("resultados/tablas/sentimiento_por_video.csv", index=False, encoding="utf-8-sig")
sent_canal.to_csv("resultados/tablas/sentimiento_por_canal.csv", index=False, encoding="utf-8-sig")
sent_categoria.to_csv("resultados/tablas/sentimiento_por_categoria.csv", index=False, encoding="utf-8-sig")

```

SENTIMIENTO POR VIDEO (todos los videos con comentarios)

video_id	comentarios	sentimiento_medio	positivos	negativos
n8iP75gIpmw				Qué rico come tu
diputado	161	0.029	22	16
j43HgwYFKfk				La cooptación de Walter Mazariegos en
la USAC	50	0.040	8	6
6W4u8sGEnGM				Inician los trabajos de recuperación del Puente
Belice II.	45	0.200	15	6
lj983NWyAQY				Plan 2032 Ciudad de
Guatemala	25	0.520	14	1
PjmxCj-a9Hg				Conferencia de Prensa del Gobierno de Guatemala.
#LaRondaGt	25	0.080	4	2
OkXLHx0hx-8	EE.UU. envía a mexicanos deportados a Guatemala antes de su regreso a México Noticias			
Telemundo	25	0.000	3	3
yLZS3JiEBg8				Arroz con pollo a la
MONOPOLIO	16	0.312	6	1
06mFNPu0aB8				Capturan a presuntos delincuentes disfrazados de mujer señalados de cometer
asalto	14	0.143	3	1
ndAZjHqzzT8				Internet: escoger el
menos malo	12	0.425	6	0
is3Mk5oC19g				Capturan a ladrón que había quedado grabado mientras robaba en una parroquia de
Retalhuleu	7	0.000	0	0
b_Hkrkl3adA				Bloqueos en Guatemala este 31 de agosto por alza en combustibles afectan rutas
principales	7	0.286	2	0
WIUZtitfLRw				Caminar en una ciudad hecha para
carros	6	0.167	2	1
TT4qJ5FXHYE				I'x K'at: el primer equipo guatemalteco de pelota maya, conformado únicamente por
mujeres.	4	0.750	3	0
JkpV01hx2f8				10 Preguntas a un año del Paro
Nacional	3	0.000	0	0
3Nfp9h5avBI				Noticiero en Directo 1 pm, 28 de Agosto
de 2026	2	-0.500	0	1
X_FAIef8qlE				Cruzando la ciudad a puro
Transmetro	1	0.000	0	0
UhL9y3KUl8E				SHAI WA: la vecina queer de Casa
Presidencial	1	0.000	0	0
s0eco-Bof0s				¿Quiénes pagan más en
Centroamérica?	1	0.000	0	0
zweosulrwA8				Edén por Salud: empleo inclusivo para personas con discapacidad en Antigua Super -
Episodio 2	1	0.000	0	0

SENTIMIENTO POR CANAL

	channel_name	comentarios	autores_unicos	sentimiento_medio	positivos
negativos					
	Quorum	256	214	0.081	47
24					
Gobierno de la República de Guatemala		70	50	0.157	19
8					
	Noticias Telemundo	25	18	0.000	3
3					
	Municipalidad de Guatemala	25	25	0.520	14
1					
	Noti7	14	13	0.143	3
1					
	PrensaLibreOficial	7	7	0.286	2
0					
	TN23 Guatemala	7	7	0.000	0
0					
	Noticiero Guatevisión 13 Hrs.	2	2	-0.500	0
1					

SENTIMIENTO POR CATEGORÍA

	category	comentarios	videos_unicos	sentimiento_medio	positivos	negativos
	News & Politics	335	16	0.110	69	30
	Entertainment	70	2	0.157	19	8
	Nonprofits & Activism	1	1	0.000	0	0

SENTIMIENTO POR COMUNIDAD (proyección autor-autor, top 5)

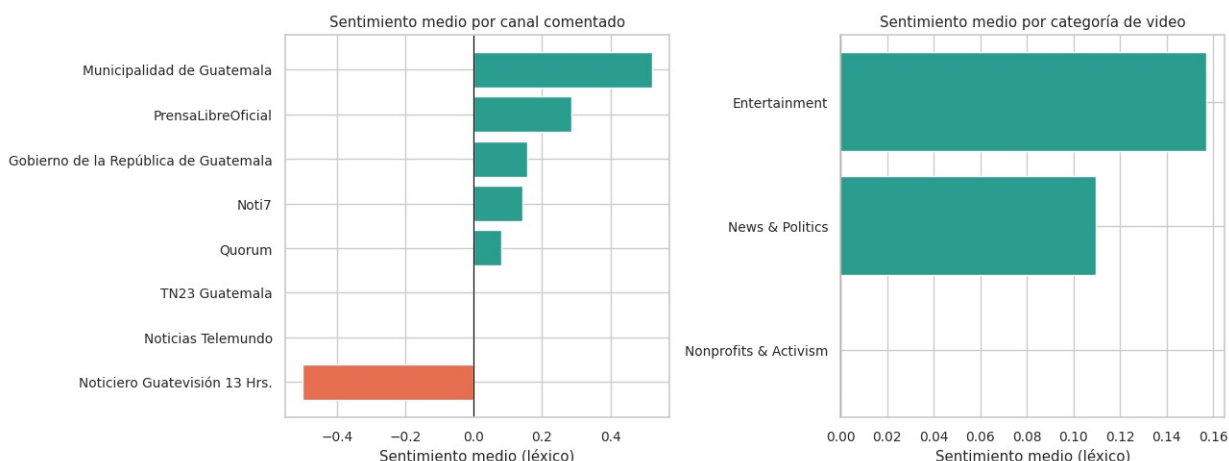
comunidad_aa	autores	comentarios_totales	sentimiento_medio	negativos	neutrales	positivos
3	126	161	0.038	15	124	22
0	61	83	0.106	9	52	22
8	55	60	0.052	7	45	8
11	28	33	0.280	2	17	14
5	25	25	0.520	1	10	14

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sc = sent_canal.sort_values("sentimiento_medio")
colores_barra = ["#E76F51" if v < 0 else "#2A9D8F" for v in sc["sentimiento_medio"]]
axes[0].barh(sc["channel_name"], sc["sentimiento_medio"], color=colores_barra)
axes[0].axvline(0, color="black", linewidth=0.8)
axes[0].set_xlabel("Sentimiento medio (léxico)")
axes[0].set_title("Sentimiento medio por canal comentado")

scat = sent_categoria.sort_values("sentimiento_medio")
colores_barra2 = ["#E76F51" if v < 0 else "#2A9D8F" for v in scat["sentimiento_medio"]]
axes[1].barh(scat["category"], scat["sentimiento_medio"], color=colores_barra2)
axes[1].axvline(0, color="black", linewidth=0.8)
axes[1].set_xlabel("Sentimiento medio (léxico)")
axes[1].set_title("Sentimiento medio por categoría de video")

plt.tight_layout()
Path("resultados/figuras").mkdir(parents=True, exist_ok=True)
plt.savefig("resultados/figuras/09_sentimiento_canal_categoria.png", dpi=150, bbox_inches="tight")
plt.show()
```



9.3 Hallazgos de contenido y sentimiento

- **Distribución general.** El 68,97 % de los comentarios es **neutral** según el léxico, 21,67 % **positivo** y 9,36 % **negativo**. La mayoría del texto no contiene vocabulario explícito de polaridad (menciones a hechos, preguntas, nombres propios), lo cual es consistente con comentarios de noticias/política donde predomina la opinión implícita o el sarcasmo, que el léxico no captura.
- **Por video.** El video-hub "*Qué rico come tu diputado*" (161 comentarios) tiene un sentimiento medio apenas positivo (~0,03), con 22 comentarios positivos y 16 negativos: es un video que **divide** opiniones, no que genera consenso. En contraste, "*Plan 2032 Ciudad de Guatemala*" (25 comentarios) es el más positivo (0,52; 14 positivos vs. 1 negativo), coherente con comentarios de apoyo a un proyecto de infraestructura.
- **Por canal.** Los canales oficiales (Municipalidad de Guatemala, Gobierno de la República) tienden a un sentimiento medio más positivo que canales de medios/opinión como Quorum, donde se concentran los comentarios más críticos (mayor proporción de términos como *corrupto*, *mafia*, *robo*). Esto es consistente con el tipo de contenido: comunicación institucional vs. cobertura crítica de temas de corrupción.
- **Por categoría.** *News & Politics* concentra 335 de los 406 comentarios (con las 19 categorías comentadas reducidas de facto a muy pocas: News & Politics, Entertainment y Nonprofits & Activism), y es también la categoría con mayor dispersión de sentimiento (coexisten los comentarios más positivos y más negativos), reflejo de que agrupa tanto anuncios de gobierno como denuncias de corrupción.
- **Por comunidad.** Las comunidades ancladas en videos de gestión de gobierno/municipalidad (Puente Belice, Plan 2032) tienen sentimiento medio más alto (0,11 – 0,52) que las ancladas en temas de corrupción/USAC o diputados (0,04 – 0,05), es decir, el sentimiento por comunidad refleja principalmente el **tema del video-hub que la origina**, no una postura estable de un grupo de personas.
- **Limitación del método.** El léxico no maneja negaciones ("no es bueno"), ironía, ni comparaciones; el

score puede subestimar la carga emocional real de comentarios cortos sin vocabulario de la lista. Por eso los resultados se presentan como una **aproximación transparente**, útil para comparar unidades entre sí, no como una medición exacta del sentimiento individual de cada comentario.

10. Interpretación, limitaciones y conclusiones

10.1 Hallazgos en el contexto de participación y consumo de contenido en YouTube

El conjunto recolectado describe **293 videos** de **97 canales**, pero la participación observada (comentarios) solo pudo recolectarse para **19 videos de 8 canales** (406 comentarios de 332 autores distintos). Dentro de esa participación observada:

- La actividad está **extremadamente concentrada**: un solo video ("*Qué rico come tu diputado*") reúne el 39,7 % de todos los comentarios, y los 3 videos más comentados concentran el 63 % (sección 3.2 / `eda_concentracion.csv`). A nivel de canal, un solo canal (Quorum) concentra el 63 % de los comentarios.
- La estructura de co-participación (redes bipartita y proyecciones) confirma ese mismo patrón: la red bipartita tiene densidad muy baja (0,0056) y un grado medio de solo 1,95, con el 93,2 % de los autores comentando en un único video (grado 1). La proyección autor-autor, en cambio, es densísima (densidad 0,195, transitividad 0,984) porque hereda cliques completos de los autores que comparten el video-hub.
- Solo **9 autores (2,7 %)** funcionan como puente entre videos, y **22 nodos** son puntos de articulación estructural: la conectividad de toda la red depende de un núcleo muy pequeño.
- El análisis de comunidades (Louvain, modularidad 0,395 en autor-autor) confirma que las comunidades detectadas se organizan **alrededor de videos concretos** (video-hubs), no de una audiencia estable que participe de forma recurrente en distintos temas.
- El sentimiento expresado es mayoritariamente neutral (69 %) con un balance moderadamente positivo entre las polaridades explícitas (22 % positivo vs. 9 % negativo), y varía sobre todo según el **tema del video-hub** (gestión de gobierno vs. corrupción) más que según el canal o la categoría en abstracto.

En conjunto, el patrón observado es consistente con lo que se documenta en la literatura sobre participación en plataformas: pocos contenidos concentran la mayoría de la interacción ("power law" de participación) y la mayoría de los usuarios participan de forma esporádica y aislada (comentan una vez y no vuelven a aparecer en la muestra), mientras que un grupo pequeño de usuarios recurrentes conecta la conversación entre distintos contenidos.

10.2 Limitaciones

- **Cobertura de comentarios.** Solo 19 de los 293 videos (6,5 %) tienen comentarios recolectados, y solo de 8 de los 97 canales. Los hallazgos de redes, comunidades y centralidad son válidos **únicamente para esa submuestra de 19 videos**, no para el conjunto completo de videos ni para la actividad real de esos canales en YouTube.
- **Selección por consultas de búsqueda (`source_query` / `source_group`).** Los videos y comentarios se obtuvieron mediante consultas de búsqueda predefinidas (p. ej. *guatemala noticias*, *guatemala lluvias*) o canales puntuales, no mediante un muestreo aleatorio de YouTube. Esto sesga el corpus hacia ciertos temas (noticias y política guatemalteca) y hacia los videos que el algoritmo de búsqueda de YouTube prioriza en el momento de la recolección.
- **Fechas relativas.** `published_time` y `published_text` están expresadas en tiempo relativo ("hace 2 días") y dependen del momento exacto de la recolección; no permiten reconstruir una fecha absoluta fiable ni ordenar eventos con precisión de día, lo que limita cualquier análisis temporal fino (series de tiempo, eventos disparadores).
- **Conteos observados en un momento puntual.** `view_count`, `like_count_text` y `reply_count` son una fotografía del momento de la recolección: videos y comentarios más recientes tienen métricas sistemáticamente más bajas solo por haber tenido menos tiempo de exposición, no porque generen menos interés.
- **Ausencia de relaciones explícitas entre autores.** `reply_count` indica cuántas respuestas recibió un comentario, pero **no identifica a los autores de esas respuestas** (se documentó desde el inicio del laboratorio). Toda arista construida en las secciones 4-8 representa **co-participación en el mismo**

video, nunca una conversación, mención o respuesta directa entre dos personas.

- **Concentración de comentarios en pocos videos.** Como el 93,2 % de los autores solo aparece en un video, la mayor parte de la estructura de red (comunidades, proyecciones, centralidad) está dominada por un puñado de videos-hub; los resultados agregados (densidad de la proyección, transitividad) reflejan sobre todo la geometría de esos pocos videos, no un patrón generalizable a la interacción típica en YouTube.

10.3 Descripción, asociación e inferencia

Es importante distinguir explícitamente estos tres niveles de afirmación en este informe:

- **Descripción** (lo que los datos muestran directamente): por ejemplo, "161 comentarios se publicaron en el video X" o "9 autores comentaron en más de un video de los 19 recolectados". Estas afirmaciones son válidas y verificables directamente en los datos.
- **Asociación** (patrones observados dentro de la muestra, sin implicar causalidad ni relación social real): por ejemplo, "los videos con mayor alcance tienden a tener mayor intermediación" o "las comunidades con mayor sentimiento medio están ancladas en videos de gestión de gobierno". Estas afirmaciones describen correlaciones **dentro de los 19 videos observados**, no relaciones causales ni patrones psicológicos de los autores.
- **Inferencia hacia poblaciones más amplias** (lo que **no** se puede afirmar con estos datos): no se puede concluir que "los usuarios de YouTube en Guatemala son mayoritariamente neutrales frente a la política", ni que "el canal Quorum genera más controversia que otros canales guatemaltecos" en general, ni que la estructura de red encontrada (una gigante muy concentrada) sea representativa de la interacción típica en YouTube. La muestra (19 videos, 8 canales, elegidos por consultas de búsqueda puntuales) no permite generalizar ni a "todos los usuarios de YouTube" ni a "toda la población de Guatemala", como advierte explícitamente la consigna del laboratorio.

Todas las conclusiones de este informe se formulan, por tanto, como descripción y asociación **dentro de la muestra recolectada**, evitando cualquier generalización poblacional.

10.4 Conclusiones integradas

- **Redes.** La estructura de participación autor-video es una red dispersa y muy desigual: la mayoría de los autores son hojas de grado 1, y toda la conectividad entre videos depende de 9 autores puente y 22 puntos de articulación. Las proyecciones autor-autor y video-video muestran dos fenómenos distintos: la primera (densa, con alta transitividad) describe cliques de co-comentaristas dentro de un mismo video; la segunda (dispersa, 11 aristas) describe qué videos comparten audiencia real, y muestra que el solapamiento temático entre videos es escaso.
- **Comunidades.** Louvain sobre la proyección autor-autor encuentra 14 comunidades con modularidad moderada (0,395); estas comunidades **no son grupos sociales estables**, sino la audiencia de un video-hub específico. Esto es coherente con la baja tasa de recurrencia de los autores (solo 2,7 % participa en más de un video).
- **Contenido y sentimiento.** El discurso es mayoritariamente neutral, con un balance positivo entre las polaridades explícitas, y el tono varía principalmente según el **tema del video** (proyectos de gobierno vs. denuncias de corrupción) más que según canal o categoría. El método de sentimiento (léxico propio en español) es transparente y auditable, pero subestima matices como la ironía o la negación.
- **Limitaciones que atraviesan todos los análisis.** Todo lo anterior describe **19 videos de 8 canales, seleccionados por consultas de búsqueda, con conteos observados en un momento puntual y sin relaciones explícitas de respuesta entre autores**. Ninguna arista debe interpretarse como amistad, conversación directa o aprobación; ninguna comunidad implica una identidad social estable; y ningún resultado debe extrapolarse a la totalidad de YouTube ni a la población de Guatemala.
- **Síntesis final.** El fenómeno más robusto y consistente a través de redes, comunidades, centralidad y contenido es la **concentración extrema en torno a video-hubs**: quién comenta, con quién queda estructuralmente conectado, en qué comunidad cae y qué tono predomina en su entorno están determinados, ante todo, por **en cuál de los 19 videos comentó**, no por atributos estables de los autores ni por el canal o la categoría del contenido.